

Processors

Processors - הגנת סייבר

תיכון אלון רמת השרון
מוגש על ידי אורי אלפי
ת.ז: 326461944
תאריך ההגשה: 21.5.25

תוכן העניינים

2	תוכן העניינים
3	פרק 1 - מבוא
3	ייזום
4	פירוט תיאור המערכת (אפיון)
7	פרק 2 - תיאור תחום הידע
7	פירוט היכולות
11	פרק 3 - מבנה הפרויקט
11	תיאור ארכיטקטורת המערכת
11	תיאור הטכנולוגיות הרלוונטיות
12	תיאור זרימת המידע
16	אלגוריתמים מרכזיים
19	תיאור סביבת פיתוח
19	פירוט כלי הפיתוח הנדרשים לפיתוח
19	כלים הנדרשים לבדיקות
19	תיאור פרוטוקול התקשורת
19	תיאור מילולי של פרוטוקול התקשורת
20	פירוט כלל ההודעות הזורמות במערכת
22	תיאור מסכי המערכת
26	תרשים מסכים
27	תיאור מבני הנתונים
27	סקירת חולשות ואיומים
27	שכבת האפליקציה
28	שכבת התעבורה
29	פרק 4 - מימוש
29	מודולים / מחלקות מיובאים
30	מחלקות שאני הכנתי
41	קטעי קוד האלגוריתמים המרכזיים
53	בדיקות
55	מדריך למשתמש
55	עץ קבצים
55	התקנת המערכת
55	משתמשי המערכת
59	רפלקציה
61	ביבלוגרפיה
62	נספחים

פרק 1 - מבוא

ייזום

תיאור הפרויקט: מערכת שבה עובדים שומרים על יעילות ובטיחות של שרת מסויים ומונעים חדירה של Processים לא רצויים מרחוק. בחרתי בפרויקט זה כי רציתי להרחיב את הידע שלי על מערכת ההפעלה, ועל processים כי זה נושא שעניין אותי. רציתי לעשות משהו הגנתי אז בחרתי בנושא זה. האתגרים העיקרים שאני חושש מהם בביצוע הפרויקט זה שלא אצליח למחוק Processים מרחוק, בגלל בעיות גישה בפייתון. עוד חשש שאני חושב שאצטרך להתמודד איתו זה שאני יכול להרוס בטעות את מערכת ההפעלה שלי במחיקת Processים הלא נכונים.

הגדרת הלקוח: הלקוח הינו אדם או חברה פרטית שרוצים לשמור על השרת/המחשב שלהם מ-Processים עוינים ולא יעילים. מי שהולך להשתמש במערכת עצמה זה העובדים והמנהל של החברה.

הגדרת יעדים/מטרות: המטרות העיקריות הינן ליצור מערכת של חברה עם עובדים בעלי הרשאות שונים ומחשבים שונים(עבודה מרחוק). על המערכת לשמור יעילות ובטיחות של שרת שעליו החברה עובדת.

בעיות תועלות וחסכונות: הבעיה המרכזית היא הפעלה של Processים לא רצויים על שרת של חברה, אשר עלולים לפגוע ביעילות העבודה או בביטחון המחשב. מה שאנחנו מנסים להשיג בפרויקט הזה זה לזהות Processים לא רצויים או עוינים על שרת או מחשב ולאפשר לעובדים לפקח על תהליכים מרחוק. התועלות הינן שיפור באבטחת המידע, שמירה על יציבות השרת, ועבודה מרחוק בקונספט של חברה תחרותית בין עובדים עם מנהל חברה שמפקח אודות מעשי העובדים. השירותים שמערכת תיתן הן זיהוי תהליכים חשודים או חריגים ברמת מערכת ההפעלה, ומחיקתם/זיהוי שלהם על ידי עובדים. בנוסף לכך, יהיה ממשק ניהול פעולות משתמשים ופרטים שלהם, שרק למשתמשי מנהל יש גישה אליו.

סקירת פתרונות קיימים: בשוק קיימים פתרונות לניהול תהליכים ואבטחת מערכות. מערכות כמו Task Manager (ב-Windows) או htop (בלינוקס) מאפשרות לצפות בתהליכים הרצים ולמחוק אותם, אך הן אינן מציעות פתרון נוח לבקרה מרחוק. כלים מתקדמים יותר כמו Ansible, Puppet או Nagios מאפשרים בקרה של מערכות מרוחקות, אך הם לרוב מורכבים להתקנה ודורשים ידע, שלא כמו במערכת שלי שהחברה עושה את הבקרה בשבילך עם עובדים. בתחום אבטחת המידע קיימות מערכות אנטי-וירוס וחומות אש (כגון Windows Defender, McAfee, Fortinet), אך מטרתן שונה – הן מגינות על המערכת ברמה של קבצים, תוכנות ותקשורת, ולא עוסקות בהפסקה של תהליכים. המערכת שאני מפתח שונה. היא מאפשרת שליטה מרחוק על Processים, כולל מחיקה שלהם ומתמקדת בפתרון קל יותר בהרבה שמתאים לפרטים או חברות קטנות שלא מעוניינות במערכות מורכבות.

סקירת טכנולוגית הפרויקט: הטכנולוגיה שבה משתמש הפרויקט אינה חדשה, אך היא משלבת מספר רכיבים קיימים בצורה מעניינת. אחת מהספריות המרכזיות בפרויקט היא psutil – ספרייה שמאפשרת גישה למידע על תהליכים, שימוש בזיכרון, CPU, ופעולות מערכת נוספות. למרות ש-psutil עצמה מוכרת, השימוש בה לזיהוי ומחיקת תהליכים לא רצויים מרחוק, ובשילוב עם הקונספט של חברה של עובדים שמבצעים בקרה, הופך את הפרויקט למעניין.

האם קיימים סייגים בהגדרת המערכת?: קיימים סייגים והגבלות שיכולים להשפיע על המערכת. ראשית, פעולת מחיקת Processים מרחוק עלולה להיות מוגבלת עקב בעיות הרשאה במחשב – במיוחד כאשר התהליך רץ תחת משתמש בעל הרשאות גבוהות עוד יותר (כמו root או

(administrator). בנוסף לכך, יש עוד סייג שהינו הסיכון שקשור למחיקת תהליכים לא נכונים, דבר שעלול להביא לקריסת שירותים חשובים במערכת ההפעלה. כמו כן, המערכת תלויה בספריה חיצונית psutil, כך שכל עדכון או בעיה בספריה עלול להשפיע על התוכנה. לבסוף, יש לקחת בחשבון שגם העובדים במערכת חייבים לפעול בזירות מרבית, כי פעולה לא נכונה עשויה לגרום לנזק בלתי הפיך בשרת ובמערכת. למרות זאת, למערכת ההפעלה יש חסמים למה שהיא מאפשרת, ולא תאפשר מחיקה של רוב התהליכים החיוניים.

תיחום הפרויקט: הפרויקט עוסק בעיקר בשני תחומים מרכזיים: מערכות הפעלה ורשתות מחשבים, בדגש על ניהול תהליכים, ורשתות. בצד של מערכות ההפעלה, המערכת מתמקדת בזיהוי, קריאה ומחיקת תהליכים שרצים במערכת ההפעלה באמצעות הספריה psutil. בנוסף, השרת מרובה משתמשים משתמש ב multi-threading בשביל לקבל כמה לקוחות במכה. בתחום הרשתות, הפרויקט עושה שימוש ב socket לצורך בניית שרת מרובה משתמשים בפרוטוקול TCP. כמובן שכל השירותים מונגשים בעזרת ממשק משתמש נוח להבנה. עם זאת, המערכת אינה מטפלת בנושאים כמו תקשורת מתקדמת יותר (כמו פרוטוקולי אבטחה), ואינה עוסקת בניהול תשתיות רשת כגון ניתוב ושירותי DNS. כמו כן, הפרויקט אינו מטפל בהשגחה של משאבי חומרה או טיפול בקבצים חשודים, אלא מתמקד בפתרון נקודתי של ניהול תהליכים.

פירוט תיאור המערכת (אפיון)

תיאור מפורט של המערכת – מה היא אמורה לעשות: המערכת מספקת תקשורת מבוססת ספריית Socket בין משתמשים מרובים לבין שרת מרכזי. יש ממשק משתמש ברור לקריאה שמקושר ללקוח. המערכת מציגה לעובד את כל 15 התהליכים שהכי כבדים בשימוש במעבד לעובד ולפי התעדוף של התהליך והשימוש בזכרון ככה העובד מחליט אם למחוק את התהליך או להשאירו. המנהל רואה שהעובד מחק ב Log, וככה יודע מי עשה מה כדי לקדם אותו בחברה, לקדם את שכרו, ומשגיח שתהליכים שבאמת לא נחוצים או מזיקים נמחקים. אבטחת המידע בפרויקט הינה ההצפנה בתקשורת עם RSA ואבטחת סיסמאות עם hash שכולל salt, והגנה מפני sql injection.

יכולת של כל משתמש:

1. כניסה למשתמש שלו באמצעות שם משתמש וסיסמה
2. עבודה מרחוק- כל משתמש יכול לעבוד מכל מחשב שבו המערכת מותקנת באותה רשת
3. לעבוד בממשק משתמש נקי- כל משתמש (מנהל/עובד) מונגש לו העבודה בממשק משתמש נקי.

משתמש מסוג מנהל:

1. לראות את כל המחיקות וסיבת המחיקה, כולל משאבי התהליך. על כל תהליך שנמחק רשום את שם העובד המוחק ותאריך ושעת המחיקה.
2. הוספת משתמשים למערכת באמצעות נתינת פרטי העובד חדש.
3. שינוי סיסמה/שכר/שם משתמש/תפקיד של עובד.
4. לתת העלאה/הורדת שכר לעובד.
5. רשימת כל המשתמשים, עם אפשרות חיפוש עובד לפי שם המשתמש שלהם.
6. פיטורי עובד שגורמים למחיקה של פרטיו מהמערכת, וניתוק חיבורו לשרת.

משתמש מסוג עובד:

אורי אלפי | Proccesors

1. לראות את כל 15 processes שהכי משתמשים במעבד, ולראות את השימוש שלהם במעבד ובזיכרון ובנוסף את התעדוף שלהם במערכת ההפעלה
2. למחוק תהליכים על השרת בהינתן סיבה רשומה.

בדיקות:

בשביל לבצע בדיקה של אי דליפת מידע בעת ההעברה של הודעות בין שרת ללקוח, אדפיס את ההודעות ואשווה ביניהם. המערכת מצפינה כל הודעה בהצפנת RSA, שהיא הצפנה בטוחה מאוד שמתבססת על לוגיקה מספרית.

בDatabases, המערכת תשתמש בפונקציות hashing כדי לאבטח את הסיסמאות של המשתמשים, ובנוסף תוסיף salt, מקדם לסיסמה שמוסיפים אותו בשביל לחזק את האבטחה, שימנע את האפשרות של הפורצים להכנסת סיסמאות נפוצות. כדי לבדוק את ההצפנות אדפיס ואשווה בשרת ובלקוח.

המערכת מתגוננת מפני תקיפת injection sql על ידי כך שתמנע הוספה של סימנים מיוחדים שמשפיעים על Database או חלקי קוד לתוך Databases. באמצעות חלוקת הקלט לבסיס נתונים לפרמטרים, ככה זה מונע הכנסת קוד ברוב המקרים. באמצעות פונקציה, הפרמטרים עוברים בדיקה של סימנים מיוחדים רגישים. פונקציה זאת מחזקת את האבטחה עוד יותר. אנסה להכניס קוד לתוך הסיסמא ואותיות מיוחדות ואראה אם זה יתן לי.

המערכת מיישמת הגנה מפני תקיפה DDOS שמציפה את השרת עם המון לקוחות כדי להקריסו. המערכת תמנע את זה בכך שהיא תגביל את מספר הלקוחות בשרת ותאפשר התחברות רק אם השרת לא מלא. כדי לבדוק אם זה עובד אשלח מספר משתמשים גבוה יותר מהמותר, שאותו הגדרתי להיות ממש נמוך לצורך הבדיקה, ואראה אם זה יתן לי להתחבר.

בדיקת תקינות ההתחברות, שינוי הפרטים של עובד, פיטורי עובד, והוספת עובד(חיבור לDatabase): אצור מספר משתמשים שונים ואנסה להתחבר למערכת עם השם משתמש וסיסמה שלהם. אחר כך, יתחבר לחשבון שמוגדר מנהל ואראה בממשק משתמש את הפרטים שלהם. אחר כך, אפטר אחד מהם. אבדוק אם הוא נותק, ושמו לא מופיע בממשק משתמש. אשנה את השם משתמש של אחד מהם, ואגרום לו לנסות להתחבר עם השם משתמש הישן. אם הוא מצליח, הבסיס נתונים באמת מחובר לתוכנה.

ברגע שהמשתמש סגר את התוכנה שלו, אראה שבאמת נשלח Disconnected בשרת, שבאמת התנתק מהשרת.

אפתח תוכנה בשרת שמשתמשת במעבד באופן כבד, ואמחוק אותה במשתמש עובד ממחשב רחוק. אם זה עובד, והתוכנה נסגרת, זה אומר שיש מספיק הרשאות לתוכנה, והמחיקה עובדת כראוי.

אפתח תוכנה בשרת שמשתמשת במעבד באופן כבד, ואראה אם המשאבים שהמערכת שלי מראה לי בערך תואמים למה שרשום בTask manager על התוכנה. ככה אבדוק אם המערכת שלי צודקת לגבי הממצאים שלה על התהליכים.

תכנון וניהול לוז הזמנים:

אורי אלפי | Proccesors

- תכנון הסיפריות במערכת והחיבור הלוגי שלהם באמצעות עקרונות הפיתוח SOLID, ובשימוש הורשה.
- בניית מערכת תקשורת שרת-לקוח מרובת משתמשים.
- יצירת דף התחברות בצד הלקוח באמצעות Costum Tkinter.
- חיבור התקשורת לדף ההתחברות.
- יצירת Database לשמירת פרטי המשתתפים והסיסמאות/שמות משתמש שלהם.
- מימוש המחלקות של העובדים בצד השרת להעברת מידע ללקוחות.
- פתיחת המחלקה על התהליכים שמשיגה את המידע על כל התהליכים בשרת, שהמחלקה של העובד שולחת ללקוח.
- יצירת המסך הראשי של העובד ויצירת כפתורים עבור כלל הפעולות בממשק הגרפי.
- חיבור הממשק משתמש לרשתות בצד העובד.
- בניית מחלקת ההצפנה RSA עבור השרת והלקוח, ושימוש במחלקה.
- הגנה מפני החדרת קוד לבסיס הנתונים, עם salti hash.
- בניית הממשק הגרפי של המנהל.
- הוספה למחלקת המנהל את החיבור לבסיס הנתונים.
- חיבור רשתות לממשק הגרפי של המנהל מצד הלקוח.
- פתיחת מחלקת Log.
- הוספה למחלקות העובדים (Worker) רשימה של Log.
- הוספה למחלקת העובד את השינוי לרשימה של Log במחיקה של תהליך.
- הגנה מפני הצפת השרת במשתמשים המערכת התקשורת.
- סגירת הממשק משתמש בניתוק השרת.
- סגירת החיבור ברגע שעובד פוטר.
- תחילת העבודה על ספר הפרויקט.

הלוח הזמנים בפועל:

- תכנון הסיפריות במערכת והחיבור הלוגי שלהם באמצעות עקרונות הפיתוח SOLID, ובשימוש הורשה. **בסוף לא הקפדתי על שימוש בעקרונות SOLID ואני חושב לתקן את זה לפני הבחינה.**
- בניית מערכת תקשורת שרת-לקוח מרובת משתמשים. **בניתי את השרת בו זמנית שבניתי את DB, לא כמו בתכנון. בנוסף כתבתי את הGUI בו זמנית.**
- יצירת Database לשמירת פרטי המשתתפים והסיסמאות/שמות משתמש שלהם.
- יצירת דף התחברות בצד הלקוח באמצעות Costum Tkinter.
- חיבור התקשורת לדף ההתחברות.
- מימוש המחלקות של העובדים בצד השרת להעברת מידע ללקוחות. **לאורך כל הפרויקט הבנתי מה אני צריך להשים במחלקות האלו אז מימוש המחלקות היה דבר שעשיתי כל הפרויקט והיה משתנה כל הזמן.**
- יצירת המסך הראשי של העובד ויצירת כפתורים עבור כלל הפעולות בממשק הגרפי. **עשיתי את זה לפני שלמדתי על איך להוציא את המידע על תהליכים, לא כמו בתכנון.**
- פתיחת המחלקה על התהליכים שמשיגה את המידע על כל התהליכים בשרת, שהמחלקה של העובד שולחת ללקוח.
- חיבור הממשק משתמש לרשתות בצד העובד.
- בניית מחלקת ההצפנה RSA עבור השרת והלקוח, ושימוש במחלקה.

אורי אלפי | Proccesors

- הגנה מפני החדרת קוד לבסיס הנתונים, עם salti hash. **קודם כל עשיתי Hash, ואז שכחתי מההצפנת RSA עד לסוף הפרויקט.**
- בניית הממשק הגרפי של המנהל.
- הוספה למחלקת המנהל את החיבור לבסיס הנתונים.
- חיבור רשתות לממשק הגרפי של המנהל מצד הלקוח. **עשיתי את זה בו זמנית שכתבתי את המחלקה של המנהל.**
- פתיחת מחלקת Log.
- הוספה למחלקות העובדים (Worker) רשימה של Log.
- הוספה למחלקת העובד את השינוי לרשימה של Log במחיקה של תהליך.
- הגנה מפני הצפת השרת במשתמשים המערכת התקשורת. **שכחתי מזה אז עשיתי את זה בסוף הפרויקט.**
- סגירת הממשק משתמש בניתוק השרת.
- סגירת החיבור ברגע שעובד פוטר.
- תחילת העבודה על ספר הפרויקט. **עשיתי את הפרויקט בזמן שעשיתי את הדברים האחרונים בפרויקט.**

ניהול הסיכונים בפרויקט והדרכים להתמודדות איתם:

הסיכון הראשון שאני יכול לחשוב עליו הינו החלק של ההרשאות. אני מפחד שלא אצליח למחוק תהליכים דרך הפייתון כי המחשב לא יתן לי גישה לזה. דרך ההתמודדות שלי עם זה הינה לדבר עם המורים והמנחה שלי שיעזרו לי לשנות ההגדרות בתוכנת ההרצה של הקוד. מה שקרה בפועל זה שלתוכנת הרצה שלי היה מספיק הרשאות.

הסיכון שאני חושש ממנו הכי הרבה הינו לכתוב את החלק של הרשתות, עם השרת והלקוח. זה משהו שאני עדיין חדש לו, ויש לו מסובכות מאוד גדולה. אני כן מבין את התאוריה באופן מופתי, וכן כתבתי שרתים אבל לא בהיקף הזה. שוב, הדרך התמודדות שלי היא לדבר עם המורה שלי לרשתות שהוא גם המנחה שלי לעזרה בעת הצורך ולאבא שלי שהוא מתכנת בעיסוקו בעצמו. מה שקרה בפועל זה שהצלחתי לכתוב את החלק בעצמי ושזה יעבוד אבל נתקלתי בכמה בעיות שטיפלתי בעזרת המורים הנהדרים שלי, ובעזרת אבא שלי שעזר לי.

הסיכון האחרון שאני מפחד ממנו הוא שאני בטעות אמחק תהליכים חיוניים בעשיית הפרויקט, הדרך שאני אתמודד עם זה היא בכך שאפתח Virtual Machine, ורק שם אעבוד על הפרויקט. בפועל, למדתי דרך המורה שלי על כך שהמחשב לא נותן לתוכנות למחוק תהליכים חיוניים לרוב, ושיש לו מגננות נגד זה. בנוסף, הוספתי עוד חלון נוסף בשביל לוודא עם העובד שהוא מחק את התהליך הנכון, ולא משהו שבטעות נלחץ לו. זה מגביר את הביטחון שלי נגד הסיכון שלקחתי בפרויקט.

פרק 2 - תיאור תחום הידע פירוט היכולות

היכולת: התחברות למערכת

מהות: כניסת משתמש קיים למערכת

אורי אלפי | Proccesors

אוסף יכולות נדרשות:

- ממשק משתמש – מסך הרשמה
- קליטת נתונים
- שליחה לשרת אחרי hash בלקוח לסיסמא
- בדיקת תקינות בשרת נגד sql injection בשרת בחלוקה לפרמטרים
- הצפנה בHash + הוספת Salt בשרת
- authentication מול בסיס הנתונים בשרת
- הצפנת התשובה בשרת
- שליחת התשובה מהשרת *עד הבגרות אוסיף הצפנה גם מהלקוח לשרת
- פענוח התשובה בלקוח
- הצגת תשובה למשתמש בעזרת ממשק המשתמש
- סגירת הממשק משתמש כראוי
- שליחה לשרת בקשה לתפקיד של המשתמש
- קבלה בשרת
- קבלה בלקוח
- הכוונה למסך הבא הנכון לפי התפקיד

אובייקטים נחוצים: LoginScreen, Worker, Guest, Server, Client

היכולת: לראות את התהליכים בשרת

מהות: העובד רואה את התהליך שנמצא על המחשב של השרת ואת משאביו.

אוסף יכולות נדרשות:

- ממשק משתמש שמכיל רשימה של תהליכים עם אפשרות מחיקה.
- שליחת בקשה לקבל את המידע על התהליכים בstream מהשרת
- פענוח של השרת של הבקשה
- קבלת כל התהליכים בשרת בלולאה שפעולת כל 6 שניות
- סינון התהליכים לפי ה15 הכי שמשתמשים בזכרון
- הפיכתם לjson
- הצפנה בשרת
- לשלוח ללקוח כל כמה שניות
- פענוח של התהליכים בלקוח והצגתם בממשק משתמש קריא

אובייקטים נחוצים: EmployeeScreen, Worker, Employee, Process, Server, Client

היכולת: למחוק את התהליכים בשרת ושמירתם בLogs

מהות: העובד רואה את התהליך שנמצא על המחשב של השרת ואת משאביו.

אוסף יכולות נדרשות: *הhash מתבצע בדרך לא טובה, אתקן בגרסה בבגרות

- ממשק משתמש שמכיל כפתורים על תהליכים למחיקה, ומסך שמוודא כפליים אם המשתמש רוצה למחוק.

אורי אלפי | Proccesors

- קליטה מהמשתמש למה הוא רוצה למחוק
- שליחת בקשה לשרת
- קליטת הבקשה בשרת
- מחיקה לפי הזיהוי שנשלח
- לעשות Logs lock שלא שני משתמשים ישתמשו בו זמנית ויגרום לקריסה
- הוספה לרשימת Logs בשרת
- הצפנת Log החדש
- שליחה לכל המנהלים את Log החדש
- פענוח בלקוח

אובייקטים נחוצים: ManagerScreen, Log, Logs, Worker, Employee, Manager, Server, Client

היכולת: לראות את Logs, התהליכים שנמחקו על ידי העובדים

מהות: לראות את הנעשה על ידי העובדים

אוסף יכולות נדרשות:

- ממשק משתמש שהוא רשימה של תהליכים מחוקים עם המשאבים שלהם
- הצפנה ושליחה של Logs מהשרת
- פענוח בלקוח
- הצגה בGui

אובייקטים נחוצים: ManagerScreen, Manager, Logs, Log, Server, Client

היכולת: לקבל רשימה של העובדים מעודכנת

מהות: המנהל יכול לראות את כל פרטי העובדים המעודכנים

אוסף יכולות נדרשות:

- ממשק משתמש שהוא רשימה של העובדים עם הפרטים שלהם ואפשרות חיפוש עובד לפי שם
- השרת מקבל מהבסיס נתונים את רשימת כל העובדים עם הפרטים שלהם
- השרת מצפין ושולח ללקוח
- פענוח בלקוח
- הצגה בGui, אם לא נמחק העובד הוא לא מוחק את הWidget בשביל יעילות, אלא רק משנה את הפרטים שלו. אם נוסף עובד הוא רק מוסיף עוד Widget.

אובייקטים נחוצים: ManagerScreen, Manager, Server, Client, Database

היכולת: לשנות את פרטי העובדים בכל דרך שהיא

מהות: לשנות את פרטי העובדים שזה כולל בתוכו את שכרם, שם משתמש שלהם או סיסמא, תפקיד.

אורי אלפי | Proccesors

אוסף יכולות נדרשות:

- ממשק משתמש שהוא רשימה של עובדים עם אפשרות חיפוש לפי שם, ועל רכיבי הרשימה יש כפתורים לשינוי פרטי העובד
- הלקוח שולח בקשה לשינוי פרט
- השרת מקבל
- השרת משנה את הפרטים בDatabases לפי מה שהלקוח ביקש
- לפני שהוא משנה את הפרטים, הוא מחלק לפרמטרים את הערכים שלהם שהם לא SQL injection
- אם הוא מכניס סיסמא חדשה, הוא עושה לHash עם Salt הקיים
- הצגה בGui בגלל שהשרת כל 5 שניות שולח את רשימת העובדים מחדש.

אובייקטים נחוצים: ManagerScreen, Manager, Database, Server, Client

*יש לציין שאין צורך בthread.lock בעת שימוש בdatabases בשני תהליכים בו זמנית כי למד נתונים יש את זה באופן מובנה.

היכולת: הוספת עובד

מהות: הוספת עובד חדש לחברה בקבלת הפרטים שלו על ידי המנהל.

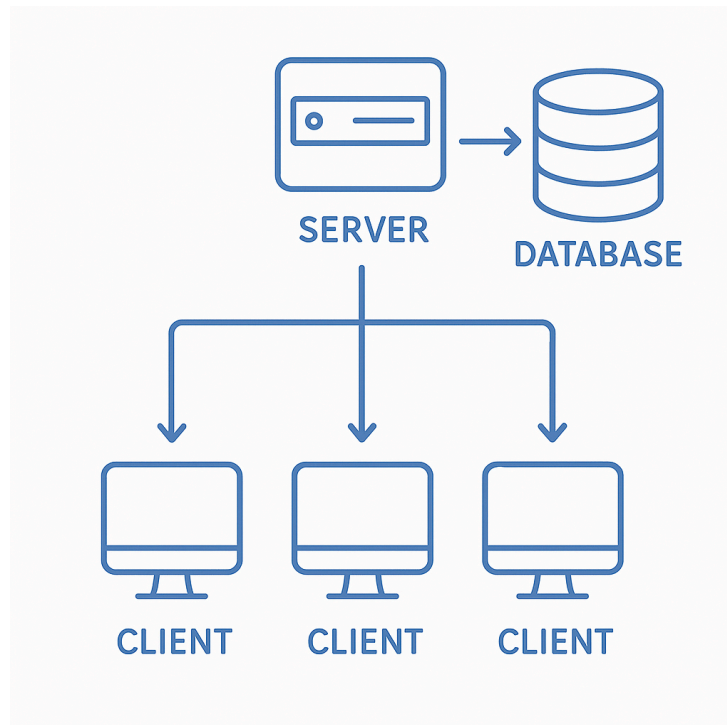
אוסף יכולות נדרשות:

- ממשק משתמש שיש לו כפתור הוספת עובדים שפותח חלון להכנסת פרטים
- הלקוח שולח את פרטי העובד החדש בבקשה להוסיף אותו לשרת
- השרת מקבל
- השרת מוסיף את העובד בDatabases לפי מה שהלקוח ביקש
- הצגה בGui בגלל שהשרת כל 5 שניות שולח את רשימת העובדים מחדש.

אובייקטים נחוצים: ManagerScreen, Manager, Database, Server, Client

פרק 3 - מבנה הפרויקט

תיאור ארכיטקטורת המערכת



שרת: מאזין לבקשות מהלקוחות כמו התחברות למערכת, מחיקת תהליכים, שינוי פרטי משתמש וכול. שולח באופן תמידי פקטות לשרת כמו רשימת העובדים ורשימת התהליכים. השרת מקשר את הלקוחות למסד הנתונים, ובכך שהלקוחות מבקשים לשינוי במסד נתונים הוא עושה זאת. על המחשב שרץ עליו השרת להיות עם מעבד טוב כדי שיוכל לטפל ביעילות מיטבית בכל המידע שמגיע אליו מהלקוחות. בנוסף לכך, על השרת להיות מחובר לרשת אינטרנט מהירה וטובה, בשביל שליחה של מידע באופן מיטבי

לקוח: מחשב שמתחבר לשרת ושולח ומקבל נתונים ממנו. הוא שולח פקודות והשרת מבצע אותם ולפעמים מחזיר תשובה. כשהוא מחזיר תשובה, הלקוח צריך לקבל את התשובה בכל רגע ולהיות מוכן לכך. בנוסף לכך, הלקוח מקבל כל כמה זמן מהשרת מידע ועליו תמיד להיות מוכן אליו. דיסק קשיח: עליו יש את בסיס הנתונים.

תיאור הטכנולוגיות הרלוונטיות

שפת תכנות פייתון: פייתון היא שפת תכנות high level, פשוטה ללמידה וקריאה, מה שהופך אותה לפופולרית. אחד היתרונות המרכזיים של פייתון הוא המאגר העצום של ספריות חיצוניות, שמספרות את היעילות בפיתוח פרויקטים מורכבים. בפרויקטים גדולים בתחום הסייבר, ספריות כמו psutil לדוגמא עוזרות: הספרייה מאפשרת אינטראקציה עם מערכת ההפעלה ברמה נמוכה – לדוגמא, מעקב אחר תהליכים, ניטור שימוש בזיכרון או CPU, ואיסוף מידע על דיסקים ותקשורת. מבנה נתונים SQL: בסיס נתונים בSQL היא מערכת לאחסון, ניהול ושליפה של מידע בצורה מאורגנת ומבנית. המידע נשמר בטבלאות שמורכבות משורות (רשומות) ועמודות (שדות). השפה מאפשרת שליטה מדויקת על המידע ומבוססת על עקרונות של תקינות, ייחודיות וקשרים בין טבלאות, מה שהופך אותה לכלי מרכזי בתשתיות מידע של ארגונים, אתרים ואפליקציות בכל תחום.

מערכת ההפעלה Windows 11: בעת כתיבת פרויקט במערכת ההפעלה Windows 11, מפתחים פועלים בסביבה נוחה לעבודה עם התאמות שמטרתן לייעל את העבודה. המערכת כוללת תמיכה מובנית בכלים נפוצים כמו PowerShell ו-Visual Studio. פרוטוקול תקשורת TCP: (ראשי תיבות של Transmission Control Protocol) פרוטוקול תקשורת נפוץ ואמין שפועל בשכבת התעבורה במודל ה-TCP/IP. הוא מאפשר העברת מידע בצורה רציפה, מאובטחת ומסודרת בין שני מחשבים ברשת, תוך הבטחה שכל המידע שנשלח אכן יגיע ליעדו ובסדר הנכון. TCP משתמש במנגנונים של אישור קבלה (ACK), בקרת זרימה ובקרת שגיאות כדי להבטיח את שלמות הנתונים.

הצפנת RSA: שיטת הצפנה א-סימטרית המבוססת על שימוש בזוג מפתחות. מפתח ציבורי להצפנה ומפתח פרטי לפענוח. השיטה מאפשרת לשדר מידע מוצפן בצורה בטוחה גם בערוצים לא מאובטחים, שכן רק מי שמחזיק במפתח הפרטי יכול לפענח את המידע שהוצפן בעזרת המפתח הציבורי.

תהליך יצירת המפתחות נעשה כך:

1. בוחרים שני מספרים ראשוניים גדולים: p ו- q

2. מחשבים את n :

$$n = p * q$$

3. מחשבים את פונקציית אוילר:

$$\phi_n = (p - 1) * (q - 1)$$

4. בוחרים מספר e כך שהוא זר ל- ϕ_n

5. מחשבים את d כך שיתקיים:

$$d * e \equiv 1 \text{ mod } \phi_n$$

לאחר יצירת המפתחות, הצפנת הודעה m נעשית באמצעות:

$$c = (m ** e) \% n$$

ופענוח ההודעה נעשה באמצעות:

$$m = (c ** d) \% n$$

RSA משמשת גם לחתימות דיגיטליות, בהן חותמים על הודעה בעזרת המפתח הפרטי, וכל אחד יכול לאמת את החתימה באמצעות המפתח הציבורי. השיטה בטוחה כל עוד לא ניתן לפרק את n לגורמיו (כלומר, לחשב את p ו- q), אך בשל הביצועים האיטיים שלה ביחס לשיטות הצפנה סימטריות, היא משמשת לרוב להצפנת מפתחות ולא להצפנת תוכן מלא.

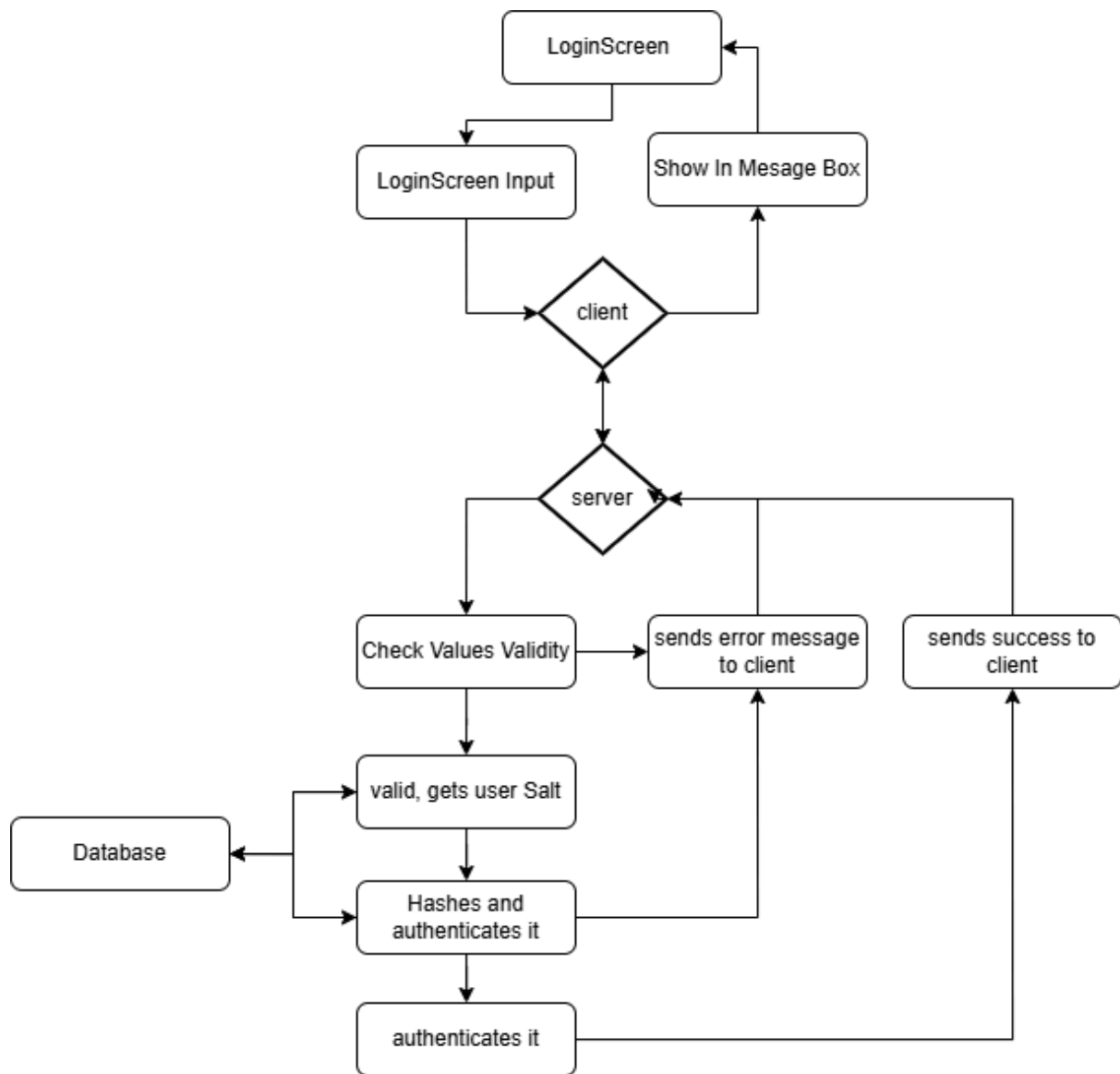
הצפנת Hash: הצפנת Hash היא תהליך חד-כיווני שבו ממירים מידע (כגון סיסמה) למחרוזת באורך קבוע בעזרת פונקציית גיבוב (Hash Function), לדוגמה SHA-256. בניגוד להצפנה רגילה, את הפלט של פונקציית Hash לא ניתן לפענח חזרה למידע המקורי – המטרה היא רק לאמת שהקלט תואם לערך הידוע. לכן היא משמשת בעיקר לאחסון סיסמאות בצורה בטוחה. עם זאת, כדי להקשות על תוקפים שמנסים לנחש סיסמאות באמצעות השוואה לטבלת Hash מוכנות (כמו Rainbow Tables), מוסיפים לכל סיסמה ערך אקראי ייחודי שנקרא salt. השימוש ב-salt מבטיח שגם אם שני משתמשים בחרו אותה סיסמה, הפלט של פונקציית ה-Hash יהיה שונה, מה שמגביר את רמת האבטחה ומונע התקפות מבוססות על ערכים חוזרים או ידועים.

JSON: (ראשי תיבות של JavaScript Object Notation) פורמט קל וקריא לאדם ולאדם מכונה לייצוג ואחסון של מידע במבנה של מפתח-ערך. הוא מבוסס על תחביר של שפת JavaScript, אך משמש כמעט בכל שפות התכנות בזכות הפשטות והיעילות שלו.

תיאור זרימת המידע

היכולת: להיכנס לחשבון

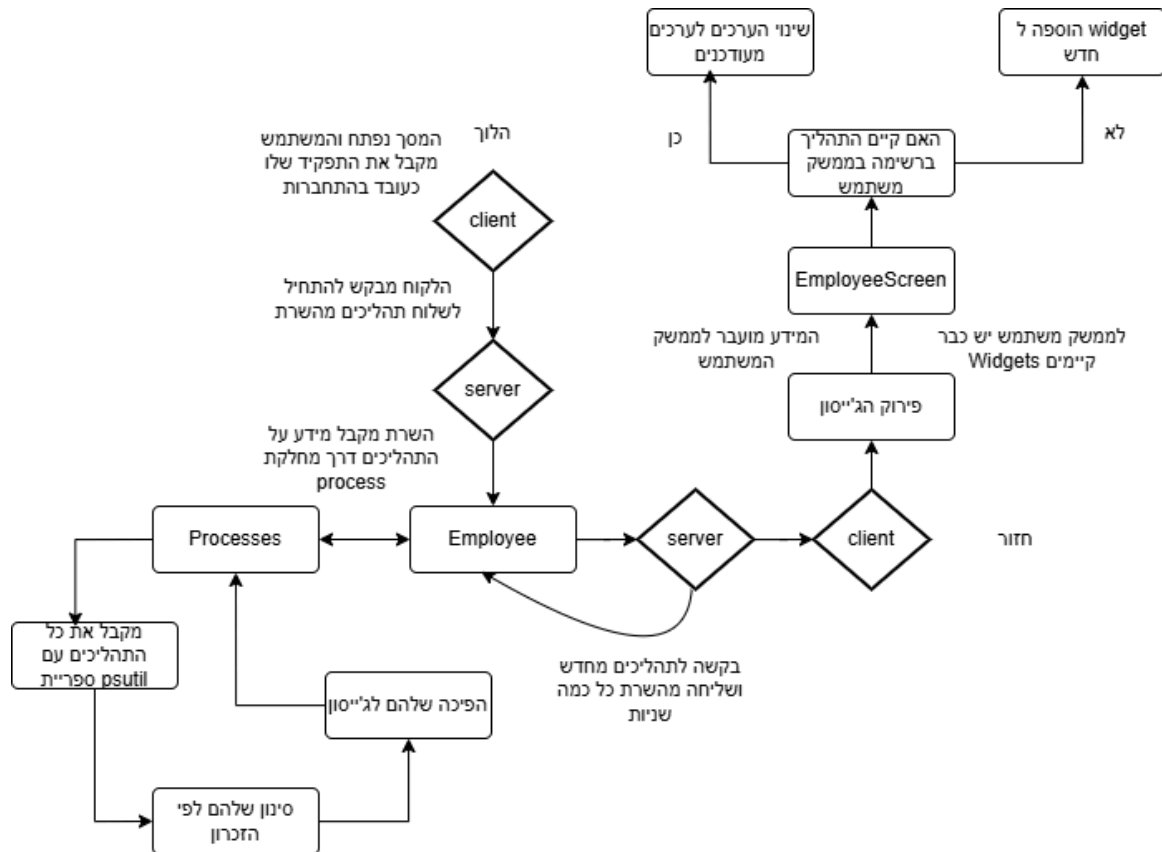
אורי אלפי | Proccesors



*כמובן שגם הלקוח מקבל תפקיד ולפי זה משתייך אבל זה לא חלק מהותי בהרשמה.
*בכל התרשימים יש הצפנה בשרת ופענוח בלקוח.

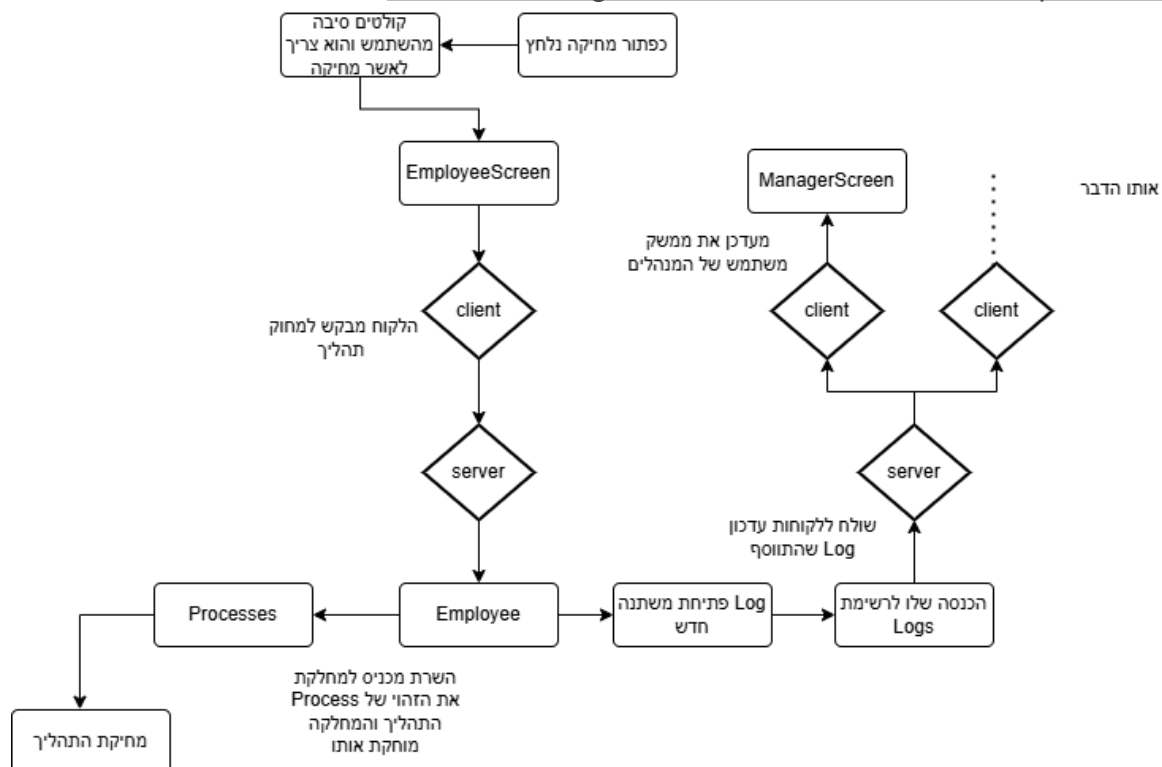
היכולת: לראות את התהליכים בשרת

אורי אלפי | Proccesors



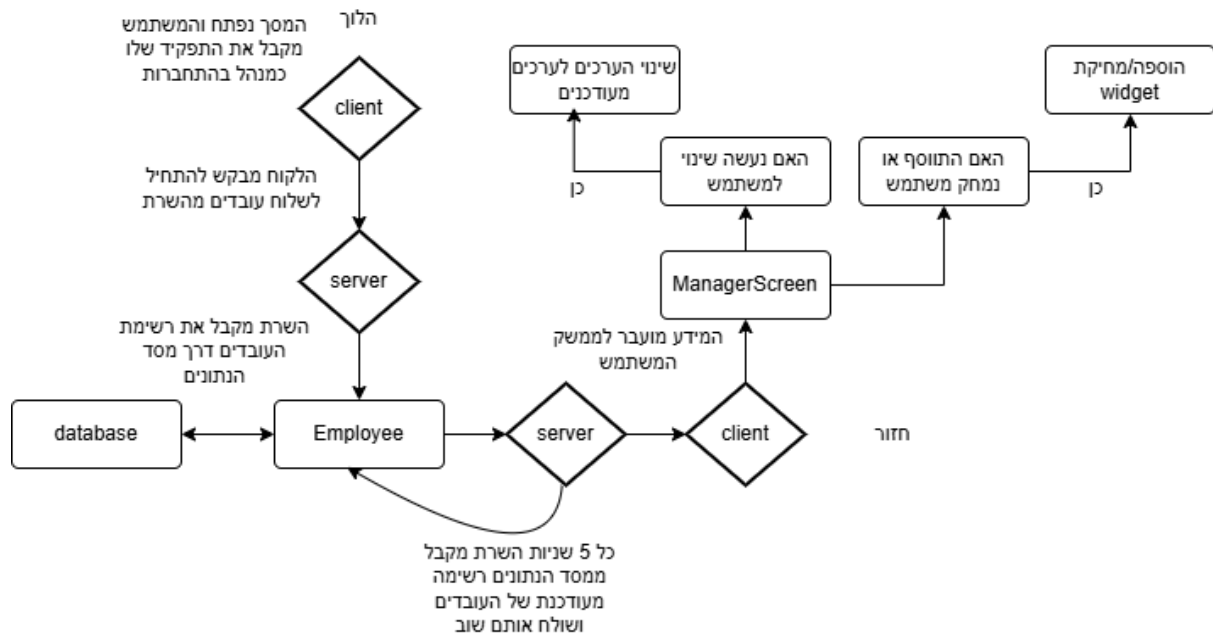
הלקוח בהתחלה מבקש מהשרת את התהליכים פעם אחת, אז השרת שולח ללקוח את כל התהליכים כל כמה זמן.

היכולת: למחוק את התהליכים בשרת ושמירתם בLogs ולראות אותם

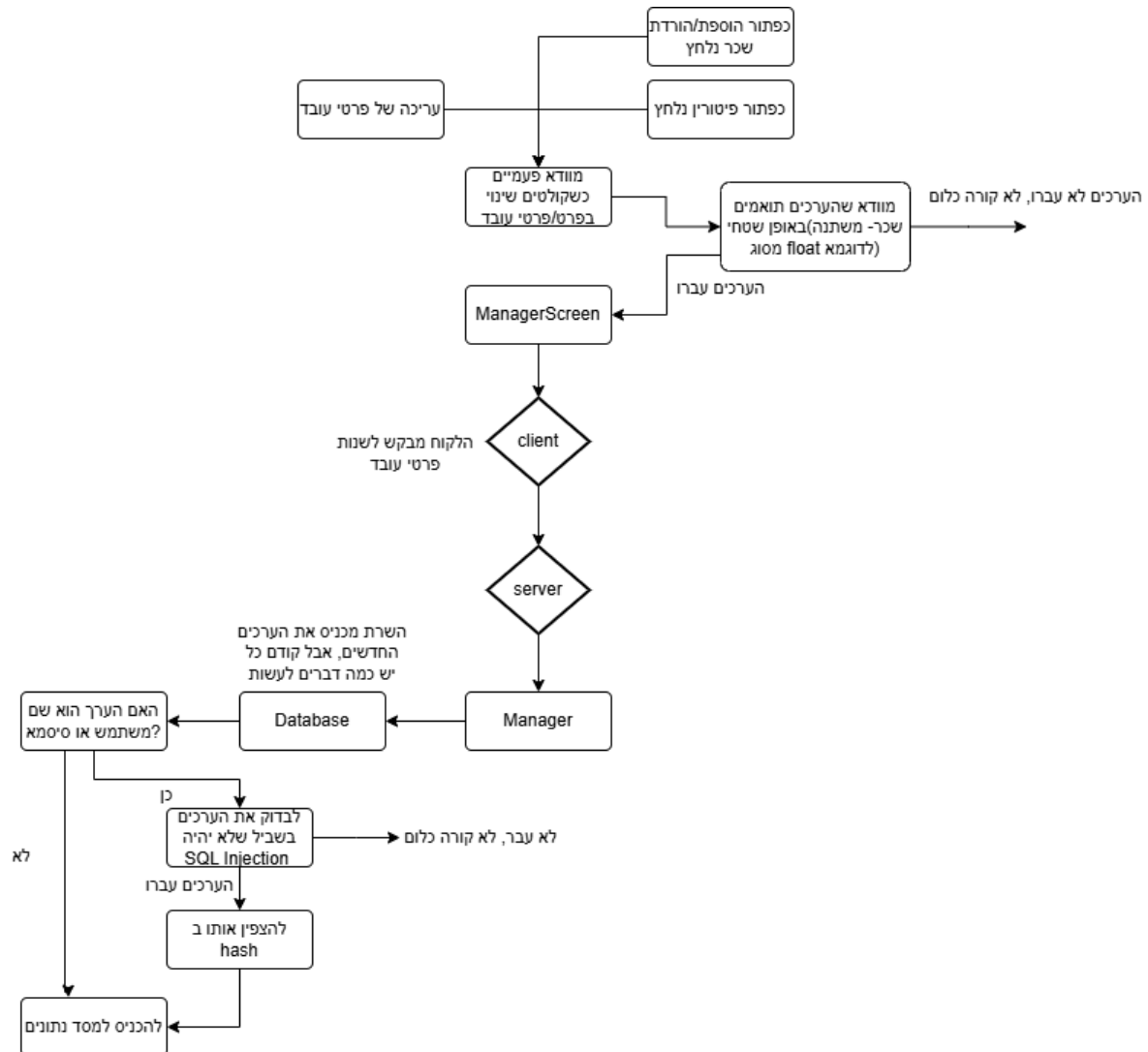


היכולת: לקבל רשימת עובדים מעודכנת

אורי אלפי | Proccesors

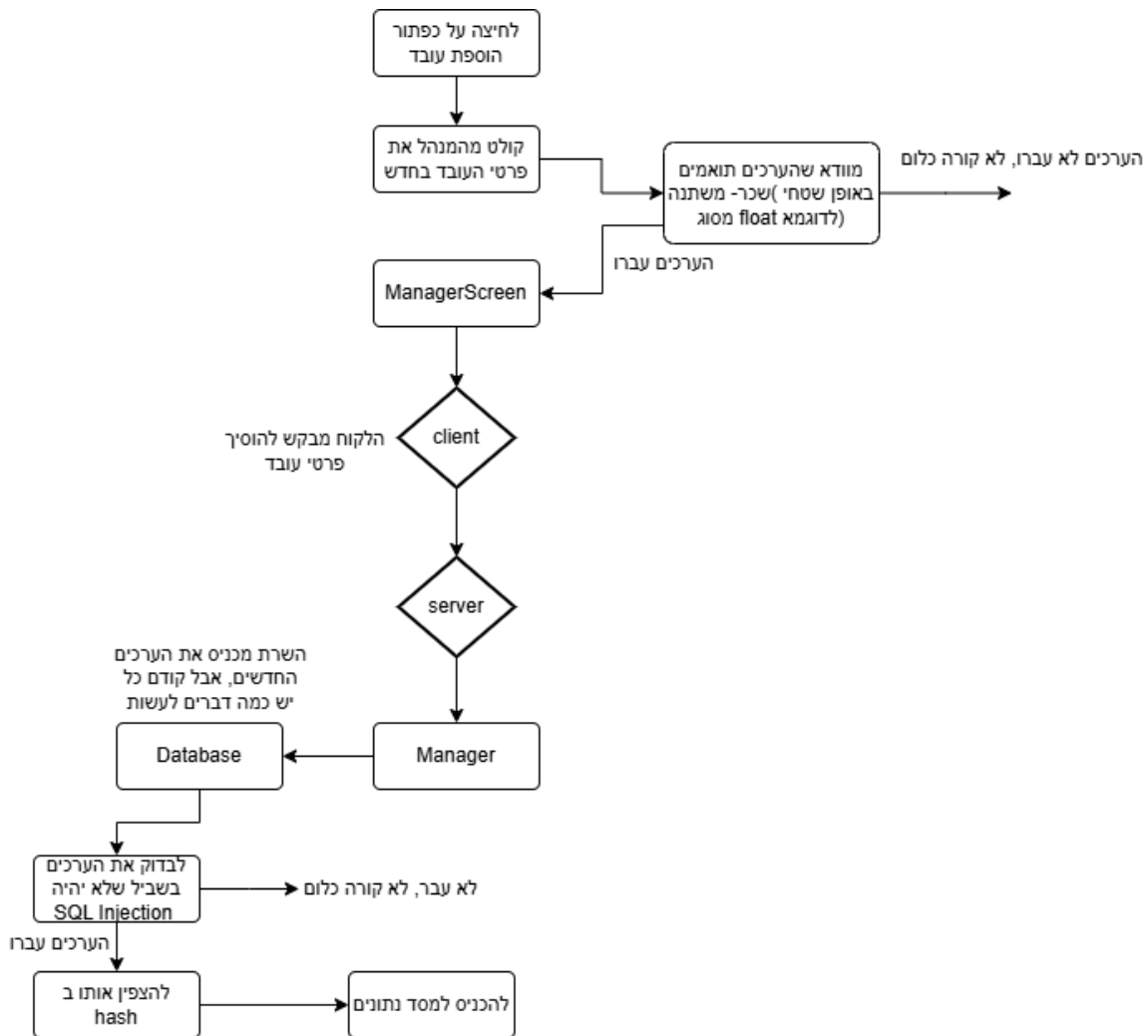


היכולת: לשנות את פרטי העובד בכל דרך שהיא



אורי אלפי | Proccesors

*בהתחלת הפרויקט חשבתי שאפשר שינוי סיסמה אבל הורדתי את זה, ולכן רואים את זה בתרשים כאפשרות.
היכולת: להוסיף עובד



*רק הסיסמא מוצפנת ב Hash והיא מוצפנת עם Salt.

אלגוריתמים מרכזיים

כל משתמש יכול לבצע רק את הפקודות המותרות לו, ויש לו מסך בהתאם לתפקיד שלו
במערכת שפיתחתי קיימים 3 תפקידים: אורח(מצב לפני ההתחברות), עובד ומנהל. כל אחד מהם מקבל פיצ'רים שונים ומסך משתמש שונה.

האתגר המרכזי שהתמודדתי איתו היה כיצד לחלק את ההרשאות בצורה מסודרת, בעקרונות הפיתוח של SOLID(בעיקר עקרון Open-Closed שאומר שהמערכת פתוחה להוספות וסגורה לשינויים) וכזו שתמנע את האפשרות שלקוח לא מורשה יכול להריץ פקודה שאינה שלו.

קודם כל, אדגיש שלכל משתמש רשום את תפקידו במסד נתונים.
הפתרונות לבעיה זאת:

1. הגישה הראשונה והפשוטה ביותר היא לבצע בדיקה ידנית של ההרשאות בכל פעם שפקודה מתקבלת – לדוגמה, לבדוק אם `role == Role.MANAGER` ורק אז להריץ את הפקודה. הבעיה בגישה זו היא קוד מסורבל, עם תנאים רבים שקל מאוד לשכוח אותם בעת כתיבת קוד בעתיד. זה יוצר קוד שלא עובד לפי עקרון Open-Closed באופן תחליטי.
2. גישה אחרת היא שימוש במפה של הרשאות – לדוגמה מילון שמגדיר אילו תפקידים רשאים להפעיל כל פיצ'ר. גם זו גישה שיכולה לעבוד, אך היא דורשת טיפול דינמי בכל קריאת פקודה, ובפועל רק מחליפה את התנאים באוסף חוקים חיצוניים שעדיין צריך לזכור לבדוק כל הזמן.
3. פתרונות נוספים, כמו שימוש במערכת הרשאות חיצונית (RBAC), מתאימים יותר למערכות מבוססות HTTP כמו Flask, ואינם ברי ביצוע בפרויקט שלי שמבוסס על socket ותקשורת מרוחקת. [מקור](#)

בגלל כל הדברים שצינתי, בחרתי בגישה של תכנות מונחה עצמים (OOP) ולפי עקרונות הפיתוח (SOLID): יצרתי היררכיה של מחלקות לפי סוג המשתמש – `Manager`, `Guest`, `Employee`, כאשר כולן יורשות ממחלקת-על `Worker`. כל מחלקה מגדירה בעצמה את הפקודות שהיא יודעת לטפל בהן, בתוך הפונקציה `handle_command`, כך שאם למשל עובד מנסה לשלוח פקודה שמיועדת רק למנהל – המערכת לא תבין את הפקודה שהוא שלח. אין צורך לבדוק בכל פעם "מי המשתמש", כי כבר בשלב החיבור לכל לקוח מותאם האובייקט המתאים לו. הפתרון הזה משתלב עם מבנה של המערכת שלי המלאת מחלקות, מבטל כמעט לחלוטין את הצורך בתנאים ובדיקות, ומאפשר תחזוקה פשוטה והרחבה קלה של תפקידים בעתיד שעלולה לקרות בחברה (למשל, הוספת תפקיד נוסף תדרוש רק יצירת מחלקה חדשה, ולא מלא בדיקות נוספות מסורבלות וקשות למעקב).

הצגת תהליכים ללקוחות

- הפיצ'ר המרכזי בפרויקט שלי שעליו כל הפרויקט מתבסס זה היכולת של המשתמש (העובד) לצפות ברשימה של תהליכים שרצים על השרת ב-Live, בעוד שהם מתעדכנים, כמובן עם תיאום של הממשק הגרפי. יש צורך לאסוף מידע שמתעדכן כל הזמן, להעביר אותו דרך socket בצורה מסודרת, ולהציגו באופן חלק שהלקוח לא יקרוס. זה דורש סינון נתונים ועיבוד גרפי יעיל וחכם, ומערכת רשתות חכמה. בעיות משניות:
1. הלקוח רק יכול לקבל 1024 בתים וכל התהליכים עלולים להיות יותר מ-1024 בתים.
 2. איך אפשר להעביר תהליכים? זה לא טיפוס מוגדר של משהו.
 3. איך אפשר להציג כל כך הרבה Widget בtkinter בו זמנית ושהיו כל הזמן משתנים?

פתרונות לבעיה

1. הלקוח כל כמה שניות שולח בקשה מהשרת לרשימת התהליכים, והשרת שולח אותה. זה יוצר עומס שלא צריך אותו על הרשת כשיש הרבה לקוחות ובקשות.
2. שימוש בWebSocket. גישה זאת היא פתרון טוב, אבל דורשת המון למידה עצמית. אני מעדיף לעשות את התקשורת באמצעות Socket. [מקור](#)

פתרונות לבעיות המשניות שהשתמשתי

1. אומנם הלקוח מקבל רק 1024 בתים, אבל ברגע שזה חורג מזה הוא פשוט מקבל עוד פקטה. לכן, השתמשתי בheader בתחילת הפקטה שאומר את אורכה בשביל שהלקוח ידע כמה הוא צריך לפרק את הפקטה לכמה פקטות. זה עדיף מהפתרון שכל פעם שולחים פקטה שגודלה 1024 או קטן מזה עד לסיום חיבור כל הפקטות ביחד. זה הרבה יותר יעיל לעשות בדרך שאני בחרתי ומסודר.

2. להעביר עצמים או סוג אחר של משתנים זה לא בעיה, בעיקר בשימוש בספריות כמו JSON שמשפוטות עצמים וכול.. אפשר גם להשתמש בstring אבל זה יותר נהוג להשתמש בJSON לדברים הללו.
3. במקום לפתוח כל פעם widget חדש, פתחתי את כל הwidgetים ושמתי להם ערכים ריקים. רק שבא המידע עם התהליכים, שיניתי את הערכים על כל הפקטות. אם מתווסף למספר התהליכים עוד, זה מוסיף Widget אבל זה לא אמור לקרות. זה רק בשביל לשמור על יציבות התוכנה ואם רוצים לשנות את כמות התהליכים שרואים בעתיד.

הפתרון שלי הוא שאחרי שליחת הודעה מהלקוח שאומרת לשרת להתחיל, השרת מריץ Thread עבור כל עובד(לקוח שמשוך למחלקת Employee), וכל כמה שניות הוא שולח ללקוח את רשימת התהליכים. כדי למנוע שליחה של כמויות מידע גדולים מידי ובכך למנוע קריסה, בחרתי לסנן בצד השרת ולשלוח רק את 15 התהליכים עם צריכת המעבד הגבוהה ביותר. בחרתי את הפתרון הזה כי הוא יותר יעיל מהפתרון הראשון שרשמתי, והוא יותר פשוט בשבילי לבצע אותו מהפתרון השני שכתבתי.

*ראוי לציין שגם קבלת רשימת העובדים מהשרת ללקוח(לקוח מסוג manager) דומה באלגוריתם של הבעיה הזאת.

הצגת Logs למנהלים

- מנהלים יכולים לצפות בכל התהליכים שנמחקו, ופרטים לגבי המחיקה כמן סיבה ומי מחק. במילים אחרות, כל מחיקה מוצגת בצורה מסודרת בממשק הגרפי של המנהל. הבעיה האלגוריתמית היא כיצד להעביר מידע זה בצורה מיטיבה מבחינת יעילות. כל מנהל יקבל את העדכון הזה בזמן אמת ברגע שתהליך נמחק, וממשק המשתמש שלו יתעדכן מיידית. בעיות משניות:
1. כמה עובדים ימחקו באותו הזמן ויגשו לLogs באותו זמן והספרייה תקרוס/יקרה ביצוע פעולות לא נכון.
 2. יהיה יותר מידי לוגים והממשק משתמש לא יוכל להעלות את כולם.

פתרונות לבעיה

1. ביצוע שמירה של כל הלוגים בקובץ או במסד נתונים, ולבצע שליחה מחדש כל כמה שניות. כלומר, כל ממשק מנהל היה שואל את השרת: "האם יש לוג חדש?". פתרון כזה יוצר עומס לא הכרחי ברשת ובממשק משתמש, ולא צריך אותו כי אין שינוי בפרטי log או מחיקה של log, שלא כמו ברשימת עובדים/תהליכים.
2. שליחת הלוגים ללקוח כל פעם שיש התחברות מחדש. זה לא מאפשר לראות עידכונים אבל בזמן אמת, וזה לא מה שאני רוצה.

הפתרון שבחרתי הוא broadcast למנהלים כל פעם שנמחק לוג. בפונקציה של מחיקת הLog מתקבל הלוג והוא נשלח ישירות לכל הלקוחות שהם מנהלים ומתווסף לממשק משתמש שלהם. הפתרון שלי הכי טוב כי הוא מאפשר עדכונים בזמן אמת לא כמו פתרון 2, והוא לא משתמש הרבה ברשת כמו פתרון 1. בהתחשב שהשרת נשאר תמיד פתוח, אני שומר את הLogs ברשימת בצד השרת, ולא במסד נתונים. אני עושה זאת כדי לפזר את העומס על המסד נתונים.

פתרונות משניים:

1. שימוש בthreading.lock בעת שימוש בLogs. זה מונע מתהליכים אחרים להשתמש בלוגים ובכך מונע התנגשויות.

2. הגבלתי את כמות הלוגים שאפשר להשים ברשימה, ובכך אם מתווסף לוג חדש שעובר את הגבול של הרשימה האיבר הראשון שנוסף לרשימה נמחק. רואים רק את האיברים הכי חדשים, בשביל שממשק המשתמש יעבוד כראוי.

תיאור סביבת פיתוח פירוט כלי הפיתוח הנדרשים לפיתוח

1. Python 3.13.3 השפה שבה כתובה הפרויקט
2. Edition Community Pycharm - לצורך כתיבה והרצה של הקוד.
3. 0 DB Browser(SQLite) - לראות את הטבלה של המסד נתונים.

כלים הנדרשים לבדיקות

1. שורת הפקודה CMD – הזנת הפקודה ipconfig לצורך מציאת כתובת ה-IP
2. מנהל משימות של המחשב בשביל לבדוק עם התהליכים שמוצגים לי קיימים/הערכים שלהם נכונים.

תיאור פרוטוקול התקשורת תיאור מילולי של פרוטוקול התקשורת

TCP - Transmission Control Protocol: הוא פרוטוקול תקשורת המבוסס על חיבור שני צדדים, שרת ולקוח.. לעומת פרוטוקולים אחרים, TCP מוודא שהמידע יעבור, בסדר הנכון ובלי שגיאות. הפרוטוקול מייצר חיבור מראש באמצעות תהליך של "לחיצת יד תלת-שלבית" (3-Way Handshake), ורק אחרי זה הוא ישלח את הנתונים. TCP מחלק את המידע לחבילות, פקטות, ודואג שכל חבילה תתקבל עם אישור. אם חבילה לא מתקבלת, היא נשלחת שוב. בנוסף, TCP כולל מנהל עומסים כדי להתאים את עצמו ליכולת של השרת והלקוח. TCP משמש בפרוטוקולים קריטיים כמו HTTP, דואר אלקטרוני, העברת קבצים, ובמערכות socket – כמו זו שבנית – שבהן צריך תקשורת אמינה. מבנה של פקטה של TCP

Transmission Control Protocol (TCP) Header 20-60 bytes

source port number 2 bytes				destination port number 2 bytes			
sequence number 4 bytes							
acknowledgement number 4 bytes							
data offset 4 bits		reserved 3 bits		control flags 9 bits		window size 2 bytes	
checksum 2 bytes				urgent pointer 2 bytes			
optional data 0-40 bytes							

פירוט כלל ההודעות הזורמות במערכת

`/key {e} {n}`

כיוון: לקוח לשרת

תיאור: שליחת מפתח ציבורי (RSA) של הלקוח לצורך הצפנת הודעות. מבוצע בתחילת החיבור.
[SERVER] ERROR: Incomplete key. Send as /key e n

כיוון: שרת ללקוח

תיאור: למפתח הציבורי שנשלח חסר פרמטר.

`/login {username} {password}`

כיוון: לקוח לשרת

תיאור: ניסיון התחברות של משתמש קיים עם שם משתמש וסיסמה.

`login_successful`

כיוון: שרת ללקוח

תיאור: התחברות בוצעה בהצלחה.

`invalid_creds`

כיוון: שרת ללקוח

תיאור: פרטי התחברות שגויים.

`not_enough_args`

כיוון: שרת ללקוח

תיאור: הפקודה נשלחה עם פחות מדי ארגומנטים.

`/role`

כיוון: לקוח לשרת

תיאור: בקשה לקבלת תפקיד המשתמש (0=עובד, 1=מנהל).

`role:{role_number}`

כיוון: שרת ללקוח

תיאור: שליחת תפקיד המשתמש בחזרה ללקוח.

`/username`

כיוון: לקוח לשרת

תיאור: בקשה לקבלת שם המשתמש.

`{username}`

כיוון: שרת ללקוח

תיאור: שליחת שם המשתמש עצמו ללקוח.

`/startstreaming`

כיוון: לקוח לשרת

תיאור: התחלת קבלת זרם חי של תהליכים מהשרת.

אורי אלפי | Proccesors

PROCESS_STREAM:{json}

כיוון: שרת ללקוח
תיאור: שליחת מידע בפורמט JSON על תהליכים רלוונטיים (עד 15 התהליכים עם הכי הרבה CPU).

/terminate {pid} {reason}

כיוון: לקוח לשרת
תיאור: בקשה לסיים תהליך ספציפי על פי מזהה PID וסיבת המחיקה.

LOG:{process_name}\n{cpu}\n{memory}\n{priority}\n{reason}\n{user}\n{timestamp}

כיוון: שרת ללקוח (מנהלים בלבד)
תיאור: שליחת לוג על תהליך שנמחק – מוצג בלייב בממשק המנהל.

/salary {user_id} {new_salary}

כיוון: לקוח לשרת
תיאור: שינוי השכר של עובד מסוים לפי מזהה.

/change_username {user_id} {new_username}

כיוון: לקוח לשרת
תיאור: שינוי שם המשתמש של עובד לפי מזהה.

/change_role {user_id} {new_role}

כיוון: לקוח לשרת
תיאור: שינוי התפקיד של משתמש (0=עובד, 1=מנהל).

/fire {user_id}

כיוון: לקוח לשרת
תיאור: פיטורין – מחיקת עובד מהמסד וניתוקו מהמערכת.

/add_user {username} {password} {role} {salary}

כיוון: לקוח לשרת
תיאור: הוספת משתמש חדש למערכת עם פרטי התחברות, תפקיד ושכר.

/quit

כיוון: לקוח לשרת
תיאור: בקשה לסיים את החיבור לשרת.

[SERVER] Bye!

כיוון: שרת ללקוח
תיאור: הודעת ניתוק סופית.

[SERVER] Unknown command
או [SERVER] Unknown command emp.\n

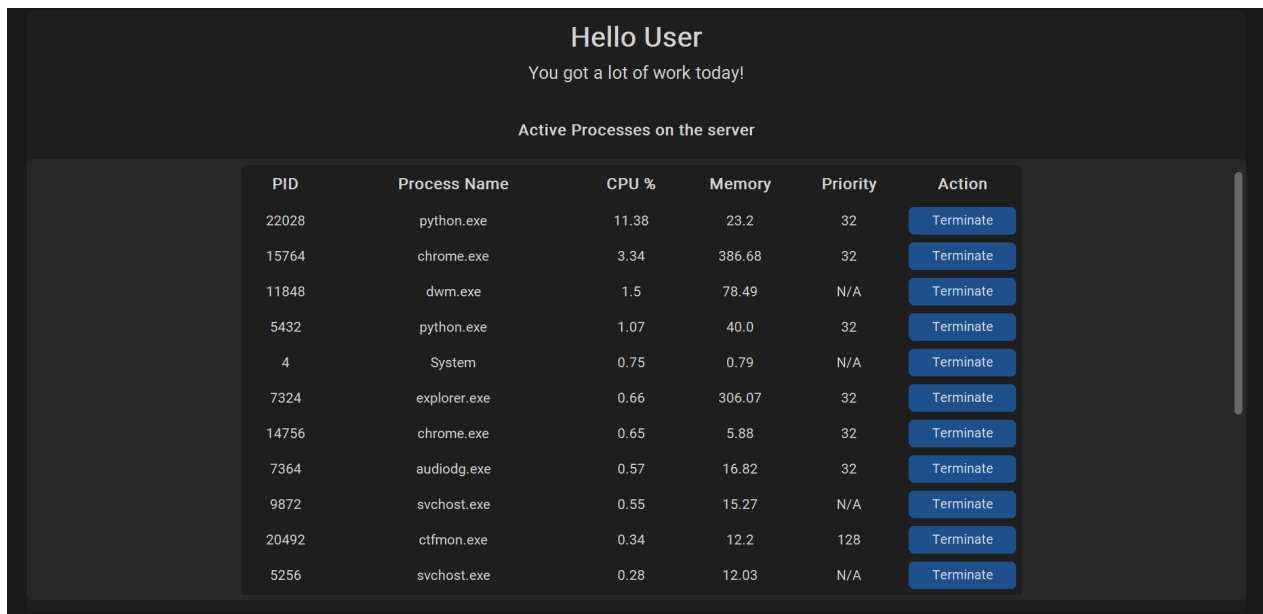
אורי אלפי | Proccesors

כיוון: שרת ללקוח
תיאור: הפקודה שהתקבלה אינה מוכרת.

[SERVER] Invalid command. Commands start with '/'
או [SERVER] Invalid command. Commands start with '/', use '/help' for help.\n

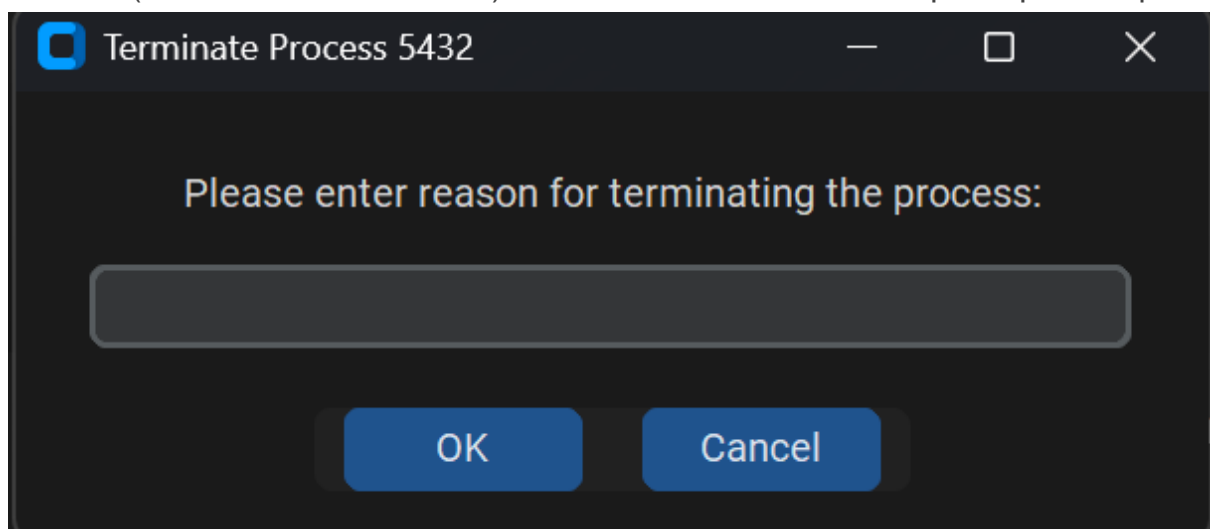
כיוון: שרת ללקוח
תיאור: השרת קיבל פקודה לא תקינה (ללא / או בפורמט לא צפוי).

תיאור מסכי המערכת



PID	Process Name	CPU %	Memory	Priority	Action
22028	python.exe	11.38	23.2	32	Terminate
15764	chrome.exe	3.34	386.68	32	Terminate
11848	dwm.exe	1.5	78.49	N/A	Terminate
5432	python.exe	1.07	40.0	32	Terminate
4	System	0.75	0.79	N/A	Terminate
7324	explorer.exe	0.66	306.07	32	Terminate
14756	chrome.exe	0.65	5.88	32	Terminate
7364	audiodg.exe	0.57	16.82	32	Terminate
9872	svchost.exe	0.55	15.27	N/A	Terminate
20492	ctfdmon.exe	0.34	12.2	128	Terminate
5256	svchost.exe	0.28	12.03	N/A	Terminate

תפקיד המסך: מסך עובד
תיאור המסך: המסך מציג את כל 15 התהליכים שהכי משתמשים במעבד במחשב של שרת, ומראה אותם ברשימה מסודרת עם המשאבים שלהם: PID - הזיהוי שלהם, השם שלהם השימוש במעבד שלהם, שימוש בזכרון, תעדוף שלהם. לצד כל תהליך, יש כפתור מחיקה שפותח חלון למחיקת התהליך. במסך מוצג גם השם משתמש של העובד (בצילום שם העובד הוא User).



Terminate Process 5432

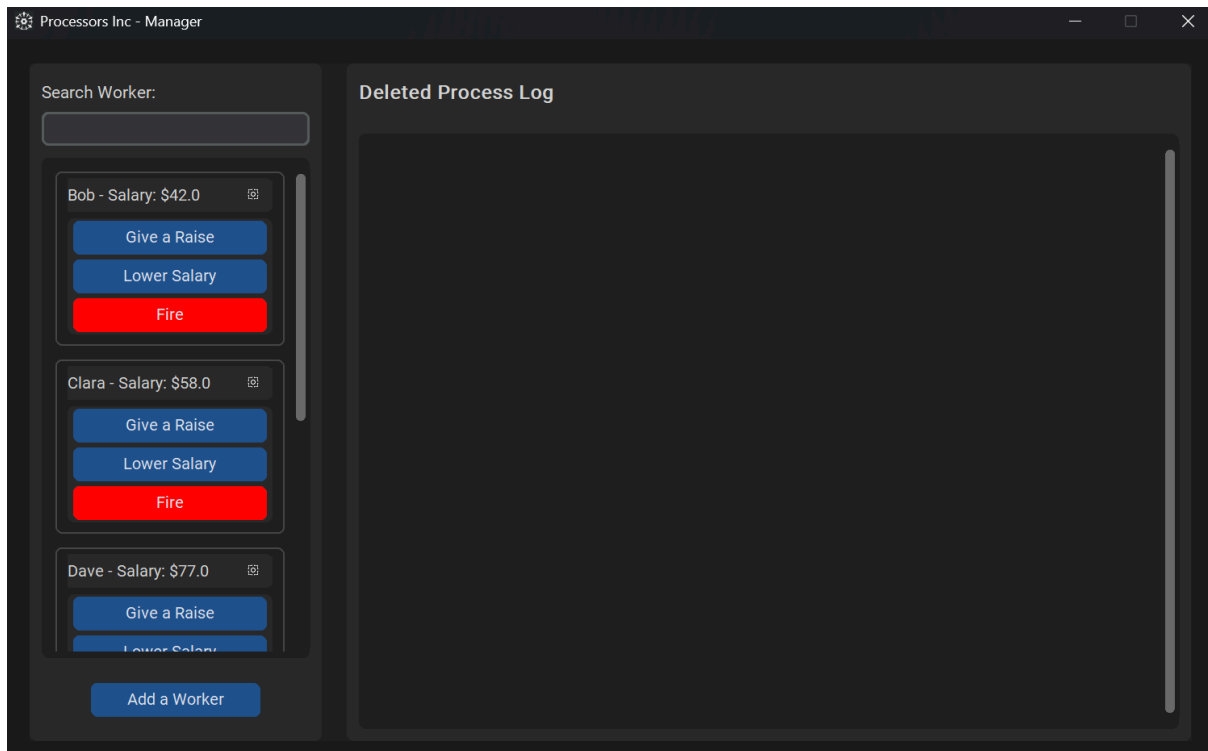
Please enter reason for terminating the process:

OK Cancel

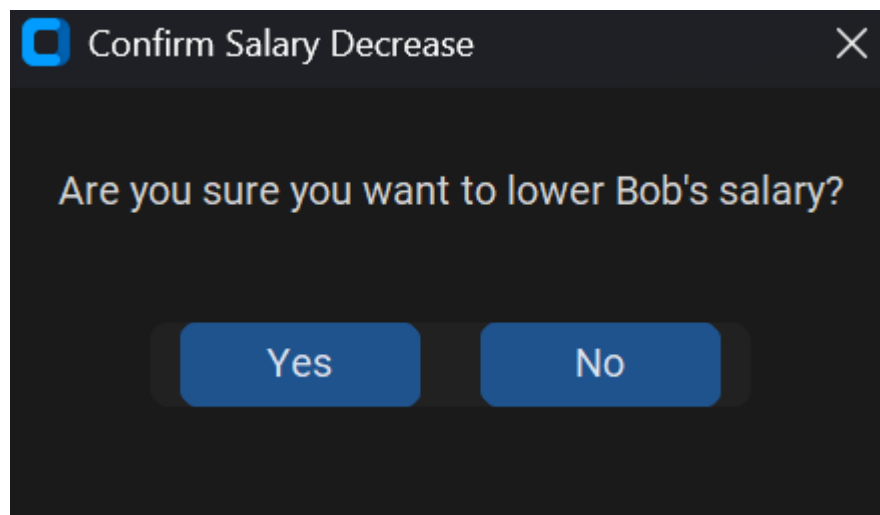
תפקיד המסך: וידוי כפול של מחיקת תהליך

Processors | אורי אלפי

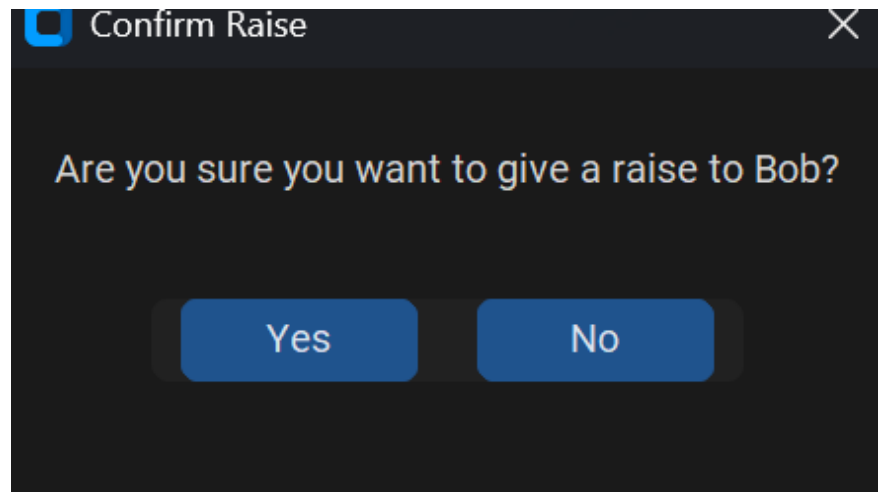
תיאור המסך: המשך מבקש מהמשתמש להכניס סיבה למחיקת התהליך הנבחר במסך הקודם. למשתמש 2 כפתורים, אחד לביטול והשני לאישור המחיקה.



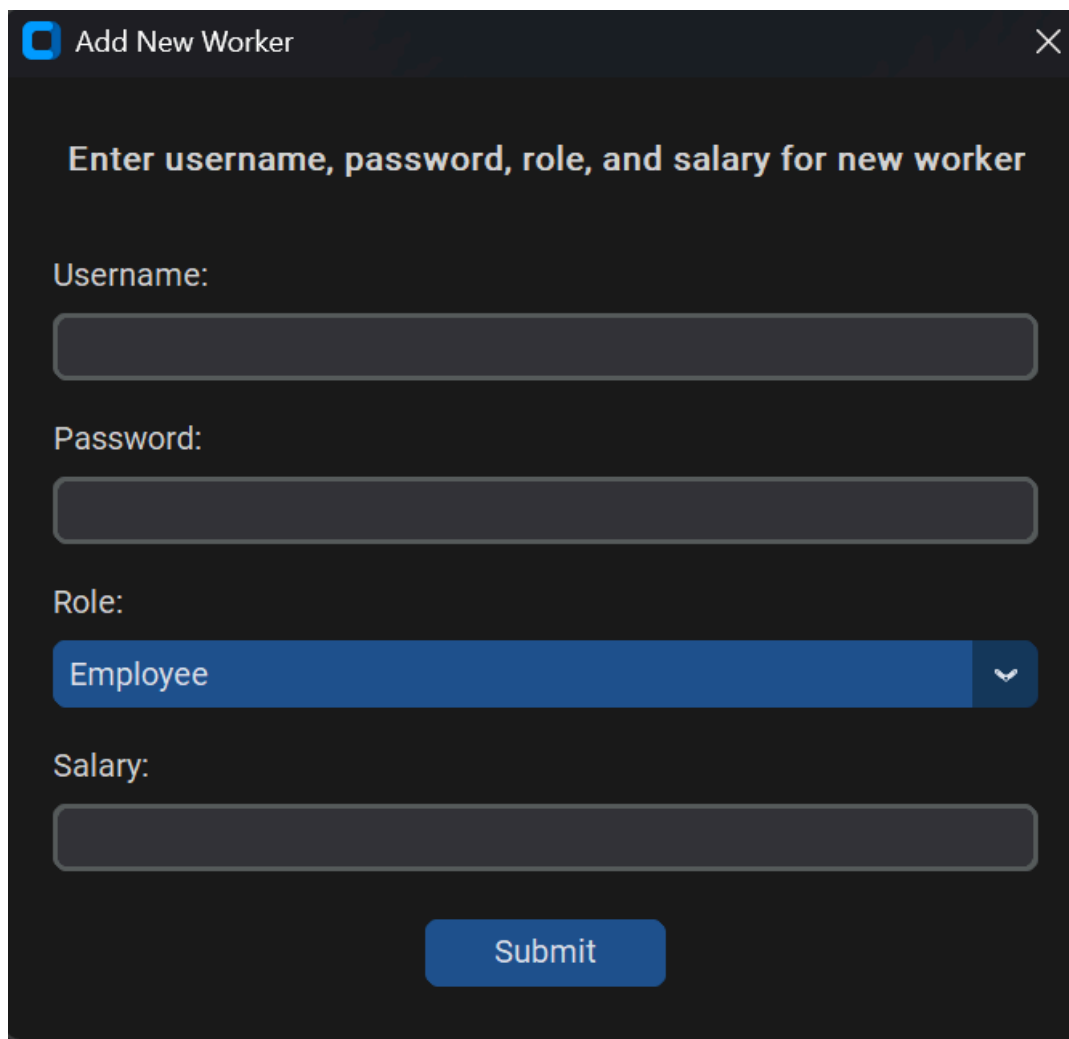
במסך השמאלי יש רשימה של כל התהליכים שנמחקו (איו תהליכים מחוקים בתמונה). כל תהליך ברשימה הזאת כולל את שם המשתמש של המוחק, סיבת המחיקה, תאריך המחיקה עם שעה, שימוש במעבד של התהליך, שימוש בזכרון של התהליך, תעדוף של התהליך, שם התהליך. התהליכים נמצאים ברשימה הראשון הוא האחרון שהתווסף, כך שזה מונע גלילה מיותרת ולא נוחה.



תפקיד המסך: וידוי כפול של הורדת שכר של משתמש תיאור המסך: ברגע שמנהל לוחץ על כפתור הורדת שכר על אחד מהעובדים, החלון הזה צץ. בחלון, המנהל נשאל אם הוא בטוח אם הוא רוצה לעשות את הפעולה זאת. החלון נותן לו 2 אופציות: או לבטל או להמשיך בפעולה זאת.



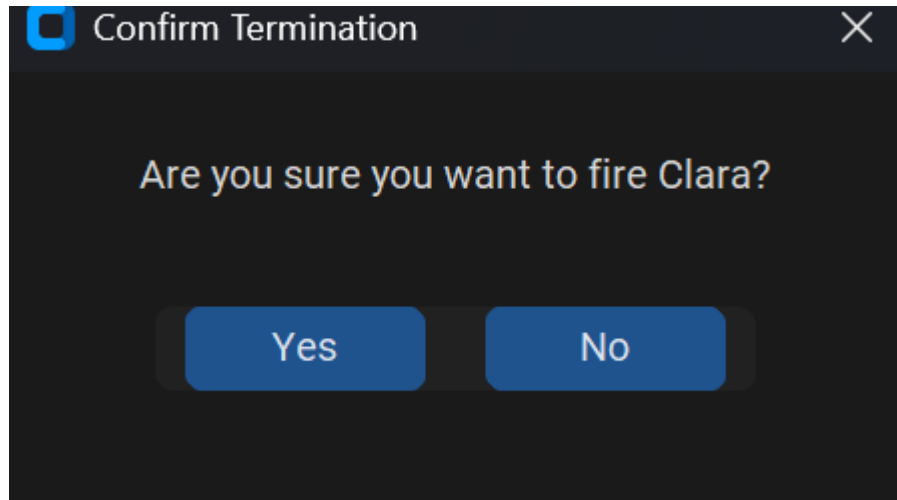
תפקיד המסך: וידוי כפול של העלאת שכר של משתמש
תיאור המסך: ברגע שמנהל לוחץ על כפתור העלאת שכר על אחד מהעובדים, החלון הזה צץ. בחלון, המנהל נשאל אם הוא בטוח אם הוא רוצה לעשות את הפעולה זאת. החלון נותן לו 2 אופציות: או לבטל או להמשיך בפעולה זאת.

A dark-themed form titled "Add New Worker" with a close button (X) in the top right corner. The main text says, "Enter username, password, role, and salary for new worker". Below this, there are four input fields: "Username:" (text input), "Password:" (password input), "Role:" (dropdown menu with "Employee" selected), and "Salary:" (text input). At the bottom, there is a blue "Submit" button.

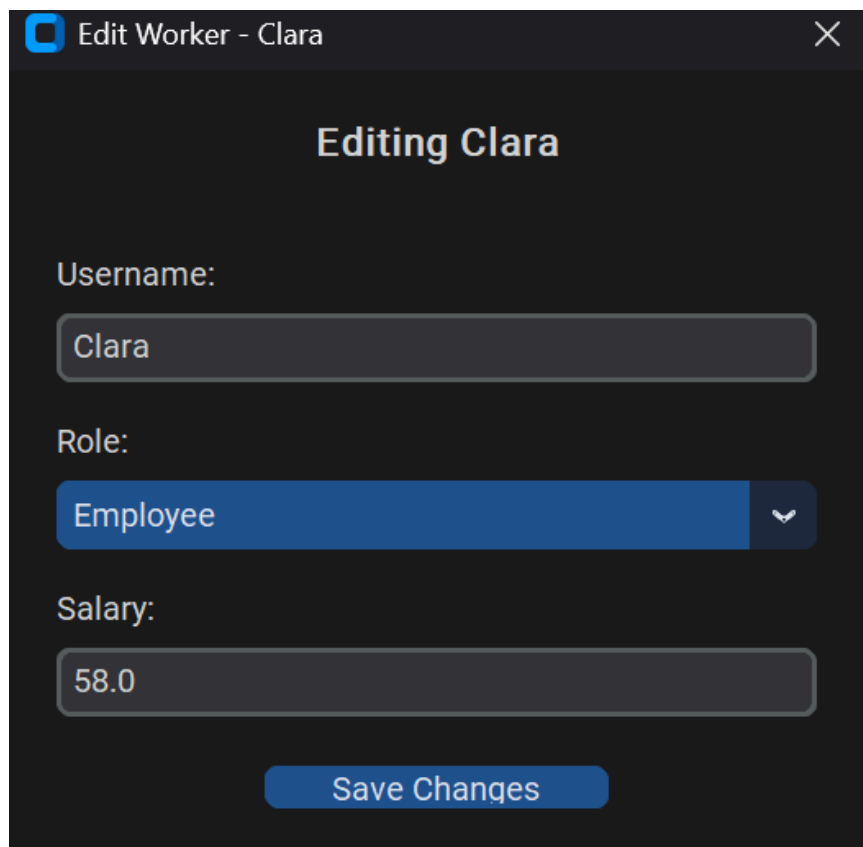
תפקיד המסך: הוספת משתמש

אורי אלפי | Proccesors

תיאור המסך: ברגע שמנהל לוחץ על כפתור הוספת משתמש, החלון הזה צץ. בחלון, המנהל מכניס את פרטי שם המשתמש של העובד, סיסמה, תפקיד בחברה, ושכר. הפעולה מבוטלת אם החלון נסגר, או הפרטים לא מתאימים. התפקיד הדיפולטי הוא עובד, ולכן מוצג ישר ככה. רק אם משנים אותו אפשר להחליט להפוך את העובד החדש למנהל.



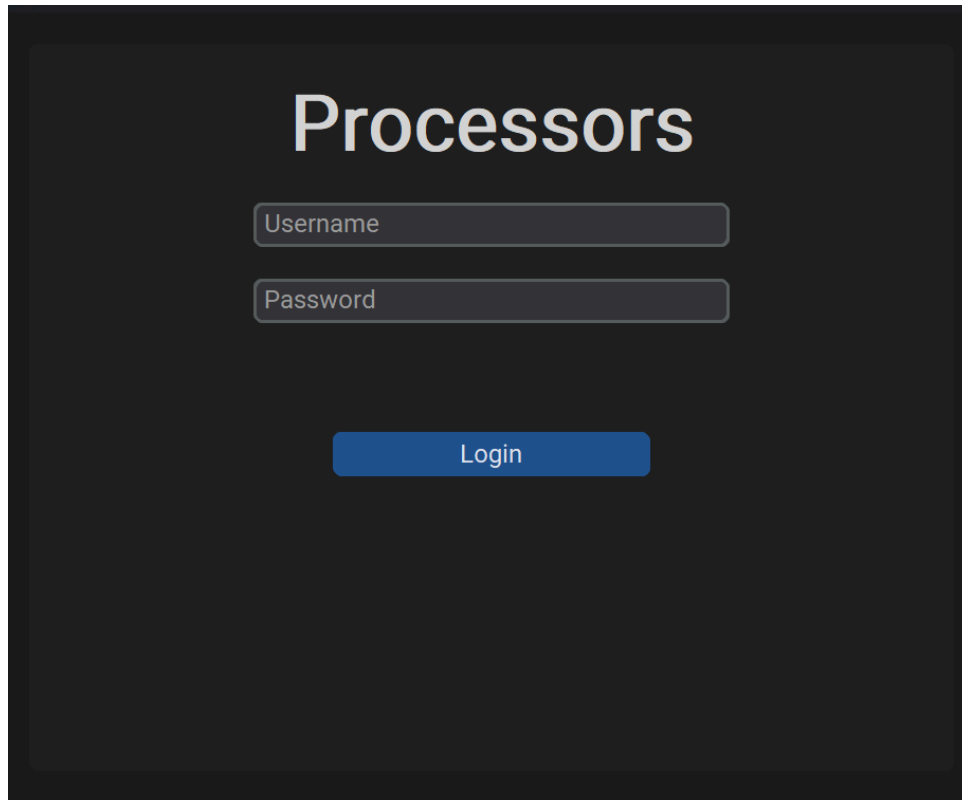
תפקיד המסך: וידוי כפול של פיטור משתמש
תיאור המסך: ברגע שמנהל לוחץ על כפתור פיטורין על אחד מהעובדים, החלון הזה צץ. בחלון, המנהל נשאל אם הוא בטוח אם הוא רוצה לפטר את העובד. החלון נותן לו 2 אופציות: או לבטל או להמשיך בפעולה זאת.

A dark-themed dialog box titled "Edit Worker - Clara" with a close button (X) in the top right corner. The main title is "Editing Clara". It contains three input fields: "Username:" with the value "Clara", "Role:" with a dropdown menu showing "Employee", and "Salary:" with the value "58.0". At the bottom, there is a blue button labeled "Save Changes".

תפקיד המסך: שינוי פרטי משתמש

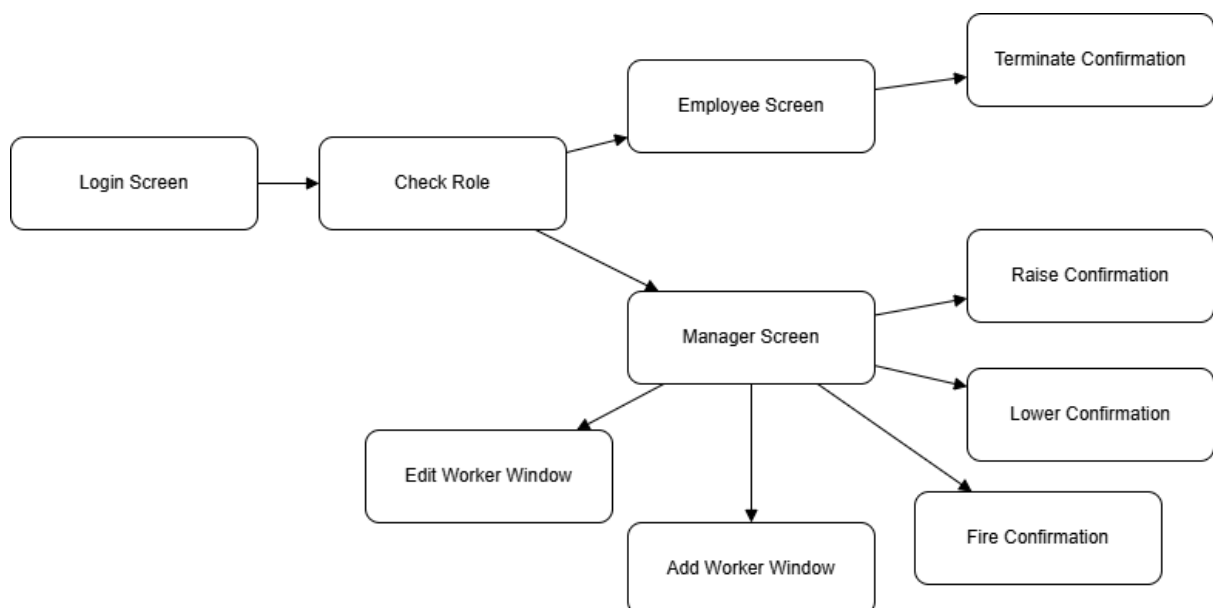
אורי אלפי | Processors

תיאור המסך: ברגע שמנהל לוחץ על כפתור שינוי פרטים על אחד מהעובדים, החלון הזה צץ. בחלון, המנהל רואה את הפרטים הקיימים על המשתמש, ויכול לשנות אותם לפי רצונו. הפרטים הם שם משתמש, תפקיד בחברה ושכר. אם המנהל מחליט לסגור בלי לשמור, הפרטים לא ישתנו, אחרת הם כן.



תפקיד המסך: מסך התחברות למשתמש
תיאור המסך: במסך, המשתמש צריך להכניס את שם המשתמש והסיסמה שלו, ובכך להתחבר לחשבון שלו. אם אחד מהם שגויים/יש בעיה כלשהי, הממשק מוסיף את הבעיה מתחת לקופסאות הכנסת טקסט. בשביל שהמשתמש יכנס, עליו ללחוץ על כפתור הLogin.

תרשים מסכים



תיאור מבני הנתונים

קבצים:

1. קבצי קוד: לדוגמא- [server.py](#) [client.py](#) [manager.py](#) בתוכנת pycharm: [WorkerDC.py](#): מחלקת נתונים המייצגת עובד במערכת.
[Logs.py](#): רשימת הLogs שהם נשמרים במשתנה רשימה מסוג מחלקת Logs.
2. [database.db](#): "בסיס נתונים מקומי מסוג SQLITE. כל נתוני המערכת נשמרים שם."

בסיס נתונים:

שם הטבלה: users - מאחסן את פרטי המשתמשים

שם שדה	טיפוס	פירוט	דוגמאות לערכים
id	INTEGER	מפתח ראשי (זיהוי של משתמש)	1,2,3,5
username	TEXT	שם משתמש של משתמש	"Ori Alfi"
password	TEXT	סיסמה של משתמש אחרי hashing	"2c7e9e...7a9f"
salt	TEXT	ערך salt יחודי	"...3af0be1f2a"
role	INTEGER	תפקיד במערכת -0 מנהל 1- עובד	0
salary	REAL	שכר של משתמש	56.0 88.0

השדה id הוא המפתח הראשי של הטבלה.
username צריך להיות ייחודי (UNIQUE NOT NULL)
role חייב להיות 0 או 1 ((CHECK(role BETWEEN 0 AND 1)

סקירת חולשות ואיומים שכבת האפליקציה

עבודה עם אתרי web: אין בפרויקט עבודה עם אתרי web, ולפיכך אין את האיומים שקשורים בהם.

תקיפת SQL Injection: תקיפה בה מוזן קלט מזיק שמשנה את מבנה שאילתת ה-SQL ופותרת גישה למידע. התקיפה יכולה לקרות בעיקר בתהליך ההתחברות למערכת, בו מתקבלים קלטים מהמשתמש (שם משתמש וסיסמה) ונשלחים לDatabase בשביל להיכנס למערכת(היא יכולה לקרות גם בממשק המנהל, אך הדבר לא סביר כי המנהל הוא בן אדם שקיבל את הסמכות של

אורי אלפי | Processors

החברה). כדי למנוע תקיפות מסוג זה, המערכת עושה שימוש בשאליות מוכנות מראש, ולא מחברת את הקלטים ישירות אל מחרוזות ה-SQL, אלא בעצם מחלקת אותם לפרמטרים.

תהליך כניסה והרשמה: יש סכנה שאדם שמסניף פקטות שעוברות בין לקוח לשרת או שבדרך מסוימת משיג גישה למסד הנתונים ויכול לצפות בפרטי משתמש ובכך לפרוץ לחשבון. בגלל זה שמת בפרויקט

1. הצפנת Hash על הסיסמה בשליחתה למסד נתונים. *אציין כי זהו פתרון זמני שעד זמן הבחינה יתוקן להיות הצפנת RSA מהלקוח לשרת. ברגע זה, יש רק הצפנת RSA מהשרת ללקוח אבל מבחינת אבטחה זה בסדר כי המידע הרגיש נשלח מהשרת אבל עדיין אוסיף זאת בעתיד.
2. הסיסמאות בבסיס הנתונים מוצפנות ב-Hash, שקשה מאוד לפענח עד כמעט בלתי אפשרי. בנוסף, מתווסף לסיסמאות Salt יחודי ששמור לכל משתמש במסד נתונים. הsalt מאפשר לאבטחה אפילו יותר טובה, ומונע את האפשרות של הכנסת הסיסמאות הכי נפוצות למערכת אחרי פונקציית הגיבוב עד שהפורץ מצליח.

MITM גורם זדוני מתווך בין התקשורת בין לקוח לשרת, וכך יגלה מידע של הלקוח או ישנה אותו בפקטה. איפה שיש מידע רגיש כמו סיסמה, אני מצפין בפונקציית hash על מנת לוודא שלא שינו את הבקשה בדרך, או שיגלו מידע על המשתמש.

RSA שיטת הצפנה א-סימטרית שמשתמשת בזוג מפתחות: מפתח ציבורי להצפנה ומפתח פרטי לפענוח. היא מבוססת על מתמטיקה, בפירוק של מספרים ראשוניים גדולים. בפרויקט שלי, RSA משמשת להצפנת הודעות בין השרת ללקוח כך שגם אם מישהו הסניף את המידע – הוא לא יוכל לקרוא אותו בלי המפתח הפרטי. *אוסף גם הצפנה לצד השני, מהלקוח לשרת עד הבחינה

התקפת DOS/DDOS: במקרה שהמון לקוחות יתחברו בו זמנית וינסו להספיק את השרת בבקשות למידע או לפעולות השרת ידע מה לעשות. לשרת יש מכסה של לקוחות שמוגדרת, ולכן היא לא תוחרג. לכן, הפרויקט שלי בטוח נגד התקפה זו.

Hash. מה זה? אלגוריתם שלוקח קלט (כמו סיסמה) וממיר אותו לערך קבוע באורך שנראה אקראי. פונקציית hash תמיד תחזיר את אותה תוצאה לאותו קלט, אך תיצור תוצאה שונה לחלוטין גם אם משנים רק תו אחד! בנוסף, אי אפשר לשחזר את הקלט המקורי מהתוצאה, ולכן אני משתמש ב-hash כדי לאחסן סיסמאות בצורה מאובטחת – משווים את הערך שהתקבל אחרי hashing ולא את הסיסמה עצמה.

שכבת התעבורה

Man In The Middle מתקפת הסנפה למידע הקשור בין שני מחשבים, על ידי התחזות של גורם שלישי, בעצם לישות אמצעית שמעבירה את המידע בין הישויות. הסכנה היא שהישות המסניפה תגנוב/תשבש מידע המועבר בין שתי הישויות. סיכון זה מטופל באמצעות הצפנת RSA. למרות זאת, זה מטופל רק כאשר מידע מועבר מהשרת ללקוח, כי בעצם המידע הרגיש בעיקר בא מהשרת. עד בחינת הפרויקט אוסיף הצפנה גם מהלקוח לשרת.

פרק 4 - מימוש

מודולים / מחלקות מיובאים

socket

מודול שמספק גישה לרשתות TCP/IP ומאפשר תקשורת בין מחשבים. בפרויקט, המודול משמש לחיבור של לקוח (Client) לשרת (Server) ולשליחת וקבלת נתונים ביניהם. הוא מאפשר פעולות כמו connect, send, recv ועוד.

threading

מודול שמאפשר להריץ תהליכים (Threads) – כלומר קטעי קוד שפועלים במקביל. זה שימושי במיוחד כאשר רוצים להאזין להודעות מהשרת תוך כדי שהממשק הגרפי פתוח ומגיב למשתמש. כך, הקוד יכול להריץ בזמנית את התקשורת מול השרת ואת ממשק המשתמש.

sys

מודול שמאפשר גישה לפעולות ברמת מערכת ההפעלה. בפרויקט הוא משמש בעיקר לפקודה sys.exit() – כדי לסגור את התוכנית בצורה מסודרת כאשר מתנתקים מהשרת או כשיש שגיאה קריטית.

enum.IntEnum

מחלקה מתוך המודול enum שמאפשרת ליצור Enumerations – קבוצות של ערכים קבועים בעלי משמעות. IntEnum הוא סוג מיוחד שבו כל ערך מתנהג כמו מספר שלם (int). בפרויקט זה משמש להגדרת תפקידים כמו Role.EMPLOYEE או Role.MANAGER.

sqlite3

מודול שמובנה בפייתון ומאפשר לעבוד עם מסדי נתונים מסוג SQLite, שהם קבצי דאטה פשוטים. משמש ליצירת טבלאות, הוספה, מחיקה ועדכון של רשומות – כל זה נשמר בתוך קובץ מקומי.

hashlib

מודול להצפנה חד-כיוונית (Hashing), שנמצא בספרייה הסטנדרטית של פייתון. משמש להצפנה חד-כיוונית של סיסמאות באמצעות אלגוריתם SHA-256, כך שהסיסמה לא נשמרת בקובץ בצורה גלויה – רק ערך ה-Hash שלה. זהו חלק קריטי באבטחת מידע.

os

מודול שמאפשר אינטראקציה עם מערכת ההפעלה.

json

מודול מובנה בפייתון שמאפשר המרה בין מבני נתונים של פייתון לפורמט JSON ולהפך.

customtkinter

ספריית ממשק גרפי חיצונית המבוססת על tkinter, המציעה רכיבים מודרניים בעיצוב מותאם אישית.

time

מודול מובנה בפייתון המאפשר עבודה עם זמן, כולל השהיית ביצוע של פעולות באמצעות sleep, מדידת זמן ועוד.

datetime

מודול מובנה בפייתון שמספק כלים לעבודה עם תאריכים, זמנים והפרשים ביניהם.

PIL.Image

ספרייה חיצונית לעיבוד תמונה (נקראת גם Pillow), שמאפשרת פתיחה, עריכה ושמירה של קבצי תמונה בפורמטים שונים.

numpy.pad

Proccesors | אורי אלפי

פונקציה מתוך ספריית NumPy שמאפשרת להוסיף שוליים (padding) למערכים – לדוגמה, הוספת שורות או עמודות עם ערכים קבועים סביב מערך קיים.

platform

מודול מובנה בפייתון שמספק מידע על מערכת ההפעלה, כמו שם המערכת, גרסה, סוג מעבד ועוד.

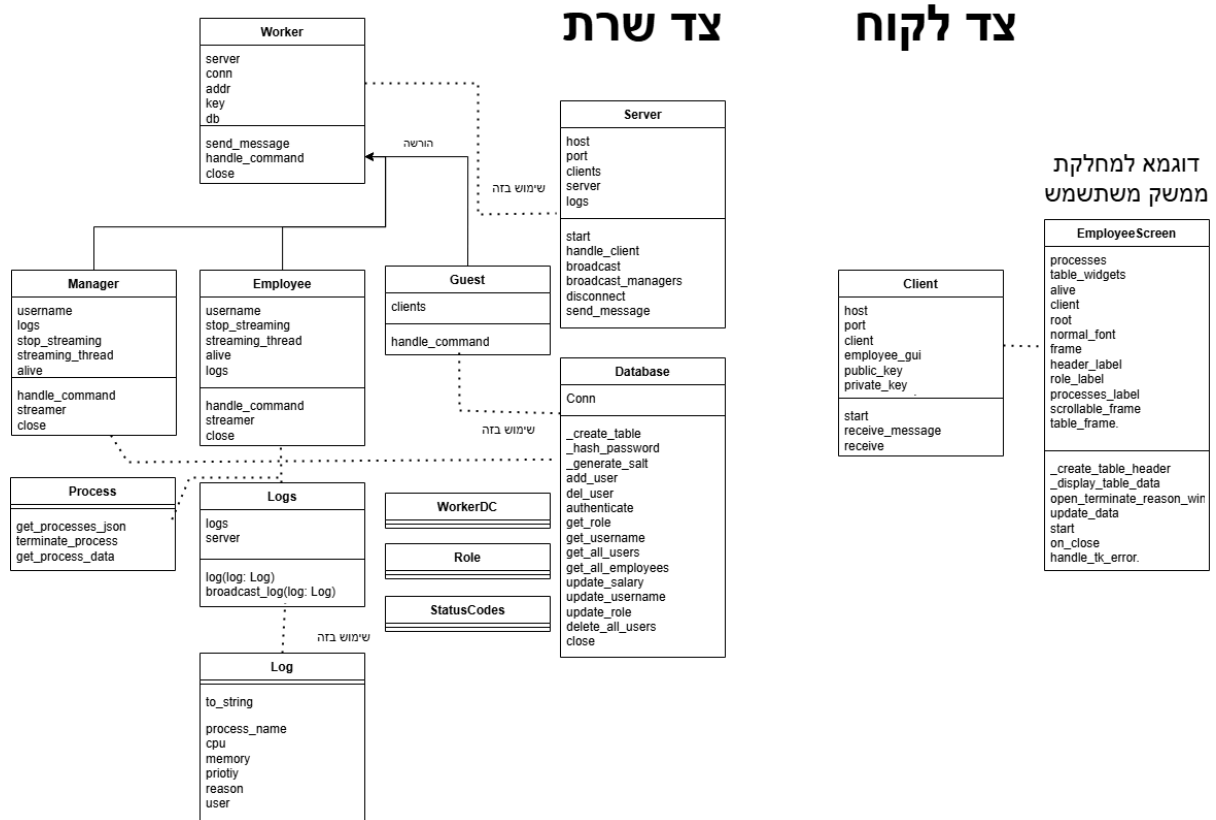
psutil

ספרייה חיצונית שמאפשרת גישה לנתונים של מערכת ההפעלה בזמן אמת, כגון תהליכים פעילים, שימוש בזיכרון ו-CPU, מידע על דיסקים ותקשורת ברשת.

random

מודול מובנה בפייתון שמספק פונקציות להגרלות, בחירה אקראית, ויצירת מספרים אקראיים (פסאודו-אקראיים).

מחלקות שאני הכנתי



*יש 3 מחלקות של ממשק משתמש אז הצגתי רק אחת לדוגמה.

Client: תיאור המחלקה:

מחלקת Client אחראית על חיבור לקוח לשרת דרך TCP, ניהול תהליך התחברות מאובטח באמצעות הצפנת RSA, הפעלת ממשק משתמש גרפי בהתאם לתפקיד (עובד או מנהל), והאזנה להודעות מהשרת ברקע תוך כדי הפעלת הממשק.

מאפיינים

host: כתובת ה-IP של השרת (ברירת מחדל '127.0.0.1'). משמש להתחברות לשרת.

port: הפורט שבו פועל השרת. משמש יחד עם הכתובת ליצירת חיבור רשת.

client: אובייקט TCP socket משמש לשליחה וקבלה של נתונים מהשרת.

אורי אלפי | Proccesors

employee_gui: משתנה השומר את ממשק המשתמש לעובד. מאותחל רק אם המשתמש הוא עובד.

public_key: מפתח ציבורי RSA. נשלח לשרת כדי שהוא יוכל להצפין הודעות.

private_key: מפתח פרטי RSA. משמש לפענוח הודעות שמתקבלות מהשרת.

manager_gui (נוצר דינאמית): ממשק המשתמש למנהל. מאותחל רק אם המשתמש הוא מנהל.

פעולות

start: יוצרת את החיבור לשרת, שולחת את המפתח הציבורי, מפעילה את מסך ההתחברות, ומפעילה את הממשק הגרפי המתאים (עובד/מנהל) בהתאם לתפקיד המשתמש. סוגרת את החיבור במקרה של ניתוק או שגיאה.

receive_message(conn): מקבלת הודעה מהשרת דרך הסוקט, מפענחת אותה באמצעות המפתח הפרטי, ומחזירה את המידע הטקסטואלי. מקבלת את החיבור. במידה והחיבור נסגר בטרם הושלמה ההודעה – תועלה חריגה.

receive(gui): פועלת כלולאה ברקע (thread), מאזינה להודעות מהשרת באופן מתמשך, ומעדכנת את הממשק הגרפי בהתאם. מקבלת את הממשק הגרפי. במקרה של ניתוק או שגיאה – סוגרת את החיבור ואת הממשק.

תיאור המחלקה: Database

מחלקת Database אחראית על ניהול מסד נתונים מסוג SQLite. היא יוצרת את טבלת המשתמשים (אם לא קיימת), ומספקת ממשק מלא לביצוע פעולות כמו הוספת משתמשים עם הצפנה, אימות משתמשים, עדכון פרטי משתמשים, מחיקה, שליפה של עובדים לפי תפקיד ועוד. הסיסמאות מאוחסנות בצורה מאובטחת באמצעות שילוב של salt ו-hash.

מאפיינים

conn: אובייקט חיבור למסד הנתונים (sqlite3.Connection), אשר באמצעותו מתבצעות כל השאילתות.

פעולות

__init__(self, db_path='database.db')

פעולה זו יוצרת חיבור למסד הנתונים. היא מקבלת פרמטר אופציונלי db_path שהוא מחרוזת המציינת את מיקום קובץ מסד הנתונים. עם ההתחברות, היא מיד יוצרת את טבלת המשתמשים אם זו לא קיימת.

_create_table(self)

יוצרת את טבלת users במסד הנתונים אם היא לא קיימת, עם העמודות: id, username, password, salt, role ו-salary. אין לפרוצדורה זו פרמטרים.

_hash_password(self, password)

מקבלת פרמטר אחד: password – מחרוזת המייצגת את הסיסמה המקורית. הפעולה מחזירה את ערך ההצפנה (Hash) של הסיסמה באמצעות האלגוריתם SHA-256.

_generate_salt(self)

יוצרת ערך salt אקראי (16 בתים) ומחזירה אותו כמחרוזת הקסדצימלית. פעולה זו לא מקבלת פרמטרים.

add_user(self, username, password, role, salary=0.0)

מוסיפה משתמש חדש למסד הנתונים. היא מקבלת את שם המשתמש (username), סיסמה (password), תפקיד (role), ערך מספרי: 0 לעובד או 1 למנהל, ושכר (salary, ברירת מחדל: 0.0). הסיסמה מאוחסנת לאחר שעברה שילוב עם salt והצפנה. הפעולה מחזירה True אם המשתמש נוסף בהצלחה, או False אם כבר קיים משתמש עם אותו שם.

del_user(self, user_id)

אורי אלפי | Proccesors

מוחקת משתמש לפי מזהה (user_id, מספר שלם). הפעולה לא מחזירה ערך.
authenticate(self, username, password)
מאמתת את זהות המשתמש. היא מקבלת שם משתמש (username) וסיסמה (password),
שולפת את ה-salt המתאים, מצרפת אותו לסיסמה, מבצעת hash, ובודקת אם הערך תואם
לרשומה במסד הנתונים. מחזירה True אם פרטי ההתחברות תקינים, אחרת False.
get_role(self, username)
מקבלת שם משתמש (username) ומחזירה את ערך התפקיד שלו (מספר שלם). אם המשתמש לא
נמצא, מוחזר None.
get_username(self, user_id)
מקבלת מזהה משתמש (user_id) ומחזירה את שם המשתמש הקשור אליו. אם לא נמצא – מוחזר
None.
get_all_users(self)
שולפת את כל המשתמשים מהטבלה כולל salt, ומחזירה רשימת רשומות. פעולה זו לא מקבלת
פרמטרים.
get_all_employees(self)
מחזירה את כל המשתמשים אשר תפקידם הוא עובד (Role.EMPLOYEE). אין לפעולה זו
פרמטרים.
update_salary(self, user_id, salary)
מעדכנת את השכר (salary) של משתמש לפי מזהה (user_id). שני הפרמטרים הם מספריים. אין
ערך מוחזר.
update_username(self, user_id, new_username)
מעדכנת את שם המשתמש (new_username, מחרוזת חדשה) של המשתמש לפי מזהה
(user_id). אין ערך מוחזר.
update_role(self, user_id, new_role)
מעדכנת את תפקיד המשתמש (new_role) לפי מזהה (user_id). אם הערך של new_role אינו 0
או 1, תועלה חריגה. אין ערך מוחזר.
delete_all_users(self)
מוחקת את כל המשתמשים ממסד הנתונים. פעולה מסוכנת – אין לה פרמטרים ואין ערך מוחזר.
close(self)

סוגרת את החיבור למסד הנתונים. אין לה פרמטרים.

תיאור המחלקה: EmployeeScreen

מחלקת EmployeeScreen אחראית על יצירת ממשק גרפי עבור משתמש בתפקיד עובד. הממשק
מציג מידע על תהליכים פעילים בשרת, כולל נתונים כמו PID, שם תהליך, שימוש ב-CPU וזיכרון,
ומאפשר שליחת בקשת סיום תהליך דרך ממשק אינטראקטיבי. המחלקה משתמשת בספרייה
customtkinter לבניית ממשק גרפי מודרני ואינטואיטיבי.

מאפיינים

client: אובייקט socket של הלקוח – משמש לשליחת פקודות לשרת.
username: שם המשתמש המחובר – מוצג בכותרת הממשק.
root: חלון ה-GUI הראשי שנבנה באמצעות customtkinter.
frame: מסגרת ראשית בתוכו נמצאים כל רכיבי ה-GUI.
table_frame: טבלת התהליכים – רכיב פנימי המציג את רשימת התהליכים מהשרת.
scrollable_frame: מסגרת נגללת המכילה את הטבלה ומאפשרת גלילה של נתונים רבים.
table_widgets: מילון המכיל את רכיבי ה-GUI של כל שורת תהליך בטבלה.
processes: מילון שמכיל את הנתונים הגולמיים של התהליכים שהתקבלו מהשרת.

אורי אלפי | Proccesors

alive: בוליאני – משמש לבדוק אם הממשק פעיל ועדכני.
normal_font: הגדרת גופן ברירת מחדל בגודל 14 – משמש להצגת הנתונים בטבלה.

פעולות

`__init__(self, client_socket, username)`

פעולה בונה את ממשק המשתמש. היא מקבלת את פרמטר `client_socket` – אובייקט `socket` דרכו יתבצעו התקשורת עם השרת, ו-`username` – שם המשתמש המחובר שמוצג בברכת הפתיחה. הפונקציה מגדירה עיצוב כזה, יוצרת טבלה עם כותרות, ומכינה את כל הרכיבים הגרפיים הדרושים לתצוגת תהליכים.

`_create_table_header(self, font)`

יוצרת את שורת כותרת הטבלה. הפונקציה מקבלת את פרמטר `font`, שהוא גופן (`font object`) שמשמש לעיצוב הטקסט בכותרות. הכותרות הן: `PID`, שם תהליך, `CPU`, זיכרון, עדיפות ופעולה (כפתור סיום תהליך).

`_display_table_data(self)`

מציגה את נתוני התהליכים בטבלה. הפונקציה משתמשת ב-`self.processes` כדי לשלוף את 20 התהליכים הראשונים, מציגה את נתוניהם בטבלה ויוצרת עבור כל תהליך כפתור "Terminate" לשליחת פקודת סיום תהליך.

`open_terminate_reason_window(self, pid)`

פותחת חלון קופץ (`popup`) שבו המשתמש מתבקש להזין סיבת סיום עבור תהליך מסוים. היא מקבלת את `pid` של התהליך שאליו מתייחסת הפעולה. לאחר אישור, ההודעה נשלחת לשרת עם הסיבה שהוזנה.

`update_data(self, data)`

מעבדת נתוני תהליכים שהתקבלו מהשרת. הפונקציה מקבלת מחרוזת `data` שמתחילה ב-`PROCESS_STREAM`: ואחריה JSON של תהליכים. אם הנתונים תקינים, הם נטענים ל-`self.processes` ומוצגים בטבלה.

`start(self)`

מתחילה את הלולאה הראשית של הממשק, מה שמציג את החלון ומאזין לפעולות המשתמש.

`on_close(self)`

סוגרת את חלון הממשק הראשי (נקרא כאשר המשתמש לוחץ על כפתור הסגירה).

תיאור המחלקה: **Employee**

המחלקה `Employee` מייצגת לקוח במערכת שמחובר כעובד. היא יורשת את המחלקה `Worker`, ומוסיפה יכולות ייחודיות לעובד – כמו התחברות לשרת, קבלת נתוני תהליכים מהשרת בזמן אמת, ניהול תהליך הסטרימינג (שליחת מידע ללקוח), והגשת פקודות שמבוצעות על גבי השרת. בנוסף, היא תומכת בלוגינג של פעולות, כולל תיעוד סיבת סיום תהליך.

מאפיינים

`username`: שם המשתמש של העובד. משמש לזיהוי וכתיעוד בפעולות כמו לוגים.
`stop_streaming`: אובייקט מסוג `threading.Event` – משמש לאותת לתהליך הסטרימינג מתי להפסיק.
`streaming_thread`: מופע של `thread` – משמש להרצת שליחת הנתונים מהשרת ללקוח ברקע.
`alive`: בוליאני – מייצג אם העובד מחובר ופועל.
`logs`: הפניה לאובייקט ניהול הלוגים של השרת – משמש לתיעוד פעולות העובד כמו סיום תהליך.

פעולות

`__init__(self, conn, addr, username, server, key=None)`

אורי אלפי | Proccesors

פעולת האתחול מקבלת חיבור סוקט (conn), כתובת הלקוח (addr), שם משתמש (username), מופע שרת (server), ואופציונלית מפתח הצפנה (key). היא קוראת לפעולת האתחול של המחלקה האב (Worker) ומאתחלת את כל המשתנים הדרושים לעובד, כולל עצירת סטרימינג ויומן הפעולות. `handle_command(self, msg)`

מטפלת בהודעות שמתקבלות מהלקוח. מקבלת מחרוזת msg שמכילה פקודת טקסט (בדרך כלל מתחילה ב-/). אם הפקודה היא role/ – מחזירה את התפקיד של המשתמש. אם זו username/ – מחזירה את שם המשתמש. אם זו startstreaming/ – מתחילה תהליך של שליחת נתונים שוטפת ללקוח. אם זו terminate/ – מסיימת תהליך לפי PID ומוסיפה לוג עם הסיבה. אם הפקודה אינה מוכרת – נשלחת הודעת שגיאה. הפקודות מעובדות תוך שימוש באובייקט Processes.

`streamer(self)`

תהליך שמופעל ברקע (thread) כל עוד `stop_streaming` לא סומן. הפעולה יוצרת מופע חדש של Processes, שולפת את כל התהליכים הפעילים, ממירה אותם ל-JSON, ושולחת ללקוח כל 5 שניות. ברגע שקורה חריג או נדרש להפסיק – הלולאה נפסקת.

`close(self)`

סוגרת את פעילות העובד. הפעולה מאותתת ל-`streaming_thread` להפסיק דרך `stop_streaming.set()`, ממתינה שה-thread יסתיים (אם קיים), ואז סוגרת את החיבור דרך `self.conn.close()`.

תיאור המחלקה: Guest

מחלקת Guest מייצגת לקוח שטרם התחבר למערכת כמשתמש מזוהה. היא יורשת את המחלקה Worker ומאפשרת ללקוח לבצע התחברות על בסיס שם משתמש וסיסמה. לאחר התחברות מוצלחת, Guest מוחלף באובייקט מתאים – Employee או Manager – בהתאם לתפקידו של המשתמש.

מאפיינים

clients: מילון שמכיל את כל החיבורים הפעילים לשרת.
כל מפתח הוא socket של לקוח, והערך הוא tuple שמכיל כתובת IP ואת האובייקט המייצג את המשתמש (אחרי התחברות).

פעולות

`__init__(self, conn, addr, clients, server, key=None)`

אתחול המחלקה. מקבלת את החיבור (conn), כתובת הלקוח (addr), מילון החיבורים הפעילים (clients), מופע השרת (server), ואופציונלית מפתח הצפנה (key). הפעולה קוראת לאתחול של המחלקה האב (Worker) ושומרת את ההפניה למילון הלקוחות.

`handle_command(self, msg)`

מטפלת בפקודות שמגיעות מהלקוח. מקבלת מחרוזת msg שיכולה לכלול פקודות שונות, כאשר הפקודה המרכזית היא login/. אם הפקודה היא login/ והיא כוללת בדיוק שלושה חלקים – שם משתמש וסיסמה – הפעולה מוודאת את נכונות פרטי ההתחברות מול בסיס הנתונים (`self.db.authenticate()`). אם ההתחברות מצליחה – נשלחת ללקוח ההודעה "login_successful", ונוצר אובייקט חדש מסוג Employee או Manager בהתאם לתפקיד. האובייקט החדש מחליף את Guest במילון הלקוחות הפעילים. אם הסיסמה שגויה – נשלחת ההודעה "invalid_creds". אם חסרים פרמטרים – נשלחת ההודעה "not_enough_args". אם הפקודה אינה מוכרת, נשלחת הודעת שגיאה.

תיאור המחלקה: StatusCodes

אורי אלפי | Proccesors

מחלקת StatusCodes היא IntEnum שמייצגת קודי סטטוס בתקשורת בין רכיבי המערכת.
SUCCESS = 0 – פעולה הצליחה.
DISCONNECTED = 1 – המשתמש או החיבור נותק.

תיאור המחלקה: Role

מחלקת Role היא IntEnum שמגדירה את התפקיד של משתמש במערכת.
EMPLOYEE = 0 – עובד רגיל.
MANAGER = 1 – מנהל.

תיאור המחלקה: WorkerDC

מחלקה מסוג dataclass שמייצגת את נתוניו הבסיסיים של עובד.
שדות המחלקה:

id (int): מזהה ייחודי של העובד.

username (str): שם המשתמש של העובד.

role (Role): תפקיד המשתמש – עובד או מנהל.

salary (int): שכר העובד.

המחלקה נועדה לייצוג נוח של אובייקטים מסוג "עובד" בתור נתונים בלבד (Data Container), עם אפשרות השוואה, המרה למחרוזת, ושימוש בפשטות במבני נתונים.

תיאור המחלקה: LoginScreen

מחלקת LoginScreen אחראית על יצירת ממשק גרפי (GUI) המאפשר למשתמש להזין שם משתמש וסיסמה ולבצע התחברות למערכת. הממשק בנוי עם הספרייה customtkinter ומבצע שליחה של בקשת התחברות לשרת, הצגת הודעות שגיאה רלוונטיות בהתאם לתשובת השרת, וניהול מצב התוכנית (מוצלח, כישלון או ניתוק). היא פועלת כחלק מתהליך התחברות לקוח במערכת.

מאפיינים

status: מייצג את סטטוס ההתחברות (StatusCodes.DISCONNECTED כברירת מחדל).
alive: בוליאני שמציין האם הממשק פעיל (true כאשר הממשק פתוח).
login_successful: בוליאני שמציין האם ההתחברות הצליחה.
client: אובייקט socket – משמש לשליחת בקשות התחברות לשרת.
root: חלון ה-GUI הראשי של tkinter.
frame: מסגרת פנימית המכילה את שאר רכיבי ה-GUI.
username_entry, password_entry: שדות להזנת שם משתמש וסיסמה.
feedback_label: תווית המשמשת להצגת הודעות שגיאה או תגובות מהשרת.
login_button: כפתור התחברות.

פעולות

__init__(self, client_socket)

מקבלת את פרמטר client_socket שהוא החיבור של הלקוח לשרת. הפעולה מאתחלת את ממשק המשתמש הגרפי, מגדירה עיצוב כהה, יוצרת את כל רכיבי הממשק (שדות, כפתורים, תגיות), ומגדירה את תגובת המערכת בלחיצה על כפתור או סגירת החלון.

start(self)

מתחילה את הלולאה הגרפית הראשית (mainloop) של tkinter. לאחר סגירת החלון, היא מחזירה StatusCodes.SUCCESS אם ההתחברות הצליחה או את הסטטוס הנוכחי (למשל DISCONNECTED) אם לא. מפסיקה את הלולאה וסוגרת את הממשק בצורה בטוחה.

login(self)

אורי אלפי | Proccesors

נשלחת כאשר המשתמש לוחץ על כפתור ההתחברות. קוראת את הערכים מהשדות username_entry ו-password_entry, ויוצרת הודעת התחברות בפורמט login <username>/<password>, ששולחת לשרת דרך socket. אם השליחה נכשלת – מוצגת הודעת שגיאה בממשק.

update_data(self, msg)

מעודכנת על ידי השרת ומעדכנת את הממשק בהתאם. אם ההודעה היא "invalid_creds" – תוצג הודעת שגיאה על פרטים לא נכונים. אם "not_enough_args" – תוצג שגיאה על קלט חסר. אם "login_successful" – תסגור את הממשק ותשנה את הסטטוס. הודעות אחרות יוצגו כטקסט חופשי בתווית.

on_close(self)

מטפלת בסגירת החלון (בלחיצה על X למשל). מגדירה את alive ל-False, מפסיקה את הלולאה הגרפית, וסוגרת את החיבור לשרת.

handleTkError(self, *error)

פונקציה ריקה שמוגדרת כ-handler לחרגות מתוך tkinter – נועדה למנוע הודעות שגיאה גרפיות לא רצויות.

תיאור המחלקה: Log

מחלקת Log מייצגת רישום פעולה שבוצעה על תהליך בשרת, כולל פרטים כמו שם התהליך, צריכת משאבים, עדיפות, סיבת הסיום, המשתמש שביצע את הפעולה, והתזמון שלה. המחלקה משמשת כתיעוד של פעולות שמתבצעות על ידי משתמשים (למשל, סיום תהליך).

מאפיינים

process_name: מחרוזת – שם התהליך עליו בוצעה הפעולה.
cpu: ערך המייצג את שימוש ה-CPU של התהליך (יכול להיות מספר או מחרוזת).
memory: ערך המייצג את השימוש בזיכרון של התהליך.
priority: עדיפות התהליך.
reason: סיבת הפעולה (למשל סיום תהליך) כפי שהוזנה ע"י המשתמש.
user: שם המשתמש שביצע את הפעולה.
time: מחרוזת – התאריך והשעה שבהם הפעולה התבצעה, בפורמט YYYY-MM-DD HH:MM:SS.

פעולות

__init__(self, process_name, cpu, memory, priority, reason, user)

פעולה בונה שמקבלת את פרטי התהליך, הסיבה לפעולה, וזהות המשתמש. בנוסף, יוצרת תיעוד של מועד ביצוע הפעולה (התאריך והשעה הנוכחיים).

to_string(self)

מחזירה מחרוזת שמכילה את כל פרטי הלוג בפורמט קבוע.

תיאור המחלקה: Logs

מחלקת Logs משמשת כמנהל יומני הפעולות (log manager). היא שומרת רשימת לוגים (אובייקטים מסוג Log) ומפיעה את הלוגים הרלוונטיים למנהלים במערכת באמצעות השרת.

מאפיינים

server: הפניה לאובייקט השרת – נדרש לצורך שליחת לוגים למנהלים.
logs: רשימה של מופעי Log – מכילה את כל הלוגים שנרשמו במהלך הריצה.

פעולות

__init__(self, server)

אורי אלפי | Proccesors

יוצרת מופע חדש של Logs. מקבלת את אובייקט השרת (server) כדי שתוכל להשתמש בו לשידור הלוגים.

```
log(self, log: Log)
```

מקבלת אובייקט מסוג Log, מוסיפה אותו לרשימת הלוגים (self.logs), ואז קוראת לפונקציה broadcast_log כדי לשדר אותו למנהלים.

```
broadcast_log(self, log: Log)
```

שולחת את הלוג שניתן לה לכל המנהלים במערכת על ידי קריאה ל-self.server.broadcast_managers עם המחרוזת של הלוג (log.to_string()).

תיאור המחלקה: ManagerScreen

מחלקת ManagerScreen אחראית על בניית והפעלת ממשק גרפי עבור משתמש בתפקיד מנהל. הממשק כולל ניהול עובדים (הוספה, עריכה, שינוי שכר, פיטורים), חיפוש עובדים, והצגת לוגים של תהליכים שנמחקו. הממשק נבנה בעזרת customtkinter ומבצע תקשורת עם השרת דרך socket.

מאפיינים

client: החיבור של המנהל לשרת (socket) – משמש לשליחת פקודות ניהול.

username: שם המשתמש של המנהל המחובר.

alive: בוליאני – מציין אם הממשק פעיל.

workers_data_: מילון של כל העובדים שמוצגים – מפתח הוא id והערך הוא WorkerDC.

worker_widgets_: מילון שמכיל את רכיבי ה-GUI של כל עובד בטבלה.

log_initialized_: מציין אם הלוג הראשון כבר הוזן למסך.

root: חלון ה-GUI הראשי.

add_worker_window, confirmation_window, edit_worker_window: חלונות popup לניהול עובדים.

search_var, search_entry: שדה חיפוש עובדים לפי שם.

worker_scroll: אזור גלילה להצגת העובדים.

log_scroll: אזור גלילה להצגת לוגים של תהליכים שנמחקו.

פעולות עיקריות

```
__init__(self, client_socket=None, username=None)
```

יוצרת את ממשק הניהול, מקבלת socket ו-username, בונה את ה-GUI, ומכינה משתנים פנימיים לאחסון עובדים ולוגים.

```
start(self)
```

מתחילה את הלולאה הגרפית של הממשק (root.mainloop()).

```
build_ui(self)
```

בונה את הממשק: פאנל שמאלי לחיפוש וניהול עובדים, פאנל ימני ללוגים, כולל הגדרות עיצוב, אזור גלילה וכפתורים לפעולות.

```
populate_worker_list(self, filter_text="")
```

מרעננת את רשימת העובדים בהתאם לטקסט חיפוש. מוסיפה, מעדכנת או מסירה רכיבי GUI בהתאם למידע החדש.

```
_create_worker_widget(self, worker: WorkerDC)
```

יוצרת תצוגה גרפית לעובד מסוים עם כפתורים להעלאת שכר, הפחתת שכר, עריכה ופיטורין.

```
_update_worker_widget(self, worker: WorkerDC)
```

מעדכנת את המידע הגרפי המוצג על העובד (למשל שם משתמש או שכר חדש).

```
update_worker_list(self, event=None)
```

אורי אלפי | Proccesors

מופעל כאשר המשתמש מקליד בשדה החיפוש. קורא ל־`populate_worker_list` עם הטקסט החדש.

פעולות לניהול עובדים

```
_open_add_worker_window_handler(self)
open_add_worker_window(self)
```

פותח חלון להוספת עובד חדש ומטפל בשליחת הפקודה לשרת.

```
_open_edit_worker_window_handler(self, worker_id)
open_edit_worker_window(self, worker: WorkerDC)
```

פותח חלון עריכה לעובד קיים – שינוי שם, תפקיד ושכר.

```
_open_confirmation_raise_handler, _apply_raise,
_open_confirmation_lower_handler, _apply_lower,
_open_confirmation_fire_handler, _confirm_fire
```

פותחים חלונות אישור עבור כל פעולה (העלאה, הורדה, פיטורים) ומבצעים את העדכון בפועל דרך שליחת פקודות לשרת.

```
add_log_entry(self, process, cpu, memory, priority, reason, worker, time)
```

מוסיפה לוג של תהליך שנמחק לתוך הפאנל הימני של הממשק.

```
update_data(self, data)
```

מקבלת נתונים מהשרת – אם זה תהליך מסוג `PROCESS_STREAM`, יעדכן את רשימת העובדים. אם זה `LOG`; יוסיף רשומת לוג למסך.

```
open_confirmation_window(self, title, message, on_confirm)
```

פותחת חלון אישור עם כפתורי "Yes" / "No", ומבצעת פעולה רק אם המשתמש מאשר.

```
handle_tk_error(self, *error)
```

מטפל בשגיאות של `tkinter` – ריק כברירת מחדל.

תיאור המחלקה: **Manager**

מחלקת `Manager` מייצגת משתמש בעל תפקיד מנהל במערכת. היא יורשת את המחלקה `Worker`, ומרחיבה אותה עם יכולות ניהול מתקדמות כגון פיטורין, שינוי תפקיד או שכר של עובדים, הוספת משתמשים חדשים ושליחת נתוני עובדים בזמן אמת ללקוח. כל הפקודות מתבצעות דרך `socket` בתקשורת מול השרת.

מאפיינים

`username`: מחרוזת – שם המשתמש של המנהל המחובר.

`logs`: הפניה ליומן הפעולות של השרת (`server.logs`) – משמש לתיעוד פעולות כמו סיום תהליך.

`stop_streaming`: אובייקט מסוג `threading.Event` – משמש לאותת לפונקציית הסטרימינג מתי להפסיק.

`streaming_thread`: מופע `thread` – משמש לשליחת נתוני עובדים ללקוח בזמן אמת.

`alive`: בוליאני – מצוין אם החיבור פעיל.

פעולות

```
__init__(self, conn, addr, username, server, key=None)
```

יוצרת מופע של מנהל. מקבלת (`conn`) (`socket`), כתובת לקוח (`addr`), שם משתמש (`username`), הפניה לשרת (`server`) ואופציונלית מפתח הצפנה (`key`). הפעולה קוראת לאתחול המחלקה האב ומאתחלת את משתני הניהול.

```
handle_command(self, msg)
```

מטפלת בפקודות שמתקבלות מהלקוח.

`role/` – שולפת את התפקיד של המשתמש ושולחת חזרה.

אורי אלפי | Proccesors

username/ – מחזירה את שם המשתמש.
<fire <user_id/ – מוחקת את המשתמש ממסד הנתונים וסוגרת את החיבור אם המשתמש עדיין מחובר.
<salary <user_id> <amount/ – מעדכנת את השכר של משתמש.
<change_username <user_id> <new_username/ – משנה את שם המשתמש.
<change_role <user_id> <role/ – משנה את תפקיד המשתמש (0 = עובד, 1 = מנהל).
<add_user <username> <password> <role> <salary/ – מוסיפה משתמש חדש למערכת.
startstreaming/ – מתחילה שליחת נתונים בזמן אמת על כל העובדים.
אם הפקודה לא מזוהה או לא מתחילה ב-/ , תישלח הודעת שגיאה.
streamer(self)
נשלחת ברקע (thread) כל עוד stop_streaming לא הופעל. כל 5 שניות מבוצעת שליפה של כל העובדים (get_all_employees), והמידע נשלח ללקוח בפורמט JSON דרך הסוקט.
close(self)
עוצרת את פעולת הסטרימינג (אם פעילה) באמצעות stop_streaming.set() וסוגרת את החיבור ללקוח (conn.close()).

תיאור המחלקה: Processes

המחלקה Processes אחראית על ניתוח תהליכים (Processes) פעילים במערכת ההפעלה בזמן אמת, כולל שליפת מידע, עיבודו לפורמט JSON, קבלת פרטים על תהליך ספציפי, וביצוע פעולת סיום (terminate) לפי מזהה תהליך (PID). המחלקה עושה שימוש בספריית psutil ואינה שומרת מצב פנימי, כך שכל פעולה מייצרת את הנתונים שלה לפי הצורך.

פעולות

get_processes_json(self)
פעולה זו אינה מקבלת פרמטרים, והיא מחזירה מחרוזת JSON המכילה מידע על עד 15 תהליכים בעלי השימוש הגבוה ביותר ב-CPU, כולל שם התהליך, שימוש ב-CPU, בזיכרון, ו-priority, תוך התעלמות מתהליכים חסומים או זומבים.
terminate_process(self, pid, reason)
מקבלת שני פרמטרים: pid שהוא מזהה התהליך שיש לסיים, ו-reason שהיא מחרוזת המתארת את סיבת הסיום, ומנסה לסיים את התהליך באמצעות psutil, תוך הדפסת תוצאה (הצלחה או שגיאה) למסוף.
get_process_data(self, pid)
מקבלת pid – מזהה תהליך, ומחזירה מילון המכיל את שם התהליך, שימוש בזיכרון (ב-MB), שימוש נוכחי ב-CPU ו-priority, או None במקרה של שגיאה או חוסר גישה לתהליך.

תיאור המחלקה: Server

המחלקה Server מייצגת את צד השרת של המערכת, ואחראית על ניהול חיבורים מלקוחות, ניתוב פקודות לפי סוג המשתמש (Guest, Manager), שליחת והפצת הודעות, שמירה על רשימת לקוחות מחוברים, וניהול יומן הפעולות. היא משתמשת ב-TCP socket, מאזינה לחיבורים נכנסים ומטפלת בכל לקוח דרך thread נפרד.

מאפיינים

host: מחרוזת המייצגת את כתובת ה-IP שהשרת יאזין עליה, ברירת מחדל היא '127.0.0.1'.
port: מספר שלם המייצג את הפורט שבו השרת יאזין לחיבורים נכנסים, ברירת מחדל היא 65432.
clients: מילון שבו כל מפתח הוא socket פעיל, והערך הוא טופל המכיל כתובת ו-אובייקט משתמש (Guest/Manager/Employee).

אורי אלפי | Proccesors

server: מופע של socket TCP שמאזין לחיבורים נכנסים.
logs: מופע של Logs, המנהל את יומן הפעולות של כל המשתמשים במערכת.
פעולות

```
__init__(self, host='127.0.0.1', port=65432)
    יוצרת מופע חדש של שרת TCP המאזין בכתובת ובפורט הנתונים, מאתחלת את מילון הלקוחות
    ואת מערכת הלוגים, ואינה מקבלת חיבורים עדיין.

start(self)
    מתחילה להאזין לחיבורים נכנסים בלולאה אינסופית, וכל חיבור חדש מועבר לטיפול בפונקציה
    handle_client ב-thread נפרד.

handle_client(self, conn, addr)
    מקבלת חיבור (conn) וכתובת (addr), יוצרת עבורו אובייקט Guest, ומאזינה לפקודות שנשלחות
    ממנו עד שיתבצע ניתוק, עיבוד הפקודות כולל קלט של מפתח הצפנה, בקשות /quit, וקריאה
    ל-handle_command.

broadcast(self, message, sender_conn)
    שולחת הודעה (message) לכל הלקוחות פרט לשולח (sender_conn), ונעשית תוך טיפול
    בתקלות וחיבורים לא תקינים.

broadcast_managers(self, message)
    שולחת הודעה (message) רק לכל הלקוחות שמחוברים כ-Manager, דרך סינון לפי סוג האובייקט
    במילון clients.

disconnect(self, conn)
    מבצעת ניתוק של לקוח לפי החיבור (conn), סוגרת את החיבור ומסירה אותו ממילון הלקוחות.

send_message(self, conn, message)
    שולחת מחרוזת (message) ללקוח דרך (conn socket), לאחר שהיא מומרת לבייטים ונשלחת עם
    אורך מקדים, ואם מתרחשת שגיאה – מבצעת ניתוק בטוח.
```

תיאור המחלקה: Worker

המחלקה Worker מייצגת משתמש כללי במערכת המחובר לשרת, ומהווה בסיס למחלקות גזירה כמו Employee, Manager, ו-Guest. היא שומרת את החיבור ללקוח, הפניה לשרת, מפתח הצפנה ציבורי, ואת ממשק הגישה למסד הנתונים. תפקידה הוא לספק ממשק בסיסי לשליחת הודעות מוצפנות ללקוח ולטיפול סביר כברירת מחדל בפקודות.

מאפיינים

server: הפניה לאובייקט השרת שאליו מחובר המשתמש, משמש למשלוח הודעות והפעלת פעולות כלליות.

conn: חיבור ה-socket בין השרת למשתמש הנוכחי, דרכו מועברות הודעות.

addr: כתובת ה-IP והפורט של המשתמש המחובר – מיועד לזיהוי ולניהול.

key: מפתח הצפנה ציבורי בצורת tuple (e, n) – משמש להצפנת הודעות outgoing.

db: מופע של המחלקה Database – משמש לקריאת וכתובת מידע לבסיס הנתונים (כמו אימות, תפקידים וכו').

פעולות

```
__init__(self, conn, addr, server, key=None)
    מקבלת את חיבור ה-conn (socket), כתובת הלקוח (addr), הפניה לשרת (server), ומפתח
    הצפנה אופציונלי (key), ומאתחלת את אובייקט הגישה למסד הנתונים.

send_message(self, message)
```


אורי אלפי | Proccesors

מקבלת מחרוזת message, מצפינה אותה באמצעות encrypt וה-key הציבורי של הלקוח, ושולחת אותה ללקוח דרך פונקציית השליחה של השרת.

handle_command(self, command)

מקבלת מחרוזת command ומחזירה את הטקסט "Worker cannot handle commands".
פעולה זו נועדה להיגזר ולהיות מוחלפת במחלקות כמו Manager או Employee.

close(self)

סוגרת את חיבור ה-socket של המשתמש, ובכך מנתקת אותו מהשרת.

קטעי קוד האלגוריתמים המרכזיים

כל משתמש יכול לבצע רק את הפקודות המותרות לו. ויש לו מסך בהתאם לתפקיד שלו
מחלקת Worker שכל סוגי התפקידים יורשים ממנה

```
class Worker:
    def __init__(self, conn, addr, server, key=None):
        self.server = server
        self.conn = conn
        self.addr = addr
        self.key = key
        self.db = Database()

    def send_message(self, message):
        encrypted_message = encrypt(self.key, message)
        self.server.send_message(self.conn, encrypted_message)

    def handle_command(self, command):
        return "Worker cannot handle commands."

    def close(self):
        self.conn.close()
```

מחלקת LoginScreen:

```
def start(self):
    self.alive = True
    self.root.mainloop()
    try:
        self.root.destroy()
    except:
        pass
    if self.login_successful:
        return StatusCodes.SUCCESS
    return self.status

def login(self):
    username = self.username_entry.get()
    password = self.password_entry.get()
    message = f"/login {username} {hash(password)}"
    try:
        self.client.sendall(message.encode())
    except:
        self.display_server_message("Failed to send login request.")

def update_data(self, msg):
    if not self.alive or not self.root.winfo_exists():
```

```

        return

    try:
        def update_ui():
            if not self.alive or not self.root.wininfo_exists():
                return

            if msg == "invalid_creds":
                self.feedback_label.configure(text="Invalid username or password.")
            elif msg == "not_enough_args":
                self.feedback_label.configure(text="Not enough arguments provided.")
            elif msg == "login_successful":
                self.status = StatusCodes.SUCCESS
                self.login_successful = True
                self.alive = False

```

מחלקת LoginScreen קוראים לה בClient

```

login_gui = LoginScreen(self.client)
threading.Thread(target=self.receive, args=([login_gui]), daemon=True).start()
login_result = login_gui.start() # This will now be None if closed normally

if login_result == StatusCodes.DISCONNECTED:
    print("[DISCONNECTED] Connection closed.")
    self.client.close()
    sys.exit()

elif login_result == StatusCodes.SUCCESS:
    self.client.sendall("/role\n".encode())

    msg = self.receive_message(self.client)
    role = int(msg[-1])

    self.client.sendall("/username\n".encode())

    msg = self.receive_message(self.client)
    username = msg

    if role == Role.EMPLOYEE:
        self.employee_gui = EmployeeScreen(self.client, username) # Create EmployeeScreen
instance
        threading.Thread(target=self.receive, args=([self.employee_gui]),
daemon=True).start()
        self.client.sendall("/startstreaming\n".encode())
        self.employee_gui.start() # Start the EmployeeScreen's main loop
    elif role == Role.MANAGER:
        self.manager_gui = ManagerScreen(self.client, username)
        threading.Thread(target=self.receive, args=([self.manager_gui]),
daemon=True).start()
        self.client.sendall("/startstreaming\n".encode())
        self.manager_gui.start()

```

לקטע קוד הכי חשוב, ברגע שלמשתמש אין תפקיד, הוא נחשב ממחלקת Guest, ובמחלקה הזאת לפי מסד הנתונים האתחול של העובד נוצר לרשימת לקוחות של השרת וזה בחירה בין Employee לManager.

```

def handle_command(self, msg):
    args = msg.split(' ')
    cmd = args[0]
    if msg.startswith('/'):
        try:
            if cmd == "/login":
                if len(args) == 3:

```

אורי אלפי | Processors

```
username = args[1]
password = args[2]
if self.db.authenticate(username, password):
    self.send_message("login_successful")
    role = self.db.get_role(username)
    if role == Role.EMPLOYEE:
        self.clients[self.conn] = (self.addr, Employee(self.conn, self.addr,
username, self.server, self.key))
    elif role == Role.MANAGER:
        self.clients[self.conn] = (self.addr, Manager(self.conn, self.addr,
username, self.server, self.key))
```

הצגת תהליכים ללקוחות

הלקוח מבקש להתחיל את שליחת התהליכים

```
if role == Role.EMPLOYEE:
    self.employee_gui = EmployeeScreen(self.client, username) # Create EmployeeScreen
instance
    threading.Thread(target=self.receive, args=([self.employee_gui]),
daemon=True).start()
    self.client.sendall("/startstreaming\n".encode())
    self.employee_gui.start() # Start the EmployeeScreen's main loop
```

השרת במחלקת Employee קולט את הפקודה

```
elif cmd == "/startstreaming":
    if self.streaming_thread is None or not self.streaming_thread.is_alive():
        self.stop_streaming.clear()
        self.streaming_thread = threading.Thread(target=self.streamer, daemon=True)
        self.streaming_thread.start()

    def streamer(self):
        processes = Processes()
        while not self.stop_streaming.is_set():
            try:
                data = processes.get_processes_json()
                data = "PROCESS_STREAM:" + data
                self.send_message(data)
                time.sleep(5)
            except Exception as e:
                break
```

הוא לוקח את המידע שלו מprocesses

```
def get_processes_json(self):
    # Prime the CPU percent cache
    for proc in psutil.process_iter():
        try:
            proc.cpu_percent(interval=None)
        except (psutil.NoSuchProcess, psutil.AccessDenied):
            continue

    time.sleep(0.1) # Allow CPU measurement window

    process_info = {}

    logical_cpu_count = psutil.cpu_count(logical=True)

    for proc in psutil.process_iter(['pid', 'name', 'memory_info']):
        try:
            pid = proc.pid
            name = proc.name()
            raw_cpu = proc.cpu_percent(interval=None)
            cpu = round(raw_cpu / logical_cpu_count, 2)
```

אורי אלפי | Proccesors

```
memory = round(proc.memory_info().rss / (1024 * 1024), 2) # MB

# Priority handling
try:
    if platform.system() == 'Windows':
        priority = proc.nice() # still numeric but corresponds to priority class
    else:
        priority = proc.nice() # Unix-style nice value
except psutil.AccessDenied:
    priority = "N/A"

process_info[pid] = {
    "pid": pid,
    "process_name": name,
    "cpu": cpu,
    "memory": memory,
    "priority": priority
}
```

מסנן אותם לתהליכים שהכי משתמשים במעבד ושולח

```
process_info = {pid: info for pid, info in process_info.items() if info["process_name"] != ""}

top_processes = sorted(process_info.values(), key=lambda x: x["cpu"], reverse=True)[:15]

return json.dumps({
    p["pid"]: {
        "process_name": p["process_name"],
        "cpu": p["cpu"],
        "memory": p["memory"],
        "priority": p["priority"]
    } for p in top_processes
})
```

בלקוח מקבל את התהליכים ומפרק אותם

```
def update_data(self, data):
    if data.startswith("PROCESS_STREAM:"):
        try:
            data = data[len("PROCESS_STREAM:"):]
            self.processes = json.loads(data)
            self._display_table_data()
        except json.JSONDecodeError:
            print("[ERROR] Failed to decode JSON data.")
```

מציג אותם בצורה חכמה כמו שתיארתי

```
def _display_table_data(self):
    max_rows = 20
    col_widths = [80, 240, 100, 100, 100, 110]

    sorted_processes = list(self.processes.items())[:max_rows]

    while len(sorted_processes) < max_rows:
        sorted_processes.append("", {})

    for row_index in range(max_rows):
        pid, info = sorted_processes[row_index]
        values = [
            str(pid) if pid else "",
            info.get("process_name", "") if pid else "",
            str(info.get("cpu", "")) if pid else "",
            str(info.get("memory", "")) if pid else "",
            str(info.get("priority", "")) if pid else ""
        ]
```

```

if row_index not in self.table_widgets:
    widgets = []
    for col_index, value in enumerate(values):
        label = ctk.CTkLabel(
            self.table_frame,
            text=value,
            font=self.normal_font,
            width=col_widths[col_index]
        )
        label.grid(row=row_index + 1, column=col_index, padx=5, pady=4, sticky="w")
        widgets.append(label)

    btn = ctk.CTkButton(
        self.table_frame,
        text="Terminate",
        width=col_widths[-1],
        command=lambda pid=pid: self.open_terminate_reason_window(pid) if pid else None
    )
    btn.grid(row=row_index + 1, column=5, padx=5, pady=4)
    widgets.append(btn)

    self.table_widgets[row_index] = widgets
else:
    for col_index, (label, value) in enumerate(zip(self.table_widgets[row_index][:-1],
values)):
        label.configure(text=value, width=col_widths[col_index])

    self.table_widgets[row_index][-1].configure(
        command=(lambda pid=pid: self.open_terminate_reason_window(pid)) if pid else
lambda: None,
        width=col_widths[-1]
    )

```

הצגת Logs למנהלים

הלקוח מקבל מהעובד סיבה למחיקה שולח בקשה למחוק

```

def open_terminate_reason_window(self, pid):
    # Popup window for terminate reason
    popup = ctk.CTkToplevel(self.root)
    popup.title(f"Terminate Process {pid}")
    popup.geometry("400x150")
    popup.grab_set() # make modal

    label = ctk.CTkLabel(popup, text="Please enter reason for terminating the process:")
    label.pack(pady=(20, 10))

    reason_entry = ctk.CTkEntry(popup, width=350)
    reason_entry.pack(pady=(0, 20))
    reason_entry.focus()

    def on_confirm():
        reason = reason_entry.get().strip()
        if reason:
            message = f"/terminate {pid} {reason}\n"
        else:
            message = f"/terminate {pid} <No reason provided>\n"
        self.client.sendall(message.encode())

```

אורי אלפי | Proccesors

```
popup.destroy()
```

השרת מוסיף לוג לרשימת הלוג ומוחק את התהליך

```
elif cmd == "/terminate":
    args = msg.split(' ', 2)
    pid = int(args[1])
    reason = args[2]
    process_data = self.processes.get_process_data(pid)
    if process_data != None:
        self.logs.log(
            Log(process_data["name"], process_data["cpu"], process_data["memory"],
                process_data["priority"], reason, self.username)
        )
        self.processes.terminate_process(pid, reason)
```

מחלקת הלוג והלוגים

```
class Logs:
    def __init__(self, server):
        self.server = server
        self.logs: list[Log] = []
        self.lock = threading.Lock()

    def log(self, log: Log):
        with self.lock:
            self.logs.append(log)
            self.broadcast_log(log)

    def broadcast_log(self, log: Log):
        self.server.broadcast_managers(log.to_string())
```

כפי שניתן לראות יש פה שימוש ב `threading.lock` כדי למנוע שימוש כפול שגורם לקריסה/שינוי נתונים שגוי.

```
def add_log_entry(self, process, cpu, memory, priority, reason, worker, time):
    frame = ctk.CTkFrame(self.log_scroll, corner_radius=8, border_width=1)
    frame.pack(fill="x", pady=5, padx=5) # Pack the new frame first

    if self._log_initialized:
        # Repack to be the first widget
        frame.pack_configure(before=self.log_scroll.winfo_children()[0])
    else:
        self._log_initialized = True

    text = (
        f"Process: {process}\n"
        f"CPU Usage: {cpu}      Memory: {memory}      Priority: {priority}\n"
        f"Reason Deleted: {reason}\n"
        f"Deleted by: {worker} \n"
        f"Time: {time}\n"
    )

    label = ctk.CTkLabel(frame, text=text, justify="left", anchor="w")
    label.pack(fill="x", padx=10, pady=5)
```

בצד הלקוח הוא מקבל את זה ומציג בצורה שאמרתי קודם

```
elif data.startswith("LOG:"):
    try:
        log_data = data[len("LOG:"):].split("\n")
        self.add_log_entry(
            process=log_data[0],
            cpu=log_data[1],
            memory=log_data[2],
```

אורי אלפי | Proccesors

```
        priority=log_data[3],
        reason=log_data[4],
        worker=log_data[5],
        time=log_data[6]
    )

    def add_log_entry(self, process, cpu, memory, priority, reason, worker, time):
        frame = ctk.CTkFrame(self.log_scroll, corner_radius=8, border_width=1)
        frame.pack(fill="x", pady=5, padx=5) # Pack the new frame first

        if self._log_initialized:
            # Repack to be the first widget
            frame.pack_configure(before=self.log_scroll.wininfo_children()[0])
        else:
            self._log_initialized = True

        text = (
            f"Process: {process}\n"
            f"CPU Usage: {cpu}      Memory: {memory}      Priority: {priority}\n"
            f"Reason Deleted: {reason}\n"
            f"Deleted by: {worker} \n"
            f"Time: {time}\n"
        )
        label = ctk.CTkLabel(frame, text=text, justify="left", anchor="w")
        label.pack(fill="x", padx=10, pady=5)
```

אלגוריתם יפה בעיני:

בשימוש אורך הודעה הוא מפרק פקטה, ממיר מחרוזות לרשימות של מספרים, ופענוח עם המפתח הפרטי.

```
def receive_message(self, conn):
    raw_msglen = self.client.recv(4)
    msglen = int.from_bytes(raw_msglen, byteorder='big')
    data = b""
    while len(data) < msglen:
        more = conn.recv(msglen - len(data))
        if not more:
            raise EOFError("Socket closed before receiving all data")
        data += more
    data = data.decode()
    ciphered = [int(char) for char in data.split(",") if char.isdigit()]
    decrypted_data = decrypt(self.private_key, ciphered)
    return decrypted_data
```

היכולת: הוספת עובד

בצד של הלקוח:

```
def authenticate(self, username, password):

    cur = self.conn.execute("SELECT salt FROM users WHERE username = ?",
                             (username,))

    row = cur.fetchone()
```

אורי אלפי | Proccesors

```
if not row:
    return False

salt = row[0]

hashed = self._hash_password(salt + password)

cur = self.conn.execute("""
    SELECT * FROM users WHERE username = ? AND password = ?
""", (username, hashed))

return cur.fetchone() is not None
```

בצד של השרת:

```
elif cmd == "/add_user":
    username = args[1]
    password = args[2]
    role = int(args[3])
    if role not in [0, 1]:
        self.send_message("[SERVER] Invalid role. Use 0 for EMPLOYEE
and 1 for MANAGER.\n")
        return
    salary = float(args[4])
    self.db.add_user(username, password, role, salary)
```

```
def add_user(self, username, password, role, salary=0.0):
    if not (0 <= role <= 1):
        raise ValueError("Role must be between 0 and 1.")
    try:
        salt = self._generate_salt()
```


אורי אלפי | Proccesors

```
with self.conn:
```

```
    self.conn.execute("""
```

```
        INSERT INTO users (username, password, salt, role, salary)
```

```
        VALUES (?, ?, ?, ?, ?)
```

```
    """, (username, self._hash_password(salt + password), salt, role, salary))
```

```
    return True
```

```
except sqlite3.IntegrityError:
```

```
    return False # Username already exists
```

היכולת: לשנות את פרטי העובדים בכל דרך שהיא

לקחתי לדוגמה את שינוי שכר

לקוח:

```
def _apply_raise(self, worker_id):
```

```
    if worker_id in self._workers_data:
```

```
        self.client.sendall(f"/salary {worker_id} {self._workers_data[worker_id].salary  
+ 10}\n".encode())
```

```
        self._workers_data[worker_id].salary += 10
```

```
        self._update_worker_widget(self._workers_data[worker_id])
```

שרת:

```
def handle_command(self, msg):
```

```
    args = msg.split(' ')
```

```
    cmd = args[0]
```

```
    if msg.startswith('/')
```

```
        try:
```

```
            if cmd == "/role":
```

```
                role = self.db.get_role(self.username)
```

```
                self.send_message(f"role:{role}")
```

```
            elif cmd == "/username":
```

```
                self.send_message(self.username)
```

```
            elif cmd == "/fire":
```

```
                user_id = int(args[1])
```

```
                username = self.db.get_username(user_id)
```

```
                self.db.del_user(user_id)
```

Proccesors | אורי אלפי

```
clients = [client[1] for client in self.server.clients.values()]
for client in clients:
    if isinstance(client, Manager) or isinstance(client, Employee):
        if client.username == username:
            client.close()
elif cmd == "/salary":
    user_id = int(args[1])
    salary = float(args[2])
    self.db.update_salary(user_id, salary)
```

מסד נתונים:

```
def update_salary(self, user_id, salary):
    with self.conn:
        self.conn.execute("""
            UPDATE users SET salary = ? WHERE id = ?
            """, (salary, user_id))
```

היכולת: לקבל רשימה של העובדים מעודכנת

הלקוח מבקש מהשרת להתחיל לשלוח את רשימת העובדים:

```
elif role == Role.MANAGER:
```

```
    self.manager_gui = ManagerScreen(self.client, username)

    threading.Thread(target=self.receive, args=([self.manager_gui]),
daemon=True).start()

    self.client.sendall("/startstreaming\n".encode())

    self.manager_gui.start()
```

בצד השרת:

```
elif cmd == "/startstreaming":

    if self.streaming_thread is None or not self.streaming_thread.is_alive():

        self.stop_streaming.clear()

        self.streaming_thread = threading.Thread(target=self.streamer,
daemon=True)

        self.streaming_thread.start()
```

```
def streamer(self):
```

אורי אלפי | Proccesors

```
while not self.stop_streaming.is_set():

    try:

        employees = self.db.get_all_employees()

        employees_json = {ID: WorkerDC(ID, username, role, salary) for ID,
username, role, salary in employees}

        employees_json = json.dumps(employees_json, default=lambda o:
o.__dict__)

        data = "PROCESS_STREAM:" + employees_json

        self.send_message(data)

        time.sleep(5)

    except Exception as e:

        break
```

השרת שולח כל 5 שניות, והלקוח מפרק

הלקוח:

```
def populate_worker_list(self, filter_text=""):

    for worker_id, worker in self._workers_data.items():

        if filter_text.lower() in worker.username.lower():

            if worker_id not in self._worker_widgets:

                self._create_worker_widget(worker)

            else:

                self._update_worker_widget(worker)

        elif worker_id in self._worker_widgets:

            # Worker no longer matches the filter, remove its widgets

            for widget in self._worker_widgets[worker_id].values():

                widget.destroy()

            del self._worker_widgets[worker_id]
```

```
# Remove widgets for workers that are no longer in the data

worker_ids_to_remove = [

    w_id for w_id in self._worker_widgets if w_id not in self._workers_data

]

for w_id in worker_ids_to_remove:

    for widget in self._worker_widgets[w_id].values():

        widget.destroy()

    del self._worker_widgets[w_id]
```

היכולת: התחברות למערכת

בצד הלקוח:

```
def start(self):
    try:
        self.client.connect((self.host, self.port))
        print("[CONNECTED] Connected to server.")
    except Exception as e:
        print(f"[ERROR] Could not connect: {e}")
        return

    self.public_key, self.private_key = generate_keypair(8)

    self.client.sendall(f"/key {self.public_key[0]} {self.public_key[1]}".encode())

    login_gui = LoginScreen(self.client)
    threading.Thread(target=self.receive, args=(login_gui), daemon=True).start()
    login_result = login_gui.start() # This will now be None if closed normally
```

מחלקת LoginScreen:

```
def login(self):
    username = self.username_entry.get()
    password = self.password_entry.get()
    message = f"/login {username} {hash(password)}"
    try:
        self.client.sendall(message.encode())
    except:
        self.display_server_message("Failed to send login request.")
```

מחלקת Guest בצד השרת:

```
def handle_command(self, msg):
```

```
    args = msg.split(' ')
```

```
    cmd = args[0]
```

```
    if msg.startswith('/'):

```

```
        try:

```

```
            if cmd == "/login":

```

```
                if len(args) == 3:

```

```
                    username = args[1]

```

```
                    password = args[2]

```

```
                    if self.db.authenticate(username, password):

```

```
                        self.send_message("login_successful")

```

מסד הנתונים בצד השרת:

```
def authenticate(self, username, password):
```

```
    cur = self.conn.execute("SELECT salt FROM users WHERE username = ?",
    (username,))
```

```
    row = cur.fetchone()
```

```
    if not row:
```

```
        return False

```

```
    salt = row[0]
```

```
    hashed = self._hash_password(salt + password)
```

```
    cur = self.conn.execute("""

```

```
        SELECT * FROM users WHERE username = ? AND password = ?

```

```
    """, (username, hashed))
```

```
    return cur.fetchone() is not None

```

בדיקות

בדיקת הצפנה בין לשרת ללקוח (RSA)

המטרה לבדוק האם ההודעות אכן מוצפנות. אשווה את ההודעות הנשלחות והמתקבלות בשרת ובלקוח כדי לוודא התאמה לאחר פענוח. עשיתי Print והשוותי וראיתי ראשית שהייתה לי בעיה ואז הבנתי שהיה לי טעות כתיבה קטנה. ישר סידרתי אותה והמערכת עבדה כמו שצריך. כמובן, היה עוד בעיות. לא הבנתי בהתחלה איך להשתמש בזה אז זה היה ניסוי ותהייה.

בדיקת אבטחת בסיס הנתונים (Hash + Salt)

המטרה לבדוק שהסיסמאות אינן נשמרות כמו שהם במסד נתונים, אלא מועברות עם Hash ו-Salt. אדפיס בשרת את הסיסמה לאחר החישוב ואוודא שאינה תואמת לסיסמה המקורית. בנוסף לכך, השתמשתי בDB browser והוא הראה לי שהמערכת עבדה. בחלק זה לא היו לי בעיות.

בדיקה נגד SQL Injection

המטרה לבדוק האם המערכת אכן מונעת הכנסת קוד דרך שדות קלט. אנסה להזין טקסט זדוני בשדות (כמו 'admin'--) ולוודא שהמערכת חוסמת את הקלט. המערכת אכן חסמה את הקלט. זה עבד פעם ראשונה באופן חלקי, באופן מפתיע.

בדיקה נגד תקיפת DDOS

המטרה לבדוק שהמערכת מזהה ריבוי חיבורים וחוסמת חיבורים חדשים מעל מגבלה. אפעיל כמות לקוחות החורגת מהמותר ואוודא שהחיבורים הנוספים נדחים. בהתחלה, לא הצלחתי להיכנס בכלל כלקוח, ואז הבנתי את הטעות שלי. כמו כל דבר, זה לקח ניסוי ותהיה אבל טוב שבדקתי את זה.

בדיקת התחברות והתחברות לבסיס הנתונים

המטרה לבדוק שההתחברות מבוססת על נתוני אמת שנשמרים ב-Database. אבדוק התחברות, פרטים, שינוי שם משתמש, ואימות מול התחברות מחדש. השתמשתי בהתחלה בPrintx ואז הבנתי כמה זה קל DB browser. הכל עבד כמו שצריך, ברור שהיה ניסוי ותהייה כי כל הנושא של מסד נתונים דיי חדש לי.

בדיקת ניתוק תקין עם סגירת לקוח

המטרה לוודא שברגע שהמשתמש סוגר את הממשק, השרת מזהה זאת. אבדוק שהשרת מדפיס "Disconnected" ומנתק את הלקוח. לקח כמה זמן כי רשתות זה דבר מאוד מסובך וארוך. אחרי שישבתי כמה ימים להבין את הבעיה שהייתה בשרת באופן כללי, התוכנית עבדה כולל זה, בעזרת Daemon לרשימת תהליכונים של הלקוחות.

בדיקת הרשאות למחיקת תהליך בשרת

המטרה לבדוק שהמשתמש מסוגל למחוק תהליך רלוונטי במערכת הפעלה מרחוק. בפועל, לא היה צורך לבדיקה הזאת. התכנית נבניתה לזה שאין גישה לפעולות שהן לא של סוג העובד שלך, והבנתי את זה תוך כדי כתיבת הקוד.

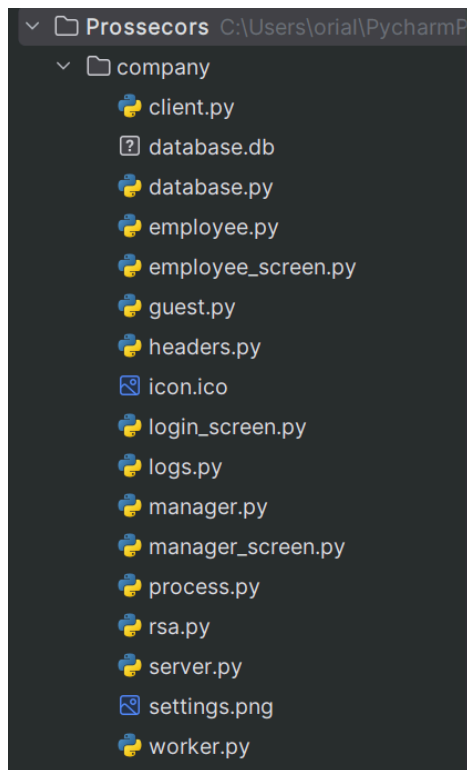
בדיקת אמינות תצוגת משאבי התהליכים מהשרת

המטרה לבדוק שהנתונים שמוצגים על שימוש ב-CPU/זיכרון אכן משקפים את המציאות. אשווה בין הנתונים של המערכת שלך לבין מה שמוצג ב-Task Manager של Windows. בהתחלה, ראיתי שנתוני המעבד לא תואמים. חיפשתי תשובות למה זה ככה, ואז גיליתי את הטעות שלי. לאור הבדיקה הזאת, העלתי את "זמן הדגימה" של התהליכים, משמע הזמן שהוא מקליט את הערכים שלהם. זה תיקן לי את זה, ויצא ערכים יותר מדויקים. לא רציתי להגיע לערכים יותר מדויקים עם זמן דגימה גדול יותר בגלל שהדבר יוצר עומס באופן לא יחסי. הגעתי למסקנה שזה בסדר האי דיוק הזה, כי הוא קטן ועדיף לאזן בין שימוש התוכנה לדיוק המשאבים.

בדיקה לא מתוכננת: בדיקת הלוגים בעת מחיקת תהליך

המטרה היא לבדוק שמערכת הלוגים מופעלת במחיקת תהליך, שהמידע נשמר, נשלח למנהלים, ומוצג בממשק משתמש. אפתח תוכנה על השרת שתשתמש במעבד הרבה, ואז מתוך ממשק המשתמש של העובד במחשב אבצע מחיקה של אותו תהליך. לאחר מכן אצפה בצד של מנהל, ואבדוק האם פרטי התהליך שנמחק (שם, שימוש CPU, זיכרון, עדיפות, סיבה למחיקה, מי מחק ומתי) מוצגים. בהתחלה, הדבר לא עבד לי. היה בעיה בקוד, והחיבור לא עבד טוב. הבנתי אחרי כמה ימים שאין צורך להשתמש בשליחה מחדש של כל log, אלא כל פעם לשלוח כשזה קורה לכולם. אחרי הרבה שינויים וטעויות מזעריות, הצלחתי לגרום למערכת לעבוד, והוספתי עוד threading.lock לרשימה שלא יוצר מצב של "התנגשות" בשינוי הנתונים, שעלול להוביל במקרה הטוב לנתונים שגויים.

מדריך למשתמש עץ קבצים



סביבת עבודה, מה צריך:

1. מערכת הפעלה: Windows 10 ומעלה (רצוי), או כל מערכת שתומכת ב-Python ו-Tkinter
2. גרסת פייתון מעודכנת
3. חבילות pip מותקנות. הקלידו את זה לתוך command prompt
`pip install customtkinter pillow numpy psutil`
4. מומלץ: סביבת עבודה כמו PyCharm

נתונים התחלתיים:

בגלל שלמנהל יש את כל הגישה להוסיף עובדים, צריך להריץ את `database.py`. אחר שהורץ, יש משתמש מנהל שאת פרטיו אתן בשביל שמוריד המערכת ידע. שם משתמש "Admin". סיסמא "admin". לפני שמריצים את השרת, יש לעשות את מה שרשמתי בחלק "משתמשי המערכת".

רשת:

בשביל שהפרויקט יעבוד, המחשבים חייבים להיות מחוברים לאותה רשת.

דרישות נוספות:

1. יש לוודא שהמחשב או המחשבים שמהם מריצים את קוד המערכת מחוברים לאינטרנט.
2. אסור לשנות את מיקומם/שמם של הקבצים הנמצאים בתיקייה הראשית.

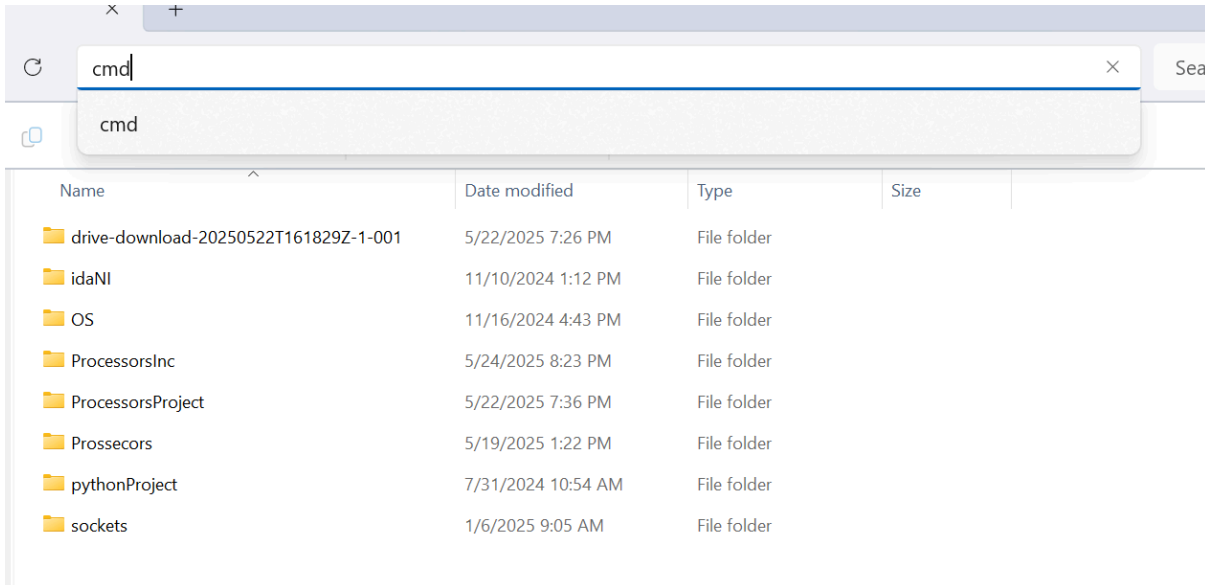
ארכיטקטורה מינימלית

משתמשי המערכת

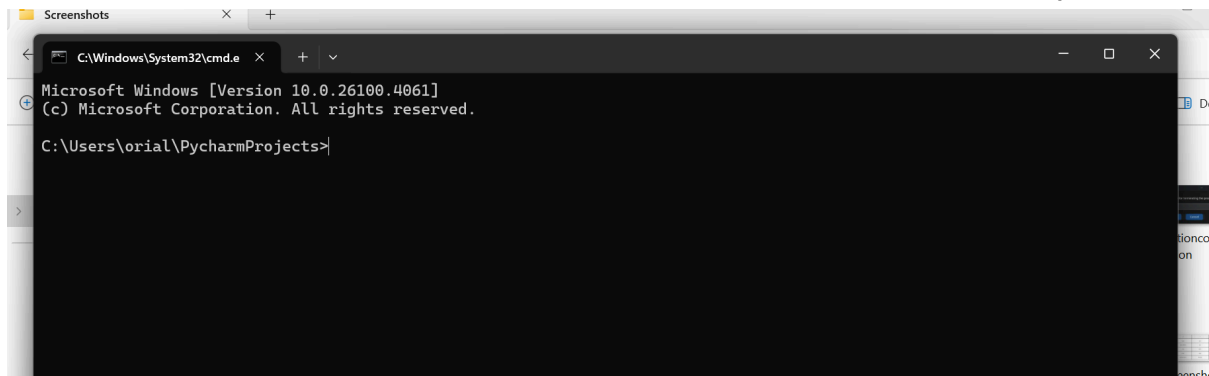
למנהל המערכת

Proccesors | אורי אלפי

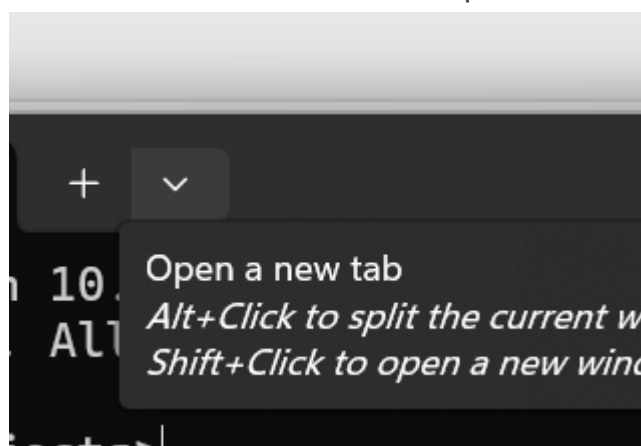
כדי להריץ את הפרויקט, תחילה יש להריץ את המסד נתונים שיתווסף עובד, ואז הקובץ py.server ממחשב השרת דרך cmd של admin. איך עושים זאת?
1. נכסים לאיפה שהפרויקט שמור ולכתוב cmd



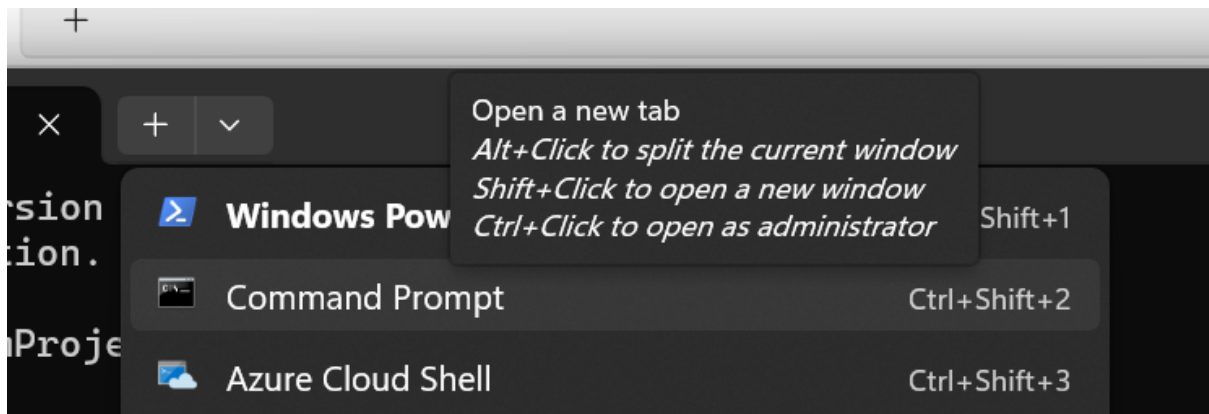
זה יפתח מסך כזה



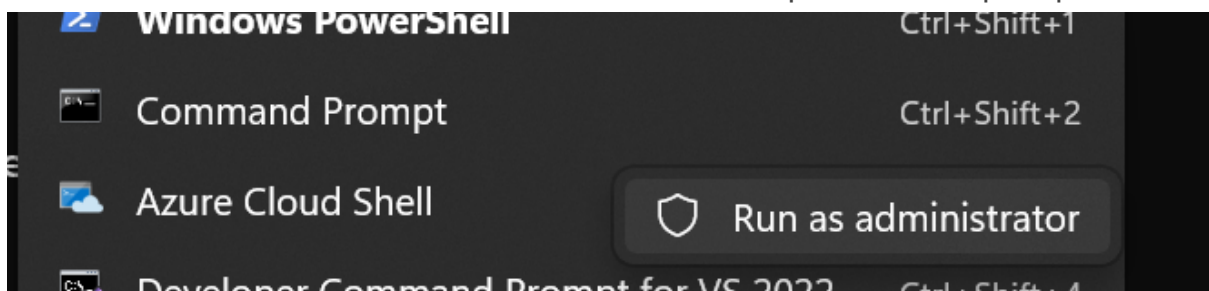
לוחצים על החץ למעלה



זה פותח את זה



לוחצים על command prompt מקש ימני וזה פותח את האפשרות "run as administrator"



עכשיו מקלידים לתוך המסך שנפתח

cd "המקום ששמור הפרויקט"

cd "שם הפרויקט"

py [server.py](#)

עכשיו אחרי כל זה אפשר להריץ [client.py](#) והתחבר למשתמש שהזכרתי בחלק של נתונים התחלתיים.

Guest – משתמש טרם החיבור (כל משתמש)

כשמפעילים את התוכנה בפעם הראשונה, נפתח חלון התחברות שבו ניתן להזין שם משתמש וסיסמה. על המשתמש להזין אותם. המשתמש במצב הזה עדיין אינו מזוהה – ולכן הוא אורח. ברגע שהפרטים מוזנים ולוחצים על כפתור ההתחברות, המערכת מעבירה את המשתמש בהתאם לתפקידו. אם הפרטים שגויים, תופיע הערה מתאימה. אורח יכול להיות או מנהל או עובד. רשמתי כל סוג מהם מדריך.

Processors

עובד – Employee

לכשהעובד מתחבר בהצלחה, המערכת מציגה במסך חדש את כל 15 התהליכים שהכי משתמשים במעבד בשרת. הוא יכול לראות מידע עליהם כמו שם של תהליך, השימוש במעבד ובזיכרון, עדיפות ועוד. אם העובד מזהה תהליך שצריך למחוק, הוא יכול ללחוץ על כפתור "Terminate" ליד אותו תהליך, להקליד סיבה ולשלוח. אין צורך לרענן את המסך – הוא מתעדכן לבד כל כמה שניות.

Hello User

You got a lot of work today!

Active Processes on the server

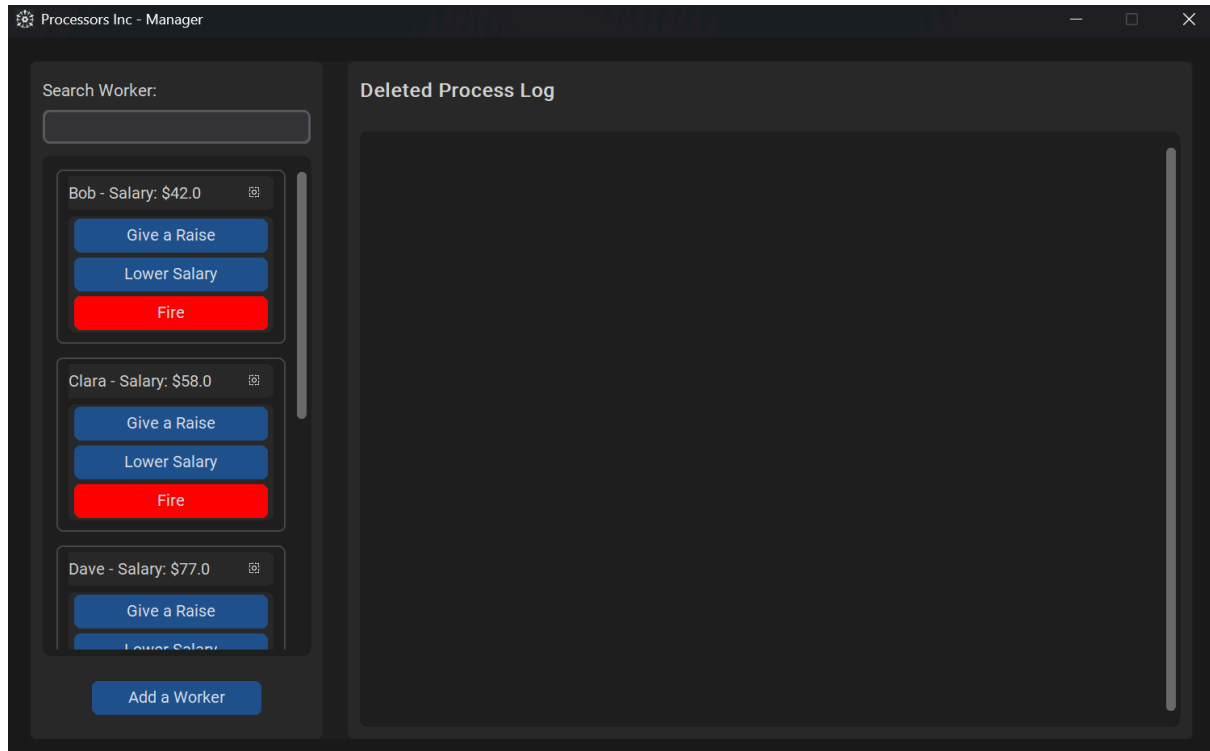
PID	Process Name	CPU %	Memory	Priority	Action
22028	python.exe	11.38	23.2	32	<button>Terminate</button>
15764	chrome.exe	3.34	386.68	32	<button>Terminate</button>
11848	dwm.exe	1.5	78.49	N/A	<button>Terminate</button>
5432	python.exe	1.07	40.0	32	<button>Terminate</button>
4	System	0.75	0.79	N/A	<button>Terminate</button>
7324	explorer.exe	0.66	306.07	32	<button>Terminate</button>
14756	chrome.exe	0.65	5.88	32	<button>Terminate</button>
7364	audiodg.exe	0.57	16.82	32	<button>Terminate</button>
9872	svchost.exe	0.55	15.27	N/A	<button>Terminate</button>
20492	ctfmon.exe	0.34	12.2	128	<button>Terminate</button>
5256	svchost.exe	0.28	12.03	N/A	<button>Terminate</button>

מנהל – Manager

לכשמנהל מתחבר למערכת, הוא מועבר לחלון ניהול. במסך הזה הוא יכול לראות את כל העובדים, לחפש אותם לפי שם, לשנות את הפרטים שלהם, כמו שם משתמש, תפקיד או שכר, או אפילו לפטר אותם. בנוסף, יש למנהל אפשרות להוסיף עובדים חדשים על ידי מילוי פרטים שלהם. בצד ימין של

אורי אלפי | Processors

המסך, המנהל רואה log של כל התהליכים שנמחקו בשרת – כולל שם התהליך, הסיבה למחיקה, מי מחק אותו ומתי.



רפלקציה

תיאור תהליך העבודה על הפרויקט, ההצלחות, האתגרים, הקשיים ודרכי הפתרון

ראשית כל, אסכם את תהליך העבודה בפרויקט. אתגר שלא הכינו אותו מראש אליו שדורש מאמצים רבים ועזרה מאחרים תוך שיתוף הלמידה בין חברי המגמה. באופן אישי, לדעתי לא הייתי יכול להגיע לאותו רמה בפרויקט לולא חברים ואבא שלי שעזרו לי. אני מרגיש שלא הייתה למידה מספיק משמעותית במגמה לפרויקט, ויש נושאים שאני פשוט למדתי מאפס. אני חושב שההצלחות שלי היו בעבודה שהיא מסודרת לעומת אחרים, למדתי המון וזה פעם ראשונה באמת שאני עושה פרויקט בעצמי בסדר גודל כזה. עמדתי בזמנים באופן מפתיע ולא היה לי את השבוע המלחיץ שיש לחברים שלי למגמה שהם עדיין לא סיימו את הפרויקט ויש להם עוד ספר פרויקט לכתוב. כן לדעתי הייתי יכול לעשות את חלק מהדברים יותר טוב אם הייתי עובד אפילו יותר טוב עם הזמנים, למשל הצפנה דו כיוונית. לדעתי, ההצלחה הכי גדולה היא שזה בנה לי רצון לעשות פרויקט בעצמי בדרך שלי ולא בדרך של מחוון עם דרישות. אני רוצה לעשות משהו שאני אתעניין באמת בלהכין אותו בדרך שלי ולא בדרך מוכתבת על ידי מורי המגמה. אתגרי הפרויקט זה בעיקר ללמוד הכל מאפס. זרקו אותנו למים ואנחנו היינו צריכים לבוא עם פרויקט גמור. הקושי בזה הוא לשבת ולא להתייאש וללמוד איך ולמה לכתוב חלק בפרויקט. הפתרון של הקושי הזה הוא להבין שלא הכל חייב להיות מושלם, ועם הזמן תמיד אפשר לשפר חלקים. עשיתי את זה בהרבה חלקים בפרויקט, וכשלא הצלחתי פניתי לעזרה מחברים ומאבא שלי שגם בעצמו מתעסק בתחום. אתגר נוסף בפרויקט הוא עבודה רציפה במשך השנה, גם בתקופות מבחנים. זה לאמר לעצמך שאתה "תעשה את הפרויקט אחרי הלחץ" ואז להגיע לשלב האחרון ואין לך פרויקט גמור. אני ראיתי בזה קושי רב כי כל הזמן היה לי לחץ. לא היה תקופה בשנה שהייתי רגוע. הפתרון לזה זה עבודה של שעה-שעה וחצי שבוע, כי זה מה שצריך

לאורך זמן. אוסיף שלדעתי במגמה לא מספיק ישבו עלינו עם תאריכים ודדליינים, שלחלק האנשים יצר מצב שאין להם פרויקט בשבוע ההגשה למשרד החינוך. לעבוד עם דדליינים זה הפתרון. תהליך למידה - תיאור הלמידה שהתקיימה, תכולות חדשות שנלמדו באופן עצמאי

אילו כלים נלקחים להמשך

אני חושב שלמדתי המון מהתהליך, כי זה בעצם טרום עבודה אקדמאית. זה נותן לי מבוא לעבודות אקדמאיות בהמשך חיי. הכלים שאני אקח להמשך זה איך לעבוד על רשתות מחשבים בשימוש מסד נתונים. לדעתי זה דבר שיעזור לי בעתיד בפרויקטים עצמיים שלי. דבר נוסף שאקח לדרך זה איך לעבוד בסביבת פרויקט גדולה עם המון חלקים, ואלמד מהטעויות שלי שלא מספיק עכבתי אחרי חוקי הפיתוח שלמדתי באופן עצמאי, clean code. עוד דבר שאקח להמשך זה ההבנה שתמיד יש עזרה בכל דבר שמתקשים בו. כמי שבדרך כלל לא נעים לו לבקש לעזרה ואוהב להסתדר לבד, בפרויקט השתמשתי בהמון עזרה ובלעדיה לא הייתי מגיע לפרויקט שיש לי עכשיו.

תובנות מהתהליך, לרבות עזרה ממומחים, שיתוף מידע ולמידת עמיתים

אני למדתי על איך שאפשר ללמוד הכל לבד, ואין באמת צורך למורה וכיתה. הקונספט של עבודה בפרויקט ושיחות אישיות עם מנחה מידי פעם הרבה יותר יעיל מאשר למידה משותפת ולדעתי גם יכול להביא לפרויקטים הרבה יותר מעניינים כי כל תלמיד יכול ללכת לנושאים שהוא רוצה ללמוד מבלי הכפייה של ללמוד ולהשתמש בנושא מסויים בפרויקט. טיפ אחד שהייתי נותן למי שעושה את הפרויקט שנה הבאה הוא לסיים אותו בתחילת שנה, שהלחץ לא התחיל. למרות שלומדים על החומר של הפרויקט בזמן זה, או לא לומדים, במילא הכל לומדים לבד. עדיף ללמוד הכל לבד ולסיים, כי זה יוצר זמן אפילו לשפר את המוצר המוגמר, מה שלא היה לי.

בראייה לאחור, האם היית מיישם אחרת חלקים בפרויקט

הייתי מיישם את החלק של ההצפנה שתהיה דו כיוונית (גם מהלקוח לשרת והשרת ללקוח) כי זה יגביר את האבטחה של הפרויקט באופן משמעותי. למרות זאת, עדיין אני מאוד מרוצה מהאבטחה של הפרויקט. הייתי יותר עוקב אחרי עקרונות הפיתוח שלמדתי באופן עצמאי. על אף כל מה שאמרתי, אני לא הייתי משנה דבר כי חלק מהלמידה זה למידה מטעויות, וזהו התהליך.

במידה והיו ברשותך משאבים נוספים, האם וכיצד ניתן לשפר את הפרויקט

הייתי במקום להסתכל על תהליכים רצים, הייתי מסתכל על תהליכים שצצים ונסגרים מהר, כי הם תהליכים מחשידים. אם היה לי יותר זמן ללמוד את זה, הייתי עושה את זה. עוד דבר שהייתי עושה זה רשימת תהליכים חסומים, כי סגירה של תהליך לא מספיקה באמת בשביל אבטחה מיטיבה. הייתי מוסף בינה מלאכותית שהייתה נותנת סיכום למה התהליכים חשודים או לא חשודים ולפי זה נותן לעובד להחליט מה לעשות.

האם הייתי מפתח פרויקט ברעיון שונה בראייה לאחור?

אני חושב שבהתחלת הכתיבה חשבתי שאם אעשה משהו יותר טכני שיש לו שימוש כמו מנהל תהליכים זה יתפס כיותר מסובך ויותר רציני. עם הזמן הבנתי שכל הפרויקטים דומים, ואין מה דווקא ללכת לנושאים שנראים יותר מסובכים כי גם אני וגם הבוחנים מבינים שהמבנה הכללי של כל פרויקט דומה, והתוספות שלי למרות שהם נשמעות מרשימות לא היו כאלו יותר מרשימות אם הייתי בוחר פרויקט בנושא שדווקא יותר מעניין אותי. הייתי רוצה לבנות תוכנה משלי עם תוכן משלי יותר למרות שגם נהנתי לעבוד על הפרויקט הזה.

תודות

אני מודה למנחה שלי, עידן פלדברג שאני רואה שאכפת לו מכל תלמיד במגמה, ומשקיע מהזמן האישי שלו שלא משלמים לו על זה לדאוג שכל תלמיד יהיה טוב. אני מודה למורים שלי: עודי רז, שלימד אותי פייתון מכיתה י', ולימד אותי ביא על המחשב באמצעות אסמבלי שדרך אגב גם עזר לי לפרויקט הזה, והשנה שלימד אותי על מערכות הפעלה. הוא עזר לי במחקר העצמי שעשיתי גם לפני הפרויקט וגם לפני מבחנים שעשיתי בנושאים חופפים לצבא. אני רוצה להודות לליאת סגל, שהיא באמת המורה עם הכי הרבה אכפתיות לכל תלמיד שיש לה, ודואגת לו עם כל הלב שלה. על איך שהיא ויתרה על יום חופשי בשביל ללמד אותנו, ולימדה באופן הכי מרוכז וענייני שיש גם נושאים שהיא לא הייתה צריכה ללמד אותנו שביקשנו. אני מודה לחברים למגמה שלי, ולחברים מבחוץ שעזרו לי לבנות את הפרויקט הקשוח הזה, כי בלעדיהם לא הייתי מצליח להגיע לזה לבד. אני מודה לאבא שלי, שעזר לי מאוד בבעיות שהיו לי בפרויקט והכווין אותי איפה שהייתי צריך.

ביבלוגרפיה

Atlas. (2023, January 1). *Converting a complex tkinter app to use customtkinter*.

YouTube. <https://www.youtube.com/watch?v=U23M379Yld8>

Breezy Codes. (2023, August 25). *Tutorial - Build your own server.py and client.py in*

Pycharm. YouTube. <https://www.youtube.com/watch?v=S4oAhXF7hcM>

Cottle, P. (n.d.). *Learn Git Branching*. Pcottle.github.io.

<https://learngitbranching.js.org/>

freeCodeCamp.org. (2025). *Intermediate Python Programming Course*. Youtu.be.

<https://youtu.be/HGOBQPFzWko?si=j2fyNNXBSzUMl4mp>

GCFLearnFree.org. (2020). Google Docs: Table of Contents. In *YouTube*.

<https://www.youtube.com/watch?v=w0ysJ1s4UZ0>

GeeksforGeeks. (2020, June 3). *Python Tkinter Tutorial*. GeeksforGeeks.

<https://www.geeksforgeeks.org/python-tkinter-tutorial/>

Lindemulder, G., & Kosinski, M. (2024, August 20). *What is role-based access*

control (RBAC)? IBM. <https://www.ibm.com/think/topics/rbac>

mental outlaw. (2025). *How RSA Encryption Works*. Youtu.be.

https://youtu.be/ZPXVSJnDA_A?si=WSVkJFc2cTnFCEggj

Patel, M. (2023, February 18). *Sleek and professional GUI with Tkinter: A*

step-by-step guide. Medium.

אורי אלפי | Proccesors

<https://medium.com/@mohit444123/sleek-and-professional-gui-with-tkinter-a-step-by-step-guide-4e9f82486380>

Programming with Mosh. (2020, September 15). *Git Tutorial for Beginners: Learn Git in 1 Hour*. Wwww.youtube.com.

<https://www.youtube.com/watch?v=8JJ101D3knE>

Rosenbaum , O., Gonen , B., & Hod, S. (2020). *רש תות מחשבים*.

<https://data.cyber.org.il/networks/networks.pdf>

Scriptopia. (2022, December 21). *Python psutil - Get running processes*. YouTube.

https://www.youtube.com/watch?v=x_PVv5eGY-c

Tech With Tim. (2022, June 25). *How To Use JSON In Python*. Wwww.youtube.com.

<https://www.youtube.com/watch?v=-51jxlQaxyA>

W3Schools. (2019). *SQL Tutorial*. W3schools.com.

<https://www.w3schools.com/sql/default.asp>

Whiteboard Crypto. (2021, May 9). *What is a Cryptographic Hashing Function? (Example + Purpose)*. Wwww.youtube.com.

<https://www.youtube.com/watch?v=gTfNtop9vzM>

נספחים

client.py

```
import socket
import threading
import sys

from numpy import byte, rec
from employee_screen import EmployeeScreen
from headers import Role, StatusCodes
from login_screen import LoginScreen
from manager_screen import ManagerScreen
from rsa import decrypt, generate_keypair

class Client:
```

```

def __init__(self, host='127.0.0.1', port=65432):
    self.host = host
    self.port = port
    self.client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    self.employee_gui = None # Separate attribute for employee GUI

def start(self):
    try:
        self.client.connect((self.host, self.port))
        print("[CONNECTED] Connected to server.")
    except Exception as e:
        print(f"[ERROR] Could not connect: {e}")
        return

    self.public_key, self.private_key = generate_keypair(8)

    self.client.sendall(f"/key {self.public_key[0]}{self.public_key[1]}".encode())

    login_gui = LoginScreen(self.client)
    threading.Thread(target=self.receive, args=([login_gui]),
daemon=True).start()
    login_result = login_gui.start() # This will now be None if closed
normally

    if login_result == StatusCodes.DISCONNECTED:
        print("[DISCONNECTED] Connection closed.")
        self.client.close()
        sys.exit()

    elif login_result == StatusCodes.SUCCESS:
        self.client.sendall("/role\n".encode())

        msg = self.receive_message(self.client)
        role = int(msg[-1])

        self.client.sendall("/username\n".encode())

        msg = self.receive_message(self.client)
        username = msg

        if role == Role.EMPLOYEE:
            self.employee_gui = EmployeeScreen(self.client, username) #
Create EmployeeScreen instance
            threading.Thread(target=self.receive,
args=([self.employee_gui]), daemon=True).start()
            self.client.sendall("/startstreaming\n".encode())
            self.employee_gui.start() # Start the EmployeeScreen's main
loop

        elif role == Role.MANAGER:
            self.manager_gui = ManagerScreen(self.client, username)
            threading.Thread(target=self.receive,
args=([self.manager_gui]), daemon=True).start()

```

```

        self.client.sendall("/startstreaming\n".encode())
        self.manager_gui.start()

def receive_message(self, conn):
    raw_msglen = self.client.recv(4)
    msglen = int.from_bytes(raw_msglen, byteorder='big')
    data = b''
    while len(data) < msglen:
        more = conn.recv(msglen - len(data))
        if not more:
            raise EOFError("Socket closed before receiving all data")
        data += more
    data = data.decode()
    ciphered = [int(char) for char in data.split(",") if char.isdigit()]
    decrypted_data = decrypt(self.private_key, ciphered)
    return decrypted_data

def receive(self, gui):
    while True:
        try:
            msg = self.receive_message(self.client)

            if getattr(gui, "alive", False):
                gui.update_data(msg.replace("[SERVER] ", ""))

            if msg.startswith("login_successful") and isinstance(gui,
LoginScreen):
                break

        except Exception as e:
            print(f"[ERROR] {str(e)}")
            if getattr(gui, "alive", False):
                gui.update_data(f"[ERROR] {str(e)}")
            print("[ERROR] Disconnected from server.")
            print("[DISCONNECTED]")
            gui.root.destroy()
            self.client.close()
            sys.exit()

if __name__ == "__main__":
    Client().start()

```

database.py

```

from enum import IntEnum
import sqlite3
import hashlib
from headers import Role, WorkerDC
import os

class Database:
    def __init__(self, db_path='database.db'):
        self.conn = sqlite3.connect(db_path, check_same_thread=False)
        self._create_table()

```



```

def _create_table(self):
    with self.conn:
        self.conn.execute("""
            CREATE TABLE IF NOT EXISTS users (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                username TEXT UNIQUE NOT NULL,
                password TEXT NOT NULL,
                salt TEXT NOT NULL,
                role INTEGER NOT NULL CHECK(role BETWEEN 0 AND 1),
                salary REAL DEFAULT 0
            )
        """)

def _hash_password(self, password):
    return hashlib.sha256(password.encode()).hexdigest()

def _generate_salt(self):
    return os.urandom(16).hex()

def add_user(self, username, password, role, salary=0.0,
is_password_hash=False):
    if not (0 <= role <= 1):
        raise ValueError("Role must be between 0 and 1.")
    try:
        if not is_password_hash:
            first_hash = self._hash_password(password)
        else:
            first_hash = password
        salt = self._generate_salt()
        with self.conn:
            self.conn.execute("""
                INSERT INTO users (username, password, salt, role, salary)
                VALUES (?, ?, ?, ?, ?)
            """, (username, self._hash_password(salt + first_hash), salt,
role, salary))
        return True
    except sqlite3.IntegrityError:
        return False # Username already exists

def del_user(self, user_id):
    with self.conn:
        self.conn.execute("DELETE FROM users WHERE id = ?", (user_id,))

def authenticate(self, username, password):
    cur = self.conn.execute("SELECT salt FROM users WHERE username = ?",
(username,))
    row = cur.fetchone()
    if not row:
        return False
    salt = row[0]
    hashed = self._hash_password(salt + password)
    cur = self.conn.execute("""

```

```

        SELECT * FROM users WHERE username = ? AND password = ?
        """ , (username, hashed))
    return cur.fetchone() is not None

def get_role(self, username):
    cur = self.conn.execute("""
        SELECT role FROM users WHERE username = ?
        """, (username,))
    row = cur.fetchone()
    return row[0] if row else None

def get_username(self, user_id):
    cur = self.conn.execute("""
        SELECT username FROM users WHERE id = ?
        """, (user_id,))
    row = cur.fetchone()
    return row[0] if row else None

def get_all_users(self):
    cur = self.conn.execute("""
        SELECT id, username, salt, role, salary FROM users
        """)
    return cur.fetchall()

def get_all_employees(self):
    cur = self.conn.execute("""
        SELECT id, username, role, salary FROM users WHERE role = ?
        """, (Role.EMPLOYEE,))
    return cur.fetchall()

def update_salary(self, user_id, salary):
    with self.conn:
        self.conn.execute("""
            UPDATE users SET salary = ? WHERE id = ?
            """, (salary, user_id))

def update_username(self, user_id, new_username):
    with self.conn:
        self.conn.execute("""
            UPDATE users SET username = ? WHERE id = ?
            """, (new_username, user_id))

def update_role(self, user_id, new_role):
    if not (0 <= new_role <= 1):
        raise ValueError("Role must be between 0 and 1.")
    with self.conn:
        self.conn.execute("""
            UPDATE users SET role = ? WHERE id = ?
            """, (new_role, user_id))

def delete_all_users(self):
    with self.conn:
        self.conn.execute("DELETE FROM users")

```

```

def close(self):
    self.conn.close()

if __name__ == "__main__":
    db = Database()
    db.add_user("Admin", "admin", Role.MANAGER, 50)
    db.add_user("User", "user", Role.EMPLOYEE, 30)

```

employee.py:

```

import threading
import time
from logs import Log
from worker import Worker
from process import Processes

class Employee(Worker):
    def __init__(self, conn, addr, username, server, key=None):
        super().__init__(conn, addr, server, key)
        self.username = username
        self.stop_streaming = threading.Event()
        self.streaming_thread = None
        self.alive = True
        self.logs = server.logs

    def handle_command(self, msg):
        self.processes = Processes()
        args = msg.split(' ')
        cmd = args[0]
        if msg.startswith('/'):
            try:
                if cmd == "/role":
                    role = self.db.get_role(self.username)
                    self.send_message(f"role:{role}")
                elif cmd == "/username":
                    self.send_message(self.username)
                elif cmd == "/startstreaming":
                    if self.streaming_thread is None or not self.streaming_thread.is_alive():
                        self.stop_streaming.clear()
                        self.streaming_thread = threading.Thread(target=self.streamer, daemon=True)
                        self.streaming_thread.start()
                elif cmd == "/terminate":
                    args = msg.split(' ', 2)
                    pid = int(args[1])
                    reason = args[2]
                    process_data = self.processes.get_process_data(pid)
                    if process_data != None:
                        self.logs.log(
                            Log(process_data["name"], process_data["cpu"],
                                process_data["memory"], process_data["priority"], reason, self.username)
                        )

```

Proccesors | אורי אלפי

```
)
    self.processes.terminate_process(pid, reason)
else:
    self.send_message("[SERVER] Unknown command emp.\n")
except ConnectionAbortedError:
    return -1
else:
    self.send_message("[SERVER] Invalid command. Commands start with
'/', use '/help' for help.\n")

def streamer(self):
    processes = Processes()
    while not self.stop_streaming.is_set():
        try:
            data = processes.get_processes_json()
            data = "PROCESS_STREAM:" + data
            self.send_message(data)
            time.sleep(5)
        except Exception as e:
            break

def close(self):
    # Signal streamer thread to stop
    self.stop_streaming.set()
    if self.streaming_thread:
        self.streaming_thread.join() # wait for thread to exit (optional)
    # Then close connection etc.
    self.conn.close()
```

employee_screen.py

```
import json
import customtkinter as ctk

class EmployeeScreen:
    def __init__(self, client_socket, username):
        ctk.set_appearance_mode("dark")
        ctk.set_default_color_theme("dark-blue")
        self.processes = {}
        self.table_widgets = {}
        self.alive = True
        self.client = client_socket

        self.root = ctk.CTk()
        self.root.iconbitmap("icon.ico")
        self.root.title("Processors Inc - Employee")
        self.root.geometry("1000x600")
        self.root.report_callback_exception = self.handle_tk_error

        # Initialize fonts AFTER the root window is created
        header_font = ctk.CTkFont(size=28, weight="bold")
        label_font = ctk.CTkFont(size=18)
        bold_font = ctk.CTkFont(size=16, weight="bold")
        self.normal_font = ctk.CTkFont(size=14)
```

```

self.frame = ctk.CTkFrame(master=self.root)
self.frame.pack(padx=20, pady=20, fill="both", expand=True)

self.header_label = ctk.CTkLabel(self.frame, text=f"Hello {username}",
font=header_font)
self.header_label.pack(pady=(10, 5))

self.role_label = ctk.CTkLabel(self.frame, text=f"You got a lot of
work today!", font=label_font)
self.role_label.pack(pady=(0, 20))

self.processes_label = ctk.CTkLabel(self.frame, text="Active Processes
on the server", font=bold_font)
self.processes_label.pack(pady=(10, 5))

# Scrollable container
self.scrollable_frame = ctk.CTkScrollableFrame(self.frame, width=760,
height=400)
self.scrollable_frame.pack(pady=10, fill="both", expand=True)

# Centered table frame inside scrollable area
self.table_frame = ctk.CTkFrame(self.scrollable_frame)
self.table_frame.pack(anchor="center")

self._create_table_header(bold_font)
self._display_table_data()

self.root.protocol("WM_DELETE_WINDOW", self.on_close)

def _create_table_header(self, font):
    headers = ["PID", "Process Name", "CPU %", "Memory", "Priority",
"Action"]

    # Configure grid columns to expand evenly
    for col in range(len(headers)):
        self.table_frame.grid_columnconfigure(col, weight=1)

    for col, text in enumerate(headers):
        header = ctk.CTkLabel(self.table_frame, text=text, font=font,
anchor="center")
        header.grid(row=0, column=col, padx=10, pady=5, sticky="nsew")

def _display_table_data(self):
    max_rows = 20
    col_widths = [80, 240, 100, 100, 100, 110]

    sorted_processes = list(self.processes.items())[:max_rows]

    while len(sorted_processes) < max_rows:
        sorted_processes.append(("", {}))

    for row_index in range(max_rows):

```

```

pid, info = sorted_processes[row_index]
values = [
    str(pid) if pid else "",
    info.get("process_name", "") if pid else "",
    str(info.get("cpu", "")) if pid else "",
    str(info.get("memory", "")) if pid else "",
    str(info.get("priority", "")) if pid else ""
]

if row_index not in self.table_widgets:
    widgets = []
    for col_index, value in enumerate(values):
        label = ctk.CTkLabel(
            self.table_frame,
            text=value,
            font=self.normal_font,
            width=col_widths[col_index]
        )
        label.grid(row=row_index + 1, column=col_index, padx=5,
pady=4, sticky="w")
        widgets.append(label)

        btn = ctk.CTkButton(
            self.table_frame,
            text="Terminate",
            width=col_widths[-1],
            command=lambda pid=pid:
self.open_terminate_reason_window(pid) if pid else None
        )
        btn.grid(row=row_index + 1, column=5, padx=5, pady=4)
        widgets.append(btn)

    self.table_widgets[row_index] = widgets
else:
    for col_index, (label, value) in
enumerate(zip(self.table_widgets[row_index][-1], values)):
        label.configure(text=value, width=col_widths[col_index])

    self.table_widgets[row_index][-1].configure(
        command=(lambda pid=pid:
self.open_terminate_reason_window(pid)) if pid else lambda: None,
        width=col_widths[-1]
    )

def open_terminate_reason_window(self, pid):
    # Popup window for terminate reason
    popup = ctk.CTkToplevel(self.root)
    popup.title(f"Terminate Process {pid}")
    popup.geometry("400x150")
    popup.grab_set() # make modal

    label = ctk.CTkLabel(popup, text="Please enter reason for terminating
the process:")

```

```

label.pack(pady=(20, 10))

reason_entry = ctk.CTkEntry(popup, width=350)
reason_entry.pack(pady=(0, 20))
reason_entry.focus()

def on_confirm():
    reason = reason_entry.get().strip()
    if reason:
        message = f"/terminate {pid} {reason}\n"
    else:
        message = f"/terminate {pid} <No reason provided>\n"
    self.client.sendall(message.encode())
    popup.destroy()

def on_cancel():
    popup.destroy()

btn_frame = ctk.CTkFrame(popup)
btn_frame.pack(pady=(0, 10))

ok_btn = ctk.CTkButton(btn_frame, text="OK", width=80,
command=on_confirm)
ok_btn.grid(row=0, column=0, padx=10)

cancel_btn = ctk.CTkButton(btn_frame, text="Cancel", width=80,
command=on_cancel)
cancel_btn.grid(row=0, column=1, padx=10)

def update_data(self, data):
    if data.startswith("PROCESS_STREAM:"):
        try:
            data = data[len("PROCESS_STREAM:")]
            self.processes = json.loads(data)
            self._display_table_data()
        except json.JSONDecodeError:
            print("[ERROR] Failed to decode JSON data.")

def start(self):
    self.root.mainloop()

def on_close(self):
    self.root.destroy()

def handle_tk_error(self, *error):
    pass

if __name__ == "__main__":
    screen = EmployeeScreen("John Doe", "Employee")
    screen.start()

from employee import Employee
from manager import Manager

```

```
from worker import Worker
from database import Role
```

guest.py

```
class Guest(Worker):
    def __init__(self, conn, addr, clients, server, key=None):
        super().__init__(conn, addr, server, key)
        self.clients = clients

    def handle_command(self, msg):
        args = msg.split(' ')
        cmd = args[0]
        if msg.startswith('/'):
            try:
                if cmd == "/login":
                    if len(args) == 3:
                        username = args[1]
                        password = args[2]
                        if self.db.authenticate(username, password):
                            self.send_message("login_successful")
                            role = self.db.get_role(username)
                            if role == Role.EMPLOYEE:
                                self.clients[self.conn] = (self.addr,
Employee(self.conn, self.addr, username, self.server, self.key))
                            elif role == Role.MANAGER:
                                self.clients[self.conn] = (self.addr,
Manager(self.conn, self.addr, username, self.server, self.key))
                            else:
                                self.send_message("invalid_creds")
                        else:
                            self.send_message("not_enough_args")
                    else:
                        self.send_message("[SERVER] Unknown command.\n")
            except ConnectionAbortedError:
                return -1
            else:
                self.send_message("[SERVER] Invalid command. Commands start with
'/', use '/help' for help.\n")
```

headers.py

```
from dataclasses import dataclass
from enum import IntEnum
```

```
class StatusCodes(IntEnum):
    SUCCESS = 0
    DISCONNECTED = 1
```

```
class Role(IntEnum):
    EMPLOYEE = 0
    MANAGER = 1
```



```
@dataclass
class WorkerDC:
    id: int
    username: str
    role: Role
    salary: int
```

login_screen.py

```
import hashlib
import os
import sys

# THIS FUNCTION SUPPRESSES THE OUTPUT FROM TKINTER
# IT IS USED TO PREVENT ERRORS FROM SHOWING UP IN THE CONSOLE
def suppress_stderr():
    # Cross-platform stderr redirect
    sys.stderr.flush()
    devnull = os.open(os.devnull, os.O_WRONLY)
    os.dup2(devnull, 2) # 2 is the file descriptor for stderr

suppress_stderr()

import customtkinter as ctk
from headers import StatusCodes

# GUI Class
class LoginScreen:
    def __init__(self, client_socket):
        ctk.set_appearance_mode("dark")
        ctk.set_default_color_theme("dark-blue")
        self.status = StatusCodes.DISCONNECTED
        self.alive = False
        self.login_successful = False # Flag to track successful login

        self.root = ctk.CTk()
        self.root.iconbitmap("icon.ico")
        self.root.title("Processors Inc - Login Screen")
        self.root.geometry("500x450")
        self.client = client_socket
        self.root.report_callback_exception = self.handle_tk_error
        self.root.interpaddr

        self.frame = ctk.CTkFrame(master=self.root)
        self.frame.pack(padx=20, pady=20, fill="both", expand=True)

        self.header_label = ctk.CTkLabel(self.frame, text="Processors",
font=ctk.CTkFont(size=50, weight="bold"))
        self.header_label.pack(pady=(20, 10))

        self.username_entry = ctk.CTkEntry(self.frame,
placeholder_text="Username", font=ctk.CTkFont(size=16), width=300)
```

```

self.username_entry.pack(pady=10)

self.password_entry = ctk.CTkEntry(self.frame,
placeholder_text="Password", show="*", font=ctk.CTkFont(size=16), width=300)
self.password_entry.pack(pady=10)

self.feedback_label = ctk.CTkLabel(self.frame, text="",
text_color="red", font=ctk.CTkFont(size=14))
self.feedback_label.pack(pady=5)

self.login_button = ctk.CTkButton(self.frame, text="Login",
font=ctk.CTkFont(size=16), width=200, command=self.login)
self.login_button.pack(pady=20)

self.root.protocol("WM_DELETE_WINDOW", self.on_close)

def start(self):
    self.alive = True
    self.root.mainloop()
    try:
        self.root.destroy()
    except:
        pass
    if self.login_successful:
        return StatusCodes.SUCCESS
    return self.status

def _hash_password(self, password):
    return hashlib.sha256(password.encode()).hexdigest()

def login(self):
    username = self.username_entry.get()
    password = self.password_entry.get()
    hashed_password = self._hash_password(password)
    message = f"/login {username} {hashed_password}\n"
    try:
        self.client.sendall(message.encode())
    except:
        self.update_data("Failed to send login request.")

def update_data(self, msg):
    if not self.alive or not self.root.winfo_exists():
        return
    try:
        def update_ui():
            if not self.alive or not self.root.winfo_exists():
                return

            if msg == "invalid_creds":
                self.feedback_label.configure(text="Invalid username or
password.")
            elif msg == "not_enough_args":

```

Proccesors | אורי אלפי

```
        self.feedback_label.configure(text="Not enough arguments
provided.")

        elif msg == "login_successful":
            self.status = StatusCodes.SUCCESS
            self.login_successful = True
            self.alive = False
            self.root.quit() # Stop the mainloop safely
        else:
            self.feedback_label.configure(text=msg)

        try:
            update_ui()
        except Exception as e:
            print(f"[GUI ERROR] {e}")
    except Exception as e:
        print(f"[GUI ERROR] {e}")

def on_close(self):
    self.alive = False
    try:
        self.root.quit() # Safely stop the mainloop
    except:
        pass
    try:
        self.client.close()
    except:
        pass

def handle_tk_error(self, *error):
    pass
```

logs.py

```
from datetime import datetime
import threading

class Log:
    def __init__(self, process_name, cpu, memory, priotiy, reason, user):
        self.process_name = process_name
        self.cpu = cpu
        self.memory = memory
        self.priority = priotiy
        self.reason = reason
        self.user = user
        self.time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    def to_string(self):
        return
        f"LOG:{self.process_name}\n{self.cpu}\n{self.memory}\n{self.priority}\n{self.
reason}\n{self.user}\n{self.time}"

class Logs:
```

```
def __init__(self, server):
    self.server = server
    self.logs: list[Log] = []
    self.lock = threading.Lock()

def log(self, log: Log):
    with self.lock:
        self.logs.append(log)
        self.broadcast_log(log)

def broadcast_log(self, log: Log):
    self.server.broadcast_managers(log.to_string())
```

manager.py

```
import hashlib
import threading
import time
from guest import Employee
from headers import WorkerDC
from worker import Worker
import json

class Manager(Worker):
    def __init__(self, conn, addr, username, server, key=None):
        super().__init__(conn, addr, server, key)
        self.username = username
        self.logs = server.logs
        self.stop_streaming = threading.Event()
        self.streaming_thread = None
        self.alive = True

    def _hash_password(self, password):
        return hashlib.sha256(password.encode()).hexdigest()

    def handle_command(self, msg):
        args = msg.split(' ')
        cmd = args[0]
        if msg.startswith('/'):
            try:
                if cmd == "/role":
                    role = self.db.get_role(self.username)
                    self.send_message(f"role:{role}")
                elif cmd == "/username":
                    self.send_message(self.username)
                elif cmd == "/fire":
                    user_id = int(args[1])
                    username = self.db.get_username(user_id)
                    self.db.del_user(user_id)
                    clients = [client[1] for client in
self.server.clients.values()]
                    for client in clients:
                        if isinstance(client, Manager) or isinstance(client,
Employee):
```

```

        if client.username == username:
            client.close()
    elif cmd == "/salary":
        user_id = int(args[1])
        salary = float(args[2])
        self.db.update_salary(user_id, salary)
    elif cmd == "/change_username":
        user_id = int(args[1])
        new_username = args[2]
        self.db.update_username(user_id, new_username)
    elif cmd == "/change_role":
        user_id = int(args[1])
        new_role = int(args[2])
        if new_role not in [0, 1]:
            self.send_message("[SERVER] Invalid role. Use 0 for
EMPLOYEE and 1 for MANAGER.\n")
            return
        self.db.update_role(user_id, new_role)
    elif cmd == "/add_user":
        username = args[1]
        password = args[2]
        hashed_password = self._hash_password(password)
        role = int(args[3])
        if role not in [0, 1]:
            self.send_message("[SERVER] Invalid role. Use 0 for
EMPLOYEE and 1 for MANAGER.\n")
            return
        salary = float(args[4])
        self.db.add_user(username, hashed_password, role, salary,
is_password_hash=True)
    elif cmd == "/startstreaming":
        if self.streaming_thread is None or not
self.streaming_thread.is_alive():
            self.stop_streaming.clear()
            self.streaming_thread =
threading.Thread(target=self.streamer, daemon=True)
            self.streaming_thread.start()
        else:
            self.send_message("[SERVER] Unknown command.\n")
    except ConnectionAbortedError:
        return -1
    else:
        self.send_message("[SERVER] Invalid command. Commands start with
'/', use '/help' for help.\n")

    def streamer(self):
        while not self.stop_streaming.is_set():
            try:
                employees = self.db.get_all_employees()
                employees_json = {ID: WorkerDC(ID, username, role, salary) for
ID, username, role, salary in employees}
                employees_json = json.dumps(employees_json, default=lambda o:
o.__dict__)

```

```

        data = "PROCESS_STREAM:" + employees_json
        self.send_message(data)
        print(self.server.clients)
        time.sleep(5)
    except Exception as e:
        break

def close(self):
    # Signal streamer thread to stop
    self.stop_streaming.set()
    if self.streaming_thread:
        self.streaming_thread.join() # wait for thread to exit (optional)
    # Then close connection etc.
    self.conn.close()

```

manager_screen.py

```

import json
from math import log
import time
import customtkinter as ctk
import tkinter as tk
from PIL import Image
from numpy import pad

from headers import Role, WorkerDC

ctk.set_appearance_mode("Dark")
ctk.set_default_color_theme("blue")

class ManagerScreen:
    def __init__(self, client_socket=None, username=None):
        self.root = ctk.CTk()
        self.root.iconbitmap("icon.ico")
        self.root.title("Processors Inc - Manager")
        self.root.geometry("1000x600")
        self.root.resizable(False, False)
        self.client = client_socket
        self.username = username
        self.alive = True
        self._log_initialized = False

        self.root.report_callback_exception = self.handle_tk_error

        self._workers_data = {} # Store the underlying worker data
        self._worker_widgets = {} # Store references to the UI elements for
each worker

        self.add_worker_window = None
        self.confirmation_window = None
        self.edit_worker_window = None

        self.build_ui()
        self.populate_worker_list() # Initial population

```

```

def start(self):
    self.root.mainloop()

def build_ui(self):
    main_frame = ctk.CTkFrame(self.root)
    main_frame.pack(fill="both", expand=True, padx=20, pady=20)

    # LEFT PANEL
    left_panel = ctk.CTkFrame(main_frame, width=400)
    left_panel.pack(side="left", fill="y", padx=(0, 20))

    ctk.CTkLabel(left_panel, text="Search Worker:",
font=ctk.CTkFont(size=14)).pack(anchor="w", padx=10, pady=(10, 2))

    self.search_var = tk.StringVar()
    self.search_entry = ctk.CTkEntry(left_panel,
textvariable=self.search_var)
    self.search_entry.pack(fill="x", padx=10)
    self.search_entry.bind("<KeyRelease>", self.update_worker_list)

    self.worker_scroll = ctk.CTkScrollableFrame(left_panel, height=400)
    self.worker_scroll.pack(fill="both", expand=True, padx=10, pady=10)

    add_worker_btn = ctk.CTkButton(left_panel, text="Add a Worker",
command=self._open_add_worker_window_handler)
    add_worker_btn.pack(pady=(10, 20), padx=10)

    # RIGHT PANEL
    right_panel = ctk.CTkFrame(main_frame)
    right_panel.pack(side="right", fill="both", expand=True)

    ctk.CTkLabel(right_panel, text="Deleted Process Log",
font=ctk.CTkFont(size=16, weight="bold")).pack(anchor="w", padx=10, pady=10)

    self.log_scroll = ctk.CTkScrollableFrame(right_panel)
    self.log_scroll.pack(fill="both", expand=True, padx=10, pady=10)

def populate_worker_list(self, filter_text=""):
    for worker_id, worker in self._workers_data.items():
        if filter_text.lower() in worker.username.lower():
            if worker_id not in self._worker_widgets:
                self._create_worker_widget(worker)
            else:
                self._update_worker_widget(worker)
        elif worker_id in self._worker_widgets:
            # Worker no longer matches the filter, remove its widgets
            for widget in self._worker_widgets[worker_id].values():
                widget.destroy()
            del self._worker_widgets[worker_id]

    # Remove widgets for workers that are no longer in the data
    worker_ids_to_remove = [

```

```

        w_id for w_id in self._worker_widgets if w_id not in
self._workers_data
    ]
    for w_id in worker_ids_to_remove:
        for widget in self._worker_widgets[w_id].values():
            widget.destroy()
        del self._worker_widgets[w_id]

    def _create_worker_widget(self, worker: WorkerDC):
        frame = ctk.CTkFrame(self.worker_scroll, corner_radius=6,
border_width=1)
        frame.pack(fill="x", pady=5, padx=5)

        header_frame = ctk.CTkFrame(frame)
        header_frame.pack(fill="x", padx=10, pady=5)

        username_salary_label = ctk.CTkLabel(
            header_frame,
            text=f"{worker.username} - Salary: ${worker.salary}",
            anchor="w"
        )
        username_salary_label.pack(side="left", fill="x", expand=True)

        settings_img = ctk.CTkImage(
            light_image=Image.open("settings.png"),
            dark_image=Image.open("settings.png"),
            size=(12, 12)
        )
        edit_btn = ctk.CTkButton(
            header_frame,
            image=settings_img,
            text="", # Hide text
            width=32,
            height=24,
            fg_color="transparent", # Optional: makes background blend in
            command=lambda w_id=worker.id:
self._open_edit_worker_window_handler(w_id),
        )
        edit_btn.pack(side="right")

        btn_frame = ctk.CTkFrame(frame)
        btn_frame.pack(fill="x", padx=10, pady=(0, 10))

        raise_btn = ctk.CTkButton(
            btn_frame,
            text="Give a Raise",
            width=160,
            command=lambda w_id=worker.id:
self._open_confirmation_raise_handler(w_id)
        )
        raise_btn.pack(pady=2)

        lower_btn = ctk.CTkButton(

```



```

        btn_frame,
        text="Lower Salary",
        width=160,
        command=lambda w_id=worker.id:
self._open_confirmation_lower_handler(w_id)
    )
    lower_btn.pack(pady=2)

    fire_btn = ctk.CTkButton(
        btn_frame,
        text="Fire",
        fg_color="red",
        hover_color="#cc0000",
        width=160,
        command=lambda w_id=worker.id:
self._open_confirmation_fire_handler(w_id)
    )
    fire_btn.pack(pady=2)

    self._worker_widgets[worker.id] = {
        "frame": frame,
        "header_frame": header_frame,
        "username_salary_label": username_salary_label,
        "edit_btn": edit_btn,
        "btn_frame": btn_frame,
        "raise_btn": raise_btn,
        "lower_btn": lower_btn,
        "fire_btn": fire_btn,
    }

def _update_worker_widget(self, worker: WorkerDC):
    if worker.id in self._worker_widgets:
        widgets = self._worker_widgets[worker.id]

widgets["username_salary_label"].configure(text=f"{worker.username} - Salary:
${worker.salary}")

def update_worker_list(self, event=None):
    self.populate_worker_list(self.search_var.get())

def _open_confirmation_raise_handler(self, worker_id):
    worker = self._workers_data.get(worker_id)
    if worker:
        self.open_confirmation_window(
            title="Confirm Raise",
            message=f"Are you sure you want to give a raise to
{worker.username}?",
            on_confirm=lambda: self._apply_raise(worker_id)
        )

def _apply_raise(self, worker_id):
    if worker_id in self._workers_data:

```

```

        self.client.sendall(f"/salary {worker_id}
{self._workers_data[worker_id].salary + 10}\n".encode())
        self._workers_data[worker_id].salary += 10
        self._update_worker_widget(self._workers_data[worker_id])

    def _open_confirmation_lower_handler(self, worker_id):
        worker = self._workers_data.get(worker_id)
        if worker:
            self.open_confirmation_window(
                title="Confirm Salary Decrease",
                message=f"Are you sure you want to lower {worker.username}'s
salary?",
                on_confirm=lambda: self._apply_lower(worker_id)
            )

    def _apply_lower(self, worker_id):
        if worker_id in self._workers_data:
            if self._workers_data[worker_id].salary > 10:
                self.client.sendall(f"/salary {worker_id}
{self._workers_data[worker_id].salary - 10}\n".encode())
                self._workers_data[worker_id].salary -= 10 # Or your logic
                self._update_worker_widget(self._workers_data[worker_id])

    def _open_confirmation_fire_handler(self, worker_id):
        worker = self._workers_data.get(worker_id)
        if worker:
            self.open_confirmation_window(
                title="Confirm Termination",
                message=f"Are you sure you want to fire {worker.username}?",
                on_confirm=lambda: self._confirm_fire(worker_id)
            )

    def _confirm_fire(self, worker_id):
        if worker_id in self._workers_data:
            if worker_id in self._worker_widgets:
                for widget in self._worker_widgets[worker_id].values():
                    widget.destroy()
                self.client.sendall(f"/fire {worker_id}\n".encode())
                del self._worker_widgets[worker_id]
            del self._workers_data[worker_id]

    def add_log_entry(self, process, cpu, memory, priority, reason, worker,
time):
        frame = ctk.CTkFrame(self.log_scroll, corner_radius=8, border_width=1)
        frame.pack(fill="x", pady=5, padx=5) # Pack the new frame first

        if self._log_initialized:
            # Repack to be the first widget
            frame.pack_configure(before=self.log_scroll.winfo_children()[0])
        else:
            self._log_initialized = True

        text = (

```

```

        f"Process: {process}\n"
        f"CPU Usage: {cpu}      Memory: {memory}      Priority: {priority}\n"
        f"Reason Deleted: {reason}\n"
        f"Deleted by: {worker} \n"
        f"Time: {time}\n"
    )
    label = ctk.CTkLabel(frame, text=text, justify="left", anchor="w")
    label.pack(fill="x", padx=10, pady=5)

    def _open_add_worker_window_handler(self):
        if self.add_worker_window is None or not
self.add_worker_window.winfo_exists():
            self.open_add_worker_window()
        else:
            self.add_worker_window.focus()

    def open_add_worker_window(self):
        self.add_worker_window = ctk.CTkToplevel(self.root)
        self.add_worker_window.title("Add New Worker")
        self.add_worker_window.geometry("450x400")
        self.add_worker_window.resizable(False, False)
        self.add_worker_window.transient(self.root)
        self.add_worker_window.grab_set()
        self.add_worker_window.focus_set()

        ctk.CTkLabel(
            self.add_worker_window,
            text="Enter username, password, role, and salary for new worker",
            font=ctk.CTkFont(size=14, weight="bold")
        ).pack(pady=(20, 10))

        ctk.CTkLabel(self.add_worker_window,
text="Username:").pack(anchor="w", padx=20, pady=(10, 2))
        username_entry = ctk.CTkEntry(self.add_worker_window)
        username_entry.pack(fill="x", padx=20)

        ctk.CTkLabel(self.add_worker_window,
text="Password:").pack(anchor="w", padx=20, pady=(10, 2))
        password_entry = ctk.CTkEntry(self.add_worker_window, show="*")
        password_entry.pack(fill="x", padx=20)

        ctk.CTkLabel(self.add_worker_window, text="Role:").pack(anchor="w",
padx=20, pady=(10, 2))
        role_var = tk.StringVar(value="Employee")
        role_menu = ctk.CTkOptionMenu(self.add_worker_window,
variable=role_var, values=["Employee", "Manager"])
        role_menu.pack(fill="x", padx=20)

        ctk.CTkLabel(self.add_worker_window, text="Salary:").pack(anchor="w",
padx=20, pady=(10, 2))
        salary_entry = ctk.CTkEntry(self.add_worker_window)
        salary_entry.pack(fill="x", padx=20)

```

```

def on_submit():
    username = username_entry.get().strip()
    password = password_entry.get().strip()
    role = Role.MANAGER if role_var.get() == "Manager" else
Role.EMPLOYEE

    try:
        salary = int(salary_entry.get().strip())
    except ValueError:
        salary = 0

    if not username or salary <= 0:
        # You can add a warning here
        return

    self.client.sendall(f"/add_user {username} {password} {role}
{salary}\n".encode())

    # You probably want to handle password saving securely somewhere
else

    self.populate_worker_list(self.search_var.get())
    self.add_worker_window.destroy()
    self.add_worker_window = None

    submit_btn = ctk.CTkButton(self.add_worker_window, text="Submit",
width=100, command=on_submit)
    submit_btn.pack(pady=20)

    def on_close():
        self.add_worker_window.destroy()
        self.add_worker_window = None

    self.add_worker_window.protocol("WM_DELETE_WINDOW", on_close)

    def open_confirmation_window(self, title, message, on_confirm):
        if self.confirmation_window is None or not
self.confirmation_window.winfo_exists():
            self.confirmation_window = ctk.CTkToplevel(self.root)
            self.confirmation_window.title(title)
            self.confirmation_window.geometry("300x150")
            self.confirmation_window.resizable(False, False)
            self.confirmation_window.transient(self.root)
            self.confirmation_window.grab_set()
            self.confirmation_window.focus_set()

            label = ctk.CTkLabel(self.confirmation_window, text=message,
wraplength=280, justify="center")
            label.pack(pady=(20, 20), padx=10)

            btn_frame = ctk.CTkFrame(self.confirmation_window)
            btn_frame.pack(pady=10)

```

```

def confirm():
    on_confirm()
    self.confirmation_window.destroy()
    self.confirmation_window = None

def cancel():
    self.confirmation_window.destroy()
    self.confirmation_window = None

    ctk.CTkButton(btn_frame, text="Yes", width=80,
command=confirm).pack(side="left", padx=10)
    ctk.CTkButton(btn_frame, text="No", width=80,
command=cancel).pack(side="right", padx=10)

def on_close():
    self.confirmation_window.destroy()
    self.confirmation_window = None

    self.confirmation_window.protocol("WM_DELETE_WINDOW", on_close)
else:
    self.confirmation_window.focus()

def _open_edit_worker_window_handler(self, worker_id):
    worker = self._workers_data.get(worker_id)
    if worker:
        if self.edit_worker_window is None or not
self.edit_worker_window.winfo_exists():
            self.open_edit_worker_window(worker)
        else:
            self.edit_worker_window.focus()

def open_edit_worker_window(self, worker: WorkerDC):
    self.edit_worker_window = ctk.CTkToplevel(self.root)
    self.edit_worker_window.title(f"Edit Worker - {worker.username}")
    self.edit_worker_window.geometry("350x320")
    self.edit_worker_window.resizable(False, False)
    self.edit_worker_window.transient(self.root)
    self.edit_worker_window.grab_set()
    self.edit_worker_window.focus_set()

    ctk.CTkLabel(self.edit_worker_window, text=f"Editing
{worker.username}", font=ctk.CTkFont(size=16, weight="bold")).pack(pady=15)

    ctk.CTkLabel(self.edit_worker_window,
text="Username:").pack(anchor="w", padx=20, pady=(10, 2))
    username_entry = ctk.CTkEntry(self.edit_worker_window)
    username_entry.insert(0, worker.username)
    username_entry.pack(fill="x", padx=20)

    ctk.CTkLabel(self.edit_worker_window, text="Role:").pack(anchor="w",
padx=20, pady=(10, 2))

```

```

        role_var = tk.StringVar(value="Manager" if worker.role == Role.MANAGER
else "Employee")
        role_menu = ctk.CTkOptionMenu(self.edit_worker_window,
variable=role_var, values=["Employee", "Manager"])
        role_menu.pack(fill="x", padx=20)

        ctk.CTkLabel(self.edit_worker_window, text="Salary:").pack(anchor="w",
padx=20, pady=(10, 2))
        salary_entry = ctk.CTkEntry(self.edit_worker_window)
        salary_entry.insert(0, str(worker.salary))
        salary_entry.pack(fill="x", padx=20)

    def save_changes():
        new_username = username_entry.get().strip()
        new_role = Role.MANAGER if role_var.get() == "Manager" else
Role.EMPLOYEE

        try:
            new_salary = float(salary_entry.get().strip())
        except ValueError:
            new_salary = worker.salary # fallback

        if not new_username or new_salary <= 0:
            return

        # Update worker info
        self.client.sendall(f"/salary {worker.id}
{new_salary}\n".encode())
        self.client.sendall(f"/change_username {worker.id}
{new_username}\n".encode())
        self.client.sendall(f"/change_role {worker.id}
{new_role}\n".encode())
        self._workers_data[worker.id] = WorkerDC(id=worker.id,
username=new_username, role=new_role, salary=new_salary)
        self._update_worker_widget(self._workers_data[worker.id])
        self.edit_worker_window.destroy()
        self.edit_worker_window = None

        save_btn = ctk.CTkButton(self.edit_worker_window, text="Save Changes",
command=save_changes)
        save_btn.pack(pady=20)

    def on_close():
        self.edit_worker_window.destroy()
        self.edit_worker_window = None

    self.edit_worker_window.protocol("WM_DELETE_WINDOW", on_close)

def update_data(self, data):
    if data.startswith("PROCESS_STREAM:"):
        try:
            data = data[len("PROCESS_STREAM:")]
            new_workers_data = json.loads(data)

```

```

        new_workers_data = {int(k): WorkerDC(**v) for k, v in
new_workers_data.items()}

        updated = False
        for worker_id, new_worker in new_workers_data.items():
            if worker_id not in self._workers_data or
self._workers_data[worker_id] != new_worker:
                self._workers_data[worker_id] = new_worker
                updated = True
                if worker_id in self._worker_widgets:
                    self._update_worker_widget(new_worker)
                else:
                    self._create_worker_widget(new_worker)

        # Remove workers that are no longer in the new data
        workers_to_remove = [
            w_id for w_id in self._workers_data if w_id not in
new_workers_data
        ]
        for w_id in workers_to_remove:
            if w_id in self._worker_widgets:
                for widget in self._worker_widgets[w_id].values():
                    widget.destroy()
                del self._worker_widgets[w_id]
            del self._workers_data[w_id]
            updated = True

        if updated:
            self.populate_worker_list(self.search_var.get()) #
Re-apply filter

    except json.JSONDecodeError:
        print("[ERROR] Failed to decode JSON data.")
    elif data.startswith("LOG:"):
        try:
            log_data = data[len("LOG:"):]
            log_data = log_data.split("\n")
            self.add_log_entry(
                process=log_data[0],
                cpu=log_data[1],
                memory=log_data[2],
                priority=log_data[3],
                reason=log_data[4],
                worker=log_data[5],
                time=log_data[6]
            )
        except Exception as e:
            print(f"[ERROR] Failed to decode log data: {e}")

    def handle_tk_error(self, *error):
        pass

if __name__ == "__main__":

```

```
app = ManagerScreen()
app.start()
```

process.py

```
import json
import psutil
import time
import platform

class Processes:
    def __init__(self):
        pass

    def get_processes_json(self):
        # Prime the CPU percent cache
        for proc in psutil.process_iter():
            try:
                proc.cpu_percent(interval=None)
            except (psutil.NoSuchProcess, psutil.AccessDenied):
                continue

        time.sleep(0.1) # Allow CPU measurement window

        process_info = {}

        logical_cpu_count = psutil.cpu_count(logical=True)

        for proc in psutil.process_iter(['pid', 'name', 'memory_info']):
            try:
                pid = proc.pid
                name = proc.name()
                raw_cpu = proc.cpu_percent(interval=None)
                cpu = round(raw_cpu / logical_cpu_count, 2)
                memory = round(proc.memory_info().rss / (1024 * 1024), 2) #
                MB

                # Priority handling
                try:
                    if platform.system() == 'Windows':
                        priority = proc.nice() # still numeric but
                        corresponds to priority class
                    else:
                        priority = proc.nice() # Unix-style nice value
                except psutil.AccessDenied:
                    priority = "N/A"

                process_info[pid] = {
                    "pid": pid,
                    "process_name": name,
                    "cpu": cpu,
                    "memory": memory,
                    "priority": priority
                }
            except:
```



```

        except (psutil.NoSuchProcess, psutil.AccessDenied,
psutil.ZombieProcess):
            continue

        # Sort by memory usage and keep top 50
        del process_info[0]
        process_info = {pid: info for pid, info in process_info.items() if
info["process_name"] != ""}
        top_processes = sorted(process_info.values(), key=lambda x: x["cpu"],
reverse=True)[:15]

        return json.dumps({
            p["pid"]: {
                "process_name": p["process_name"],
                "cpu": p["cpu"],
                "memory": p["memory"],
                "priority": p["priority"]
            } for p in top_processes
        })

    def terminate_process(self, pid, reason):
        try:
            proc = psutil.Process(pid)
            proc.terminate()
            print(f"[INFO] Terminated PID {pid} | Reason: {reason}")
        except psutil.NoSuchProcess:
            print(f"[ERROR] No such process with PID {pid} | Reason:
{reason}")
        except psutil.AccessDenied:
            print(f"[ERROR] Access denied when terminating PID {pid} | Reason:
{reason}")
        except Exception as e:
            print(f"[ERROR] Failed to terminate PID {pid} | Reason: {reason} |
Exception: {e}")

    def get_process_data(self, pid):
        try:
            proc = psutil.Process(pid)
            return {
                "name": proc.name(),
                "memory": round(proc.memory_info().rss / (1024 * 1024), 2), #
MB
                "cpu": proc.cpu_percent(interval=None),
                "priority": proc.nice()
            }
        except psutil.NoSuchProcess:
            return None
        except psutil.AccessDenied:
            return None
        except Exception as e:
            print(f"[ERROR] Failed to get data for PID {pid} | Exception:
{e}")

```

```
return None
```

rsa.py

```
import random

def is_prime(num):
    if num == 2:
        return True
    if num < 2 or num % 2 == 0:
        return False
    for n in range(3, int(num**0.5)+2, 2):
        if num % n == 0:
            return False
    return True

def generate_prime(bits):
    while True:
        randnum = random.getrandbits(bits)
        if is_prime(randnum):
            return randnum

def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

# function for extended Euclidean Algorithm
def multiplicative_inverse(e, phi):
    d = 0
    x1 = 0
    x2 = 1
    y1 = 1
    temp_phi = phi

    while e > 0:
        temp1 = temp_phi // e
        temp2 = temp_phi - temp1 * e
        temp_phi = e
        e = temp2

        x = x2 - temp1 * x1
        y = d - temp1 * y1

        x2 = x1
        x1 = x
        d = y1
        y1 = y

    if temp_phi == 1:
        return d + phi

    return None # added to handle the case where inverse doesn't exist

def generate_keypair(bits):
```

Proccesors | אורי אלפי

```
#Generate p and q
p = generate_prime(bits)
q = generate_prime(bits)

if not (is_prime(p) and is_prime(q)):
    raise ValueError('Both numbers must be prime.')
elif p == q:
    raise ValueError('p and q cannot be equal')
#n = pq
n = p * q

#Phi is the totient of n
phi = (p-1) * (q-1)

#Choose an integer e such that e and phi(n) are coprime
e = random.randrange(2, phi)

#Use Euclid's Algorithm to verify that e and phi(n) are coprime
g = gcd(e, phi)
while g != 1:
    e = random.randrange(2, phi)
    g = gcd(e, phi)
#Use Extended Euclid's Algorithm to generate the private key
d = multiplicative_inverse(e, phi)
if d is None:
    raise ValueError("e and phi(n) are not coprime")

#Return public and private keypair
#Public key is (e, n) and private key is (d, n)
return (e, n), (d, n)

def encrypt(pk, plaintext):
    key, n = pk

    #Convert each letter in the plaintext to numbers based on the character
    #using a^b mod m
    cipher = [(ord(char) ** key) % n for char in plaintext]

    #Return the array of bytes as a string
    return ','.join(map(lambda x: str(x), cipher))

def decrypt(pk, ciphertext):
    #Unpack the key into its components
    key, n = pk
    #Generate the plaintext based on the ciphertext and key using a^b mod m
    plain = [chr((char ** key) % n) for char in ciphertext]
    #Return the array of bytes as a string
    return ''.join(plain)
```

server.py

```
import socket
import threading

from guest import Guest
```

```

from logs import Logs
from manager import Manager

class Server:
    def __init__(self, host='127.0.0.1', port=65432): # Just a usable port
        self.host = host
        self.port = port
        self.clients = {}
        self.server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.server.bind((self.host, self.port))
        self.server.listen()
        self.logs = Logs(self)
        self.max_clients = 5

    def start(self):
        print(f"[SERVER STARTED] Listening on {self.host}:{self.port}")
        while True:

            conn, addr = self.server.accept()

            if len(self.clients) >= self.max_clients:
                print(f"[WARNING] Connection from {addr} rejected: server
full.")
                conn.sendall(b"[SERVER] Connection rejected: too many
users.\n")
                conn.close()
                continue

            threading.Thread(target=self.handle_client, args=(conn, addr),
                            daemon=True).start() # daemon=True makes the
thread exit when the main program exits

    def handle_client(self, conn, addr):
        address = f"{addr[0]}:{addr[1]}"
        self.clients[conn] = (address, Guest(conn, addr, self.clients, self))
        print(f"[NEW CONNECTION] {address}")

        try:
            while True:
                msg = conn.recv(1024)
                decoded = msg.decode().strip()
                commands = decoded.split("\n")
                for command in commands:
                    if command == "":
                        continue
                    elif command.startswith("/key"):
                        args = command.split(" ")
                        if len(args) == 3: # Ensure we have both parts of the
key

                            try:
                                key_e = int(args[1])
                                key_n = int(args[2])

```

```

        self.clients[conn][1].key = (key_e, key_n)
    except ValueError:
        print(f"[ERROR] Invalid key format from
{address}")

        self.send_message(conn, "[SERVER] ERROR:
Invalid key format.")

        # Consider disconnecting the client here if
the key is crucial

    else:
        print(f"[ERROR] Incomplete key from {address}")
        self.send_message(conn, "[SERVER] ERROR:
Incomplete key. Send as /key e n")
        elif command == "/quit":
            conn.sendall(b"[SERVER] Bye!")
            raise ConnectionAbortedError
        else:
            status = self.clients[conn][1].handle_command(command)
            if status == -1:
                break

except Exception as e:
    print(f"[ERROR] {address} - {e}")
finally:
    self.disconnect(conn)

def broadcast(self, message, sender_conn):
    for client in list(self.clients):
        if client != sender_conn:
            try:
                self.send_message(client, message)
            except:
                self.disconnect(client)

def broadcast_managers(self, message):
    for client in list(self.clients):
        try:
            if isinstance(self.clients[client][1], Manager):
                self.clients[client][1].send_message(message)
        except Exception as e:
            print(f"[ERROR] {self.clients[client]} - {e}")
            self.disconnect(client)

def disconnect(self, conn):
    if conn in self.clients:
        print(f"[DISCONNECT] {self.clients[conn]}")
        self.clients[conn][1].close()
        del self.clients[conn]

def send_message(self, conn, message):
    print("Message:" + message)
    try:
        encoded = message.encode()
        length = len(encoded)

```

Proccesors | אורי אלפי

```
conn.sendall(length.to_bytes(4, byteorder='big'))
conn.sendall(encoded)
except Exception as e:
    if isinstance(e, (ConnectionResetError, BrokenPipeError,
ConnectionAbortedError)):
        print(f"[ERROR] Connection lost with {self.clients[conn]} -
Disconnecting.")
        self.disconnect(conn)
    else:
        print(f"[ERROR] Failed to send message to {self.clients[conn]}
- {e}")

if __name__ == "__main__":
    Server().start()
```

worker.py

```
from database import Database
from rsa import encrypt

class Worker:
    def __init__(self, conn, addr, server, key=None):
        self.server = server
        self.conn = conn
        self.addr = addr
        self.key = key
        self.db = Database()

    def send_message(self, message):
        encrypted_message = encrypt(self.key, message)
        self.server.send_message(self.conn, encrypted_message)

    def handle_command(self, command):
        return "Worker cannot handle commands."

    def close(self):
        self.conn.close()
```

icon.ico



settings.png

